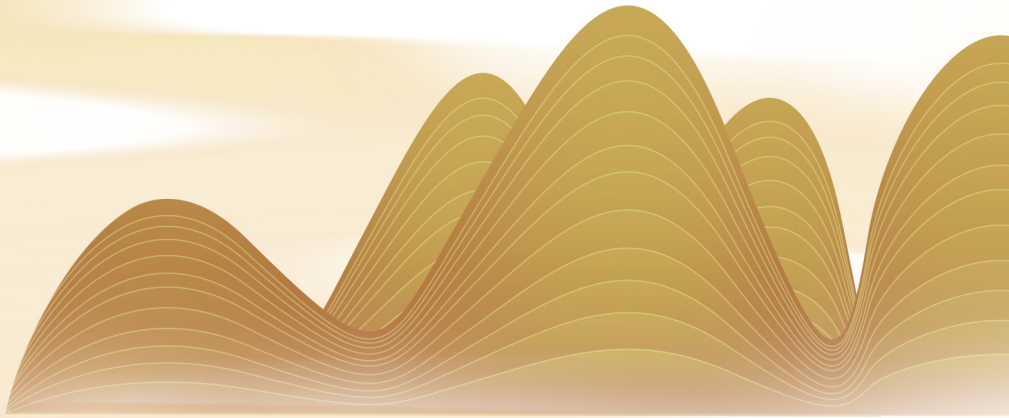
阿里云开发者社区
ALIBABA CLOUD DEVELOPER COMMUNITY

Java 单元测试实战

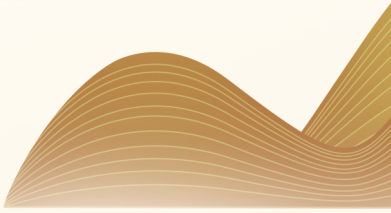
一位阿里程序员的单测之道

作 者 陈昌毅（常意）



卜算子·致码农

业务要精通，眼界需开**阔**。
谈笑风生略几回，方案心头**过**。
编写守规则，测试知结**果**。
奋笔疾书数万行，代码无一**错**。



阿里云开发者“藏经阁”
海量电子手册免费下载

卷首语

《卜算子·致码农》

作者：常意

业务要精通，眼界需开阔。
谈笑风生略几回，方案心头过。
编写守规则，测试知结果。
奋笔疾书数万行，代码无一错。

一、 什么是单元测试

维基百科是这样描述的：

在计算机编程中，单元测试又称为模块测试，是针对程序模块来进行正确性检验的测试工作。程序单元是应用的最小可测试部件。在过程化编程中，一个单元就是单个程序、函数、过程等；对于面向对象编程，最小单元就是方法，包括基类、抽象类、或者派生类中的方法。

作者认为单元测试分为**狭义单元测试**和**广义单元测试**：

1. 狭义单元测试

狭义单元测试是针对一个类的一个公有、私有、保护方法的测试。

2. 广义单元测试

广义单元测试是针对一个类的一个公有方法的测试。首先，对于类的使用者来说，大多数情况下只能调用公有方法；其次，公有方法可能有私有、保护方法组成，独立存在的私有、保护方法是没有意义的；最后，在测试公有方法时，已经附带着测试私有、保护方法。

在日常的单元测试中，大多使用广义单元测试，针对公有方法进行测试。当然，为了覆盖某些代码分支，也可以采用狭义单元测试，直接针对私有、保护方法进行测试。

二、 单元测试的意义

1. 验证代码逻辑

对于一个代码模块，编写单元测试的过程，就是对代码逻辑进行重新 Review 的过程；而执行单元测试的过程，就是验证代码是否按既定逻辑运行的过程。

2. 减少代码缺陷

我们的工程都是分层分模块的，每个模块都是独立的逻辑部分。通过单元测试保障工程各个“零件”按“规格”（需求）执行，就能保证整个“机器”（项目）运行正确，最大限度地减少 bug。

3. 促进代码设计

在编写单测的过程中，如果发现单测代码非常难写，一般表明被测试的代码包含了太多的依赖或职责，需要反思代码的合理性，进而促进代码设计的优化。

4. 便于多人协作

在多人协助的项目中，所依赖的服务接口不一定已经开发完毕，导致服务进行联调工作。此时，单元测试有效地解决了这个问题——只需 Mock 服务接口数据，便可以完成自己代码的测试。

5. 便于缺陷定位

由于单元规模较小，复杂性较低，因而发现错误后容易隔离和定位，有利于代码调试工作。

6. 放心代码重构

如今持续型的项目越来越多，代码不断的在变化和重构，通过单元测试，开发可以放心的修改重构代码，减少改代码时的心理负担，提高重构的成功率。

7. 增强代码信心

写完代码，单元测试通过，虽说单元测试并不能百分之百保证代码完全正确运行，但起码大部分测过的逻辑都是可用的,这会增强我们的信心,也会增加工作成就感。

三、单元测试的推广

很多软件研发人员认为：进行单元测试会花费大量时间来编写测试用例。因此，他们做单元测试的意愿比较低，导致单元测试很难推广开来。

首先，要解决意识问题。需要让研发人员意识到单元测试的作用，比如：验证代码逻辑、减少代码缺陷、促进代码设计、增强代码信心等。有时候，只有出现了线上问题、产生了资损和客诉，才能真正地意识到单元测试的作用了。

再次，要解决认知问题。在单元测试的推广过程中，遇到的最大问题就是——很多人把集成测试当成单元测试。因为有时候，集成测试也具备“解除依赖”、"安全重构"、“增强信心”等功能，所以很容易与单元测试混淆。只有在弄清楚集成测试和单元测试的区别后，才能更好地做好单元测试。

第三，要解决方法问题。在本书中，提供了大量的单元测试实践方法，可以从根本上解决这个问题。比如：《Java 编程技巧之单元测试用例编写流程》解决了编写方法问题，《Java 单元测试技巧之 JSON 序列化》解决了编写技巧问题，《Java 编程技巧之单元测试用例简化方法》解决了简化技巧问题，《Java 单元测试典型案例集锦》解决了编写案例问题。

最后，要解决态度问题。自上而下地推广单元测试，大多数人会把单元测试当成一个任务和累赘,没有体会到单元测试的真实作用,编写单测只是为了应付上级任务，甚至于为了追求覆盖率而编写无效单元测试。这里，《那些年，我们写过的无效单元测试》介绍了如何识别无效单元测试,并提出了有效单元测试所需要遵循的原则。

总之，单元测试的推广是一个长期而艰巨的任务。

常意
2023 年春
北京

目录

基础知识：Java 单元测试技巧之 PowerMock 7

编写方法：Java 编程技巧之单元测试用例编写流程 32

编写技巧：Java 单元测试技巧之 JSON 序列化..... 85

简化技巧：Java 编程技巧之单元测试用例简化方法 123

无效单测：那些年，我们写过的无效单元测试 138

案例集锦：Java 单元测试典型案例集锦..... 179

单测认证：单元测试认证问题、答案和解析..... 237

基础知识：Java 单元测试技巧之 PowerMock

前言

高德的技术大佬向老师在谈论方法论时说到：“复杂的问题要简单化，简单的问题要深入化。”

这句话让我感触颇深，这何尝不是一套编写代码的方法——把一个复杂逻辑拆分为许多简单逻辑，然后把每一个简单逻辑进行深入实现，最后把这些简单逻辑整合为复杂逻辑，总结为八字真言即是“化繁为简，由简入繁”。

编写 Java 单元测试用例，其实就是把“复杂的问题要简单化”——即把一段复杂的代码拆解成一系列简单的单元测试用例；写好 Java 单元测试用例，其实就是把“简单的问题要深入化”——即学习一套方法、总结一套模式并应用到实践中。这里，作者根据日常的工作经验，总结了一些 Java 单元测试技巧，以供大家交流和学习。

一、 准备环境

PowerMock 是一个扩展了其它如 EasyMock 等 mock 框架的、功能更加强大的框架。PowerMock 使用一个自定义类加载器和字节码操作来模拟静态方法、构造方法、final 类和方法、私有方法、去除静态初始化器等等。

1. 引入 PowerMock 包

为了引入 PowerMock 包，需要在 pom.xml 文件中加入下列 maven 依赖：

```
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-module-junit4</artifactId>
    <version>2.0.9</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-api-mockito2</artifactId>
    <version>2.0.9</version>
    <scope>test</scope>
</dependency>
```

2. 集成 SpringMVC 项目

在 SpringMVC 项目中，需要在 pom.xml 文件中加入 JUnit 的 maven 依赖：

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.12</version>
  <scope>test</scope>
</dependency>
```

3. 集成 SpringBoot 项目

在 SpringBoot 项目中，需要在 pom.xml 文件中加入 JUnit 的 maven 依赖：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

4. 一个简单的测试用例

这里，用 List 举例，模拟一个不存在的列表，但是返回的列表大小为 100。

```
public class ListTest {
    @Test
    public void testSize() {
        Integer expected = 100;
        List list = PowerMockito.mock(List.class);
        PowerMockito.when(list.size()).thenReturn(expected);
        Integer actual = list.size();
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

二、 mock 语句

1. mock 方法

声明： `T PowerMockito.mock(Class<T> clazz);`

用途：可以用于模拟指定类的对象实例。

当模拟非 final 类（接口、普通类、虚基类）的非 final 方法时，不必使用`@RunWith`和`@PrepareForTest`注解。当模拟 final 类或 final 方法时，必须使用`@RunWith`和`@PrepareForTest`注解。

注解形如：

```
@RunWith(PowerMockRunner.class)
@PrepareForTest({TargetClass.class})
```

1） 模拟非 final 类普通方法

```
@Getter
@Setter
@ToString
public class Rectangle implements Sharp {
    private double width;
    private double height;
    @Override
    public double getArea() {
        return width * height;
    }
}

public class RectangleTest {
    @Test
    public void testGetArea() {
        double expectArea = 100.0D;
        Rectangle rectangle = PowerMockito.mock(Rectangle.class);
        PowerMockito.when(rectangle.getArea()).thenReturn(expectArea);
        double actualArea = rectangle.getArea();
        Assert.assertEquals("返回值不相等", expectArea, actualArea, 1E-6D);
    }
}
```

2) 模拟 final 类或 final 方法

```
@Getter
@Setter
@ToString
public final class Circle {
    private double radius;
    public double getArea() {
        return Math.PI * Math.pow(radius, 2);
    }
}

@RunWith(PowerMockRunner.class)
@PrepareForTest({Circle.class})
public class CircleTest {
    @Test
    public void testGetArea() {
        double expectArea = 3.14D;
        Circle circle = PowerMockito.mock(Circle.class);
        PowerMockito.when(circle.getArea()).thenReturn(expectArea);
        double actualArea = circle.getArea();
        Assert.assertEquals("返回值不相等", expectArea, actualArea, 1E-6D);
    }
}
```

2. mockStatic 方法

声明： `PowerMockito.mockStatic(Class<T> clazz);`

用途： 可以用于模拟类的静态方法，必须使`@RunWith`和`@PrepareForTest`注解。

```
@RunWith(PowerMockRunner.class)
@PrepareForTest({StringUtils.class})
public class StringUtilsTest {
    @Test
    public void testIsEmpty() {
        String string = "abc";
        boolean expected = true;
        PowerMockito.mockStatic(StringUtils.class);
        PowerMockito.when(StringUtils.isEmpty(string)).thenReturn(expected);
        boolean actual = StringUtils.isEmpty(string);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

三、 spy 语句

如果一个对象，我们只希望模拟它的部分方法，而希望其它方法跟原来一样，可以使用`PowerMockito.spy`方法代替`PowerMockito.mock`方法。于是，通过 when

语句设置过的方法，调用的是模拟方法；而没有通过 when 语句设置的方法，调用的是原有方法。

1. spy 类

声明： `PowerMockito.spy(Class<T> clazz);`

用途： 用于模拟类的部分方法。

案例：

```
public class StringUtils {
    public static boolean isNotEmpty(final CharSequence cs) {
        return !isEmpty(cs);
    }
    public static boolean isEmpty(final CharSequence cs) {
        return cs == null || cs.length() == 0;
    }
}

@RunWith(PowerMockRunner.class)
@PrepareForTest({StringUtils.class})
public class StringUtilsTest {
    @Test
    public void testIsNotEmpty() {
        String string = null;
        boolean expected = true;
        PowerMockito.spy(StringUtils.class);
        PowerMockito.when(StringUtils.isEmpty(string)).thenReturn(!expected);
        boolean actual = StringUtils.isNotEmpty(string);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

2. spy 对象

声明： `T PowerMockito.spy(T object);`

用途： 用于模拟对象的部分方法。

案例：

```
public class UserService {
    private Long superUserId;
    public boolean isNotSuperUser(Long userId) {
        return !isSuperUser(userId);
    }
    public boolean isSuperUser(Long userId) {
        return Objects.equals(userId, superUserId);
    }
}

@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Test
    public void testIsNotSuperUser() {
        Long userId = 1L;
        boolean expected = false;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.when(userService.isSuperUser(userId)).thenReturn(!expected);
        boolean actual = userService.isNotSuperUser(userId);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

四、 when 语句

1. when().thenReturn()模式

声明：

```
PowerMockito.when(mockObject.someMethod(someArgs)).thenReturn(expectedValue);

PowerMockito.when(mockObject.someMethod(someArgs)).thenThrow(expectedThrowable);

PowerMockito.when(mockObject.someMethod(someArgs)).thenAnswer(expectedAnswer);

PowerMockito.when(mockObject.someMethod(someArgs)).thenCallRealMethod();
```

用途：用于模拟对象方法，先执行原始方法，再返回期望的值、异常、应答，或调用真实的方法。

1) 返回期望值

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 0;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.when(mockList.get(index)).thenReturn(expected);
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

2) 返回期望异常

```
public class ListTest {
    @Test(expected = IndexOutOfBoundsException.class)
    public void testGet() {
        int index = -1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.when(mockList.get(index)).thenThrow(new IndexOutOfBoundsException());
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

3) 返回期望应答

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.when(mockList.get(index)).thenAnswer(invocation -> {
            Integer value = invocation.getArgument(0);
            return value * 100;
        });
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

4) 调用真实方法

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 0;
        Integer expected = 100;
        List<Integer> oldList = new ArrayList<>();
        oldList.add(expected);
        List<Integer> spylist = PowerMockito.spy(oldList);
        PowerMockito.when(spylist.get(index)).thenCallRealMethod();
        Integer actual = spylist.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

2. doReturn().when()模式

声明：

```
PowerMockito.doReturn(expectedValue).when(mockObject).someMethod(someArgs);

PowerMockito.doThrow(expectedThrowable).when(mockObject).someMethod(someArgs);

PowerMockito.doAnswer(expectedAnswer).when(mockObject).someMethod(someArgs);

PowerMockito.doNothing().when(mockObject).someMethod(someArgs);

PowerMockito.doCallRealMethod().when(mockObject).someMethod(someArgs);
```

用途：用用于模拟对象方法，直接返回期望的值、异常、应答，或调用真实的方法，无需执行原始方法。

注意：千万不要使用以下语法，虽然不会出现编译错误，但是在执行时会抛出 UnfinishedStubbingException 异常。

```
PowerMockito.doReturn(expectedValue).when(mockObject.someMethod(someArgs));

PowerMockito.doThrow(expectedThrowable).when(mockObject.someMethod(someArgs));

PowerMockito.doAnswer(expectedAnswer).when(mockObject.someMethod(someArgs));
```

```
PowerMockito.doNothing().when(mockObject.someMethod(someArgs));

PowerMockito.doCallRealMethod().when(mockObject.someMethod(someArgs));
```

1) 返回期望值

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 0;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doReturn(expected).when(mockList).get(index);
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

2) 返回期望异常

```
public class ListTest {
    @Test(expected = IndexOutOfBoundsException.class)
    public void testGet() {
        int index = -1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doThrow(new IndexOutOfBoundsException()).when(mockList).get(index);
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

3) 返回期望应答

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doAnswer(invocation -> {
            Integer value = invocation.getArgument(0);
            return value * 100;
        }).when(mockList).get(index);
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

4) 模拟无返回值

```
public class ListTest {
    @Test
    public void testClear() {
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doNothing().when(mockList).clear();
        mockList.clear();
        Mockito.verify(mockList).clear();
    }
}
```

5) 调用真实方法

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 0;
        Integer expected = 100;
        List<Integer> oldList = new ArrayList<>();
        oldList.add(expected);
        List<Integer> spylist = PowerMockito.spy(oldList);
        PowerMockito.doCallRealMethod().when(spylist).get(index);
        Integer actual = spylist.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

3. 两种模式的主要区别

两种模式都用于模拟对象方法，在 mock 实例下使用时，基本上是没有差别的。但是，在 spy 实例下使用时，when().thenReturn()模式会执行原方法，而 doReturn().when()模式不会执行原方法。

测试服务类：

```
@Slf4j
@Service
public class UserService {
    public long getUserCount() {
        log.info("调用获取用户数量方法");
        return 0L;
    }
}
```

使用 when().thenReturn()模式：

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Test
    public void testGetUserCount() {
        Long expected = 1000L;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.when(userService.getUserCount()).thenReturn(expected);
        Long actual = userService.getUserCount();
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

在测试过程中，将会打印出“调用获取用户数量方法”日志。

使用 doReturn().when()模式：

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Test
    public void testGetUserCount() {
        Long expected = 1000L;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.doReturn(expected).when(userService).getUserCount();
        Long actual = userService.getUserCount();
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

在测试过程中，不会打印出“调用获取用户数量方法”日志。

4. whenNew 模拟构造方法

声明：

```
PowerMockito.whenNew(MockClass.class).withNoArguments().thenReturn(expectedObject);

PowerMockito.whenNew(MockClass.class).withArguments(someArgs).thenReturn(expectedObject);
```

用途：用于模拟构造方法。

案例：

```
public final class FileUtils {
    public static boolean isFile(String fileName) {
        return new File(fileName).isFile();
    }
}

@RunWith(PowerMockRunner.class)
@PrepareForTest({FileUtils.class})
public class FileUtilsTest {
    @Test
    public void testIsFile() throws Exception {
        String fileName = "test.txt";
        File file = PowerMockito.mock(File.class);
        PowerMockito.whenNew(File.class).withArguments(fileName).thenReturn(file);
        PowerMockito.when(file.isFile()).thenReturn(true);
        Assert.assertTrue("返回值为假", FileUtils.isFile(fileName));
    }
}
```

注意：需要加上注解`@PrepareForTest({FileUtils.class})`，否则模拟方法不生效。

五、 参数匹配器

在执行单元测试时，有时候并不关心传入的参数的值，可以使用参数匹配器。

1. 参数匹配器（any）

Mockito 提供 `Mockito.anyInt()`、`Mockito.anyString`、`Mockito.any(Class<T> clazz)`等来表示任意值。

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.when(mockList.get(Mockito.anyInt())).thenReturn(expected);
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

2. 参数匹配器（eq）

当我们使用参数匹配器时，所有参数都应使用匹配器。 如果要为某一参数指定特定值时，就需要使用 `Mockito.eq()` 方法。

```
@RunWith(PowerMockRunner.class)
@PrepareForTest({StringUtils.class})
public class StringUtilsTest {
    @Test
    public void testStartWith() {
        String string = "abc";
        String prefix = "b";
        boolean expected = true;
        PowerMockito.spy(StringUtils.class);
        PowerMockito.when(StringUtils.startsWith(Mockito.anyString(),
Mockito.eq(prefix))).thenReturn(expected);
        boolean actual = StringUtils.startsWith(string, prefix);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

3. 附加匹配器

Mockito 的 AdditionalMatchers 类提供了一些很少使用的参数匹配器，我们可以进行参数大于（gt）、小于（lt）、大于等于（geq）、小于等于（leq）等比较操作，也可以进行参数与（and）、或（or）、非（not）等逻辑计算等。

```
public class ListTest {
    @Test
    public void testGet() {
        int index = 1;
        Integer expected = 100;
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.when(mockList.get(AdditionalMatchers.geq(0))).thenReturn(expected);
        PowerMockito.when(mockList.get(AdditionalMatchers.lt(0))).thenThrow(new
IndexOutOfBoundsException());
        Integer actual = mockList.get(index);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

六、 verify 语句

验证是确认在模拟过程中，被测试方法是否已按预期方式与其任何依赖方法进行了交互。

格式：

```
Mockito.verify(mockObject[,times(int)]).someMethod(somgArgs
);
```

用途：用于模拟对象方法，直接返回期望的值、异常、应答，或调用真实的方法，无需执行原始方法。

案例：

1. 验证调用方法

```
public class ListTest {
    @Test
    public void testGet() {
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doNothing().when(mockList).clear();
        mockList.clear();
        Mockito.verify(mockList).clear();
    }
}
```

2. 验证调用次数

```
public class ListTest {
    @Test
    public void testGet() {
        List<Integer> mockList = PowerMockito.mock(List.class);
        PowerMockito.doNothing().when(mockList).clear();
        mockList.clear();
        Mockito.verify(mockList, Mockito.times(1)).clear();
    }
}
```

除 times 外，Mockito 还支持 atLeastOnce、atLeast、only、atMostOnce、atMost 等次数验证器。

3. 验证调用顺序

```
public class ListTest {
    @Test
    public void testAdd() {
        List<Integer> mockedList = PowerMockito.mock(List.class);
        PowerMockito.doReturn(true).when(mockedList).add(Mockito.anyInt());
        mockedList.add(1);
        mockedList.add(2);
        mockedList.add(3);
        InOrder inOrder = Mockito.inOrder(mockedList);
        inOrder.verify(mockedList).add(1);
        inOrder.verify(mockedList).add(2);
        inOrder.verify(mockedList).add(3);
    }
}
```

4. 验证调用参数

```
public class ListTest {
    @Test
    public void testArgumentCaptor() {
        Integer[] expecteds = new Integer[] {1, 2, 3};
        List<Integer> mockedList = PowerMockito.mock(List.class);
        PowerMockito.doReturn(true).when(mockedList).add(Mockito.anyInt());
        for (Integer expected : expecteds) {
            mockedList.add(expected);
        }
        ArgumentCaptor<Integer> argumentCaptor = ArgumentCaptor.forClass(Integer.class);
        Mockito.verify(mockedList, Mockito.times(3)).add(argumentCaptor.capture());
        Integer[] actuals = argumentCaptor.getAllValues().toArray(new Integer[0]);
        Assert.assertArrayEquals("返回值不相等", expecteds, actuals);
    }
}
```

5. 确保验证完毕

Mockito 提供 `Mockito.verifyNoMoreInteractions` 方法，在所有验证方法之后可以使用此方法，以确保所有调用都得到验证。如果模拟对象上存在任何未验证的调用，将会抛出 `NoInteractionsWanted` 异常。

```
public class ListTest {
    @Test
    public void testVerifyNoMoreInteractions() {
        List<Integer> mockedList = PowerMockito.mock(List.class);
        Mockito.verifyNoMoreInteractions(mockedList); // 执行正常
        mockedList.isEmpty();
        Mockito.verifyNoMoreInteractions(mockedList); // 抛出异常
    }
}
```

备注：

`Mockito.verifyZeroInteractions` 方法与 `Mockito.verifyNoMoreInteractions` 方法相同，但是目前已经被废弃。

6. 验证静态方法

Mockito 没有静态方法的验证方法，但是 PowerMock 提供这方面的支持。

```
@RunWith(PowerMockRunner.class)
@PrepareForTest({StringUtils.class})
public class StringUtilsTest {
    @Test
    public void testVerifyStatic() {
        PowerMockito.mockStatic(StringUtils.class);
        String expected = "abc";
        StringUtils.isEmpty(expected);
        PowerMockito.verifyStatic(StringUtils.class);
        ArgumentCaptor<String> argumentCaptor = ArgumentCaptor.forClass(String.class);
        StringUtils.isEmpty(argumentCaptor.capture());
        Assert.assertEquals("参数不相等", argumentCaptor.getValue(), expected);
    }
}
```

七、 私有属性

1. ReflectionTestUtils.setField 方法

在用原生 JUnit 进行单元测试时，我们一般采用 `ReflectionTestUtils.setField` 方法设置私有属性值。

注意：在测试类中，UserService 实例是通过@Autowired 注解加载的，如果该实例已经被动态代理，`ReflectionTestUtils.setField` 方法设置的是代理实例，从而导致设置不生效。

```
@Service
public class UserService {
    @Value("${system.userLimit}")
    private Long userLimit;
    public Long getUserLimit() {
        return userLimit;
    }
}

public class UserServiceTest {
    @Autowired
    private UserService userService;
    @Test
    public void testGetUserLimit() {
        Long expected = 1000L;
        ReflectionTestUtils.setField(userService, "userLimit", expected);
        Long actual = userService.getUserLimit();
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

2. Whitebox.setInternalState 方法

现在使用 PowerMock 进行单元测试时，可以采用 `Whitebox.setInternalState` 方法设置私有属性值。

```
@Service
public class UserService {
    @Value("${system.userLimit}")
    private Long userLimit;
    public Long getUserLimit() {
        return userLimit;
    }
}

@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @InjectMocks
    private UserService userService;
    @Test
    public void testGetUserLimit() {
        Long expected = 1000L;
        Whitebox.setInternalState(userService, "userLimit", expected);
        Long actual = userService.getUserLimit();
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

注意：需要加上注解`@RunWith(PowerMockRunner.class)`。

八、 私有方法

1. 模拟私有方法

1) 通过 when 实现

```
public class UserService {
    private Long superUserId;
    public boolean isNotSuperUser(Long userId) {
        return !isSuperUser(userId);
    }
    private boolean isSuperUser(Long userId) {
        return Objects.equals(userId, superUserId);
    }
}

@RunWith(PowerMockRunner.class)
@PrepareForTest({UserService.class})
public class UserServiceTest {
    @Test
    public void testIsNotSuperUser() throws Exception {
        Long userId = 1L;
        boolean expected = false;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.when(userService, "isSuperUser", userId).thenReturn(!expected);
        boolean actual = userService.isNotSuperUser(userId);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

2) 通过 stub 实现

通过模拟方法 stub（存根），也可以实现模拟私有方法。但是，只能模拟整个方法的返回值，而不能模拟指定参数的返回值。

```
@RunWith(PowerMockRunner.class)
@PrepareForTest({UserService.class})
public class UserServiceTest {
    @Test
    public void testIsNotSuperUser() throws Exception {
        Long userId = 1L;
        boolean expected = false;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.stub(PowerMockito.method(UserService.class, "isSuperUser",
Long.class)).toReturn(!expected);
        boolean actual = userService.isNotSuperUser(userId);
        Assert.assertEquals("返回值不相等", expected, actual;
    }
}
```

2. 测试私有方法

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest9 {
    @Test
    public void testIsSuperUser() throws Exception {
        Long userId = 1L;
        boolean expected = false;
        UserService userService = new UserService();
        Method method = PowerMockito.method(UserService.class, "isSuperUser", Long.class);
        Object actual = method.invoke(userService, userId);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

3. 验证私有方法

```
@RunWith(PowerMockRunner.class)
@PrepareForTest({UserService.class})
public class UserServiceTest10 {
    @Test
    public void testIsNotSuperUser() throws Exception {
        Long userId = 1L;
        boolean expected = false;
        UserService userService = PowerMockito.spy(new UserService());
        PowerMockito.when(userService, "isSuperUser", userId).thenReturn(!expected);
        boolean actual = userService.isNotSuperUser(userId);
        PowerMockito.verifyPrivate(userService).invoke("isSuperUser", userId);
        Assert.assertEquals("返回值不相等", expected, actual);
    }
}
```

这里，也可以用 Method 那套方法进行模拟和验证方法。

九、 主要注解

PowerMock 为了更好地支持 SpringMVC/SpringBoot 项目，提供了一系列的注解，大大地简化了测试代码。

1. @RunWith 注解

`@RunWith(PowerMockRunner.class)`

指定 JUnit 使用 PowerMock 框架中的单元测试运行器。

2. @PrepareForTest 注解

```
@PrepareForTest({ TargetClass.class })
```

当需要模拟 final 类、final 方法或静态方法时，需要添加@PrepareForTest 注解，并指定方法所在的类。如果需要指定多个类，在{}中添加多个类并用逗号隔开即可。

3. @Mock 注解

@Mock 注解创建了一个全部 Mock 的实例，所有属性和方法全被置空（0 或者 null）。

4. @Spy 注解

@Spy 注解创建了一个没有 Mock 的实例，所有成员方法都会按照原方法的逻辑执行，直到被 Mock 返回某个具体的值为止。

注意：@Spy 注解的变量需要被初始化，否则执行时会抛出异常。

5. @InjectMocks 注解

@InjectMocks 注解创建一个实例，这个实例可以调用真实代码的方法，其余用@Mock 或@Spy 注解创建的实例将被注入到用该实例中。

```
@Service
public class UserService {
    @Autowired
    private UserDAO userDAO;
    public void modifyUser(UserVO userVO) {
        UserDO userDO = new UserDO();
        BeanUtils.copyProperties(userVO, userDO);
        userDAO.modify(userDO);
    }
}

@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Mock
    private UserDAO userDAO;
    @InjectMocks
    private UserService userService;
    @Test
    public void testCreateUser() {
        UserVO userVO = new UserVO();
        userVO.setId(1L);
        userVO.setName("changyi");
        userVO.setDesc("test user");
        userService.modifyUser(userVO);
        ArgumentCaptor<UserDO> argumentCaptor = ArgumentCaptor.forClass(UserDO.class);
        Mockito.verify(userDAO).modify(argumentCaptor.capture());
        UserDO userDO = argumentCaptor.getValue();
        Assert.assertNotNull("用户实例为空", userDO);
        Assert.assertEquals("用户标识不相等", userVO.getId(), userDO.getId());
        Assert.assertEquals("用户名称不相等", userVO.getName(), userDO.getName());
        Assert.assertEquals("用户描述不相等", userVO.getDesc(), userDO.getDesc());
    }
}
```

6. @Captor 注解

@Captor 注解在字段级别创建参数捕获器。但是，在测试方法启动前，必须调用`MockitoAnnotations.openMocks(this)`进行初始化。

```
@Service
public class UserService {
    @Autowired
    private UserDAO userDAO;
    public void modifyUser(UserVO userVO) {
        UserDO userDO = new UserDO();
        BeanUtils.copyProperties(userVO, userDO);
        userDAO.modify(userDO);
    }
}

@RunWith(PowerMockRunner.class)
```

```
public class UserServiceTest {
    @Mock
    private UserDAO userDAO;
    @InjectMocks
    private UserService userService;
    @Captor
    private ArgumentCaptor<UserDO> argumentCaptor;
    @Before
    public void beforeTest() {
        MockitoAnnotations.openMocks(this);
    }
    @Test
    public void testCreateUser() {
        UserVO userVO = new UserVO();
        userVO.setId(1L);
        userVO.setName("changyi");
        userVO.setDesc("test user");
        userService.modifyUser(userVO);
        Mockito.verify(userDAO).modify(argumentCaptor.capture());
        UserDO userDO = argumentCaptor.getValue();
        Assert.assertNotNull("用户实例为空", userDO);
        Assert.assertEquals("用户标识不相等", userVO.getId(),
userDO.getId());
        Assert.assertEquals("用户名称不相等", userVO.getName(),
userDO.getName());
        Assert.assertEquals("用户描述不相等", userVO.getDesc(),
userDO.getDesc());
    }
}
```

7. @PowerMockIgnore 注解

为了解决使用 PowerMock 后，提示 ClassLoader 错误。

十、 相关观点

1. 《Java 开发手册》规范

【强制】好的单元测试必须遵守 AIR 原则。

说明：单元测试在线上运行时，感觉像空气（AIR）一样感觉不到，但在测试质量的保障上，却是非常关键的。好的单元测试宏观上来说，具有自动化、独立性、可重复执行的特点。

- A：Automatic（自动化）
- I：Independent（独立性）
- R：Repeatable（可重复）

【强制】单元测试应该是全自动执行的，并且非交互式的。测试用例通常是被定期执行的，执行过程必须完全自动化才有意义。输出结果需要人工检查的测试不是一个好的单元测试。单元测试中不准使用 System.out 来进行人肉验证，必须使用 assert 来验证。

【强制】单元测试是可以重复执行的，不能受到外界环境的影响。

说明：单元测试通常会被放到持续集成中，每次有代码 check in 时单元测试都会被执行。如果单测对外部环境（网络、服务、中间件等）有依赖，容易导致持续集成机制的不可用。

正例：为了不受外界环境影响，要求设计代码时就把 SUT 的依赖改成注入，在测试时用 spring 这样的 DI 框架注入一个本地（内存）实现或者 Mock 实现。

【推荐】编写单元测试代码遵守 BCDE 原则，以保证被测试模块的交付质量。

- B：Border，边界值测试，包括循环边界、特殊取值、特殊时间点、数据顺序等。
- C：Correct，正确的输入，并得到预期的结果。
- D：Design，与设计文档相结合，来编写单元测试。
- E：Error，强制错误信息输入（如：非法数据、异常流程、业务允许外等），并得到预期的结果。

2. 为什么要使用 Mock?

根据网络相关资料，总结观点如下：

1) Mock 可以用来解除外部服务依赖，从而保证了测试用例的独立性。

现在的互联网软件系统，通常采用了分布式部署的微服务，为了单元测试某一服务而准备其它服务，存在极大的依耐性和不可行性。

2) Mock 可以减少全链路测试数据准备，从而提高了编写测试用例的速度。

传统的集成测试，需要准备全链路的测试数据，可能某些环节并不是你所熟悉的。最后，耗费了大量的时间和经历，并不一定得到你想要的结果。现在的单元测试，只需要模拟上游的输入数据，并验证给下游的输出数据，编写测试用例并进行测试的速度可以提高很多倍。

3) Mock 可以模拟一些非正常的流程，从而保证了测试用例的代码覆盖率。

根据单元测试的 BCDE 原则，需要进行边界值测试（Border）和强制错误信息输入（Error），这样有助于覆盖整个代码逻辑。在实际系统中，很难去构造这些边界值，也能难去触发这些错误信息。而 Mock 从根本上解决了这个问题：想要什么样的边界值，只需要进行 Mock；想要什么样的错误信息，也只需要进行 Mock。

4) Mock 可以不用加载项目环境配置，从而保证了测试用例的执行速度。

在进行集成测试时，我们需要加载项目的所有环境配置，启动项目依赖的所有服务接口。往往执行一个测试用例，需要几分钟乃至几十分钟。采用 Mock 实现的测试用例，不用加载项目环境配置，也不依赖其它服务接口，执行速度往往在几秒之内，大大地提高了单元测试的执行速度。

3. 单元测试与集成测试的区别

在实际工作中，不少同学用集成测试代替了单元测试，或者认为集成测试就是单元测试。这里，总结为了单元测试与集成测试的区别：

1) 测试对象不同

单元测试对象是实现了具体功能的程序单元，集成测试对象是概要设计规划中的模块及模块间的组合。

2) 测试方法不同

单元测试中的主要方法是基于代码的白盒测试，集成测试中主要使用基于功能的黑盒测试。

3) 测试时间不同

集成测试要晚于单元测试。

4) 测试内容不同

单元测试主要是模块内程序的逻辑、功能、参数传递、变量引用、出错处理及需求和设计中具体要求方面的测试；而集成测试主要验证各个接口、接口之间的数据传递关系，及模块组合后能否达到预期效果。

编写方法：Java 编程技巧之单元测试用例编写流程

前言

清代杰出思想家章学诚有一句名言：“学必求其心得，业必贵其专精。”

意思是：学习上一定要追求心得体会，事业上一定要贵以专注精深。做技术就是这样，一件事如果做到了极致，就必然会有所心得体会。作者最近在一个项目上，追求单元测试覆盖率到极致，所以才有了这篇心得体会。

上一篇文章《Java 单元测试技巧之 PowerMock》除了介绍单元测试基础知识外，主要介绍了“为什么要编写单元测试”。很多同学读完后，还是不能快速地编写单元测试用例。而这篇文章，立足于“如何来编写单元测试用例”，能够让同学们“有章可循”，能快速地编写出单元测试用例。

一、 编写单元测试用例

1. 测试框架简介

Mockito 是一个单元测试模拟框架，可以让你写出优雅、简洁的单元测试代码。Mockito 采用了模拟技术，模拟了一些在应用中依赖的复杂对象，从而把测试对象和依赖对象隔离开来。

PowerMock 是一个单元测试模拟框架，是在其它单元测试模拟框架的基础上做出扩展。通过提供定制的类加载器以及一些字节码篡改技术的应用，PowerMock 实现了对静态方法、构造方法、私有方法以及 final 方法的模拟支持等强大的功能。但是，正因为 PowerMock 进行了字节码篡改，导致部分单元测试用例并不被 JaCoco 统计覆盖率。

通过作者多年单元测试的编写经验，优先推荐使用 Mockito 提供的功能；只有在 Mockito 提供的功能不能满足需求时，才会采用 PowerMock 提供的功能；但是，不推荐使用影响 JaCoco 统计覆盖率的 PowerMock 功能。在本文中，我们也不会对影响 JaCoco 统计覆盖率的 PowerMock 功能进行介绍。

下面，将以 Mockito 为主、以 PowerMock 为辅，介绍一下如何编写单元测试用例。

2. 测试框架引入

为了引入 Mockito 和 PowerMock 包，需要在 maven 项目的 pom.xml 文件中加入以下包依赖：

```
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-core</artifactId>
    <version>${powermock.version}</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-api-mockito2</artifactId>
    <version>${powermock.version}</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-module-junit4</artifactId>
    <version>${powermock.version}</version>
    <scope>test</scope>
</dependency>
```

其中，powermock.version 为 2.0.9，为当前的最新版本，可根据实际情况修改。在 PowerMock 包中，已经包含了对应的 Mockito 和 JUnit 包，所以无需单独引入 Mockito 和 JUnit 包。

3. 单测测试案例

1) 一个最简单的单元测试

一个最简单的单元测试，只需要一大步骤：

调用方法：传入参数对象、调用测试方法、验证返回值或异常。

案例代码：

```
public final class NumberHelper {
    public static final int INT_ZERO = 0;
    public static boolean isPositive(Integer value) {
        return Objects.nonNull(value) && Integer.compare(value, INT_ZERO) > 0;
    }
    ...
}
```

测试用例：

```
public class NumberHelperTest {
    @Test
    public void testIsPositiveInt() {
        Assert.assertFalse("返回值不为假", NumberHelper.isPositive((Integer)null));
        Assert.assertFalse("返回值不为假", NumberHelper.isPositive(Integer.valueOf(-1)));
        Assert.assertFalse("返回值不为假", NumberHelper.isPositive(Integer.valueOf(0)));
        Assert.assertTrue("返回值不为真", NumberHelper.isPositive(Integer.valueOf(1)));
    }
    ...
}
```

2) 一个无依赖的单元测试

一个无依赖的单元测试，只需要两大步骤：

- **定义对象**：定义测试对象；
- **调用方法**：传入参数对象、调用测试方法、验证返回值或异常。

案例代码：

```
public class Rectangle implements Shape {
    private final double width;
    private final double height;
    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }
    @Override
    public double getArea() {
        return width * height;
    }
    ...
}
```

测试用例：

```
public class RectangleTest {
    @Test
    public void testGetArea() {
        Rectangle rectangle = new Rectangle(3.0D, 4.0D);
        Assert.assertEquals("面积不一致", 12.0D, rectangle.getArea(), 1E-6D);
    }
    ...
}
```

3) 一个有依赖的单元测试

一个有依赖的单元测试，需要四大步骤：

- **定义对象：**定义测试对象、模拟依赖对象、注入依赖对象；
- **模拟方法：**模拟参数或返回值、模拟依赖方法；
- **调用方法：**传入参数对象、调用测试方法、验证返回值或异常；
- **验证方法：**验证依赖方法、验证方法参数、验证依赖对象。

案例代码：

```
/**
 * 用户服务类
 */
@Service
public class UserService {
    /** 定义依赖对象 */
    /** 用户 DAO */
    @Autowired
    private UserDAO userDAO;
    /** 标识生成器 */
    @Autowired
    private IdGenerator idGenerator;

    /** 定义依赖参数 */
    /** 可以修改 */
    @Value("${userService.canModify}")
    private Boolean canModify;

    /**
     * 保存用户
     *
```

```
     * @param userSave 用户保存
     * @return 用户标识
     */
    public Long saveUser(UserVO userSave) {
        // 获取用户标识
        Long userId = userDAO.getIdByName(userSave.getName());

        // 根据存在处理
        // 根据存在处理：不存在则创建
        if (Objects.isNull(userId)) {
            userId = idGenerator.next();
            UserDO userCreate = new UserDO();
            userCreate.setId(userId);
            userCreate.setName(userSave.getName());
            userCreate.setDescription(userSave.getDescription());
            userDAO.create(userCreate);
        }
        // 根据存在处理：已存在可修改
        else if (Boolean.TRUE.equals(canModify)) {
            UserDO userModify = new UserDO();
            userModify.setId(userId);
            userModify.setName(userSave.getName());
            userModify.setDescription(userSave.getDescription());
            userDAO.modify(userModify);
        }
        // 根据存在处理：已存在禁修改
        else {
            throw new UnsupportedOperationException("不支持修改");
        }

        // 返回用户标识
        return userId;
    }
}
```

测试用例：

```
/**
 * 用户服务测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class UserServiceTest {

    /** 定义静态常量 */
```

```
    /** 资源路径 */
    private static final String RESOURCE_PATH =
"testUserService/";

    /** 模拟依赖对象 */
    /** 用户 DAO */
    @Mock
    private UserDAO userDAO;
    /** 标识生成器 */
    @Mock
    private IdGenerator idGenerator;

    /** 定义测试对象 */
    /** 用户服务 */
    @InjectMocks
    private UserService userService;

    /**
     * 在测试之前
     */
    @Before
    public void beforeTest() {
        Whitebox.setInternalState(userService, "canModify",
Boolean.TRUE);
    }

    /**
     * 测试：保存用户-创建
     */
    @Test
    public void testSaveUserWithCreate() {
        // 模拟依赖方法
        // 模拟依赖方法：userDAO.getIdByName

Mockito.doReturn(null).when(userDAO).getIdByName(Mockito.anyStri
ng());
        // 模拟依赖方法：idGenerator.next
        Long userId = 123L;
        Mockito.doReturn(userId).when(idGenerator).next();

        // 调用测试方法
        String path = RESOURCE_PATH + "testSaveUserWithCreate/";
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"userSave.json");
```

```
        UserSaveVO userSave = JSON.parseObject(text,
UserSaveVO.class);
        Assert.assertEquals("用户标识不一致", userId,
userService.saveUser(userSave));

        // 验证依赖方法
        // 验证依赖方法：userDAO.getIdByName
        Mockito.verify(userDAO).getIdByName(userSave.getName());
        // 验证依赖方法：idGenerator.next
        Mockito.verify(idGenerator).next();
        // 验证依赖方法：userDAO.create
        ArgumentCaptor<UserDO> userCreateCaptor =
ArgumentCaptor.forClass(UserDO.class);

Mockito.verify(userDAO).create(userCreateCaptor.capture());
        text = ResourceHelper.getResourceAsString(getClass(),
path + "userCreate.json");
        Assert.assertEquals("用户创建不一致", text,
JSON.toJSONString(userCreateCaptor.getValue()));

        // 验证依赖对象
        Mockito.verifyNoMoreInteractions(userDAO, idGenerator);
    }

    /**
     * 测试：保存用户-修改
     */
    @Test
    public void testSaveUserWithModify() {
        // 模拟依赖方法
        // 模拟依赖方法：userDAO.getIdByName
        Long userId = 123L;

Mockito.doReturn(userId).when(userDAO).getIdByName(Mockito.anySt
ring());

        // 调用测试方法
        String path = RESOURCE_PATH + "testSaveUserWithModify/";
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"userSave.json");
        UserSaveVO userSave = JSON.parseObject(text,
UserSaveVO.class);
        Assert.assertEquals("用户标识不一致", userId,
userService.saveUser(userSave));
```

```
        // 验证依赖方法
        // 验证依赖方法：userDAO.getIdByName
        Mockito.verify(userDAO).getIdByName(userSave.getName());
        // 验证依赖方法：userDAO.modify
        ArgumentCaptor<UserDO> userModifyCaptor =
ArgumentCaptor.forClass(UserDO.class);

Mockito.verify(userDAO).modify(userModifyCaptor.capture());
        text = ResourceHelper.getResourceAsString(getClass(),
path + "userModify.json");
        Assert.assertEquals("用户修改不一致", text,
JSON.toJSONString(userModifyCaptor.getValue()));

        // 验证依赖对象
        Mockito.verifyNoMoreInteractions(userDAO, idGenerator);
    }

    /**
     * 测试：保存用户-异常
     */
    @Test
    public void testSaveUserWithException() {
        // 注入依赖对象
        Whitebox.setInternalState(userService, "canModify",
Boolean.FALSE);

        // 模拟依赖方法
        // 模拟依赖方法：userDAO.getIdByName

Mockito.doReturn(123L).when(userDAO).getIdByName(Mockito.anyStri
ng());

        // 调用测试方法
        String path = RESOURCE_PATH +
"testSaveUserWithException/";
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"userSave.json");
        UserSaveVO userSave = JSON.parseObject(text,
UserSaveVO.class);
        UnsupportedOperationException exception =
Assert.assertThrows("异常类型不一致",
UnsupportedOperationException.class,
            () -> userService.saveUser(userSave));
```

```
        Assert.assertEquals("异常消息不一致", "不支持修改",
exception.getMessage());

        // 验证依赖方法
        // 验证依赖方法：userDAO.getIdByName
        Mockito.verify(userDAO).getIdByName(userSave.getName());

        // 验证依赖对象
        Mockito.verifyNoMoreInteractions(userDAO, idGenerator);
    }
}
```

其中，加载的 JSON 资源文件内容如下：

userSave.json：

```
{"name":"name","description":"description"}
```

userCreate.json：

```
{"description":"description","id":123,"name":"name"}
```

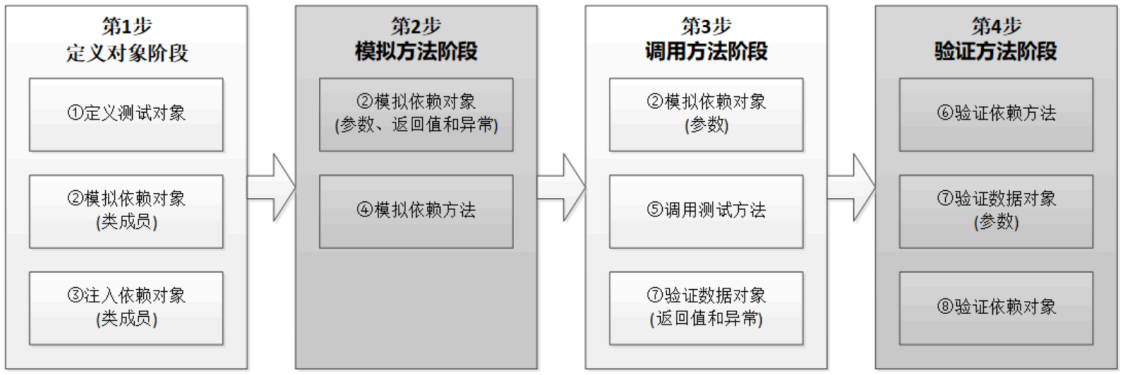
userModify.json：

```
{"description":"description","id":123,"name":"name"}
```

通过执行以上测试用例，可以看到对源代码进行了 100%的行覆盖。

二、 测试用例编写流程

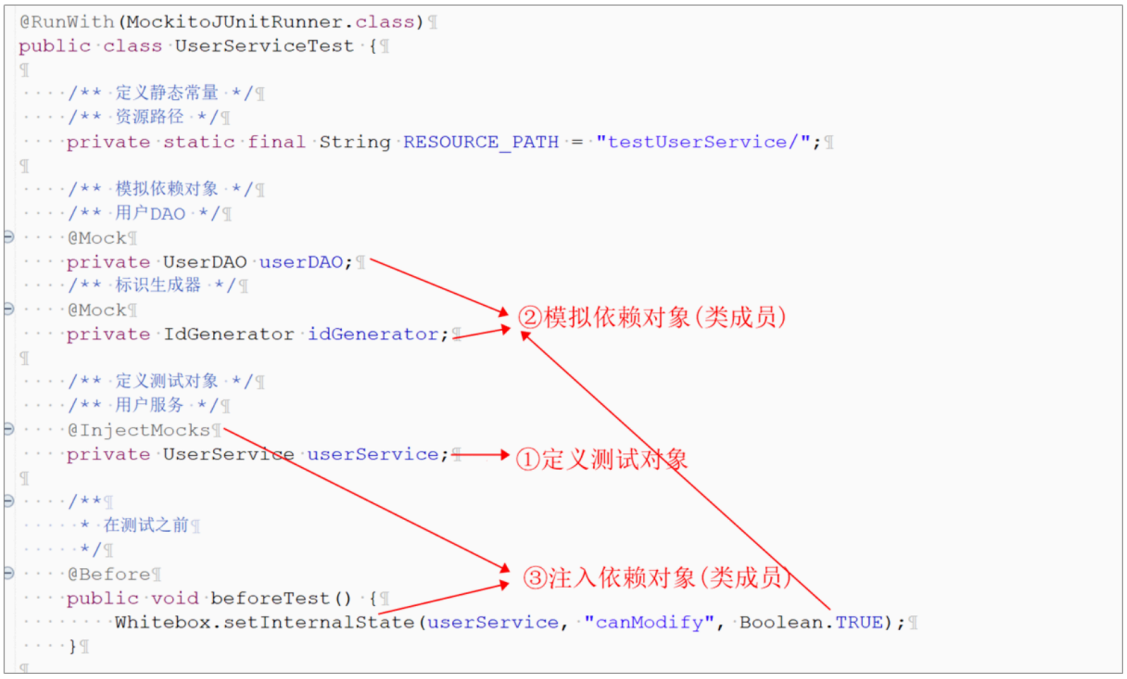
通过上一章编写 Java 类单元测试用例的实践，可以总结出以下 Java 类单元测试用例的编写流程：



如上图所示，第 1、3 步适用于大多数单元测试，而第 2、4 步只适用于有外部依赖的单元测试。这里，以测试用例 testCreateUserWithNew（测试：创建用户——新）为例，来说明单元测试用例编写流程。

1. 定义对象阶段

第 1 步是定义对象阶段，主要包括定义被测对象、模拟依赖对象（类成员）、注入依赖对象（类成员）3 大部分。



1） 定义被测对象

在编写单元测试时，首先需要定义被测对象，或直接初始化、或通过 Spy 包装……其实，就是把被测试服务类进行实例化。

```
/** 定义被测对象 */
/** 用户服务 */
@InjectMocks
private UserService userService;
```

2) 模拟依赖对象（类成员）

在一个服务类中，我们定义了一些类成员对象——服务（Service）、数据访问对象（DAO）、参数（Value）等。在 Spring 框架中，这些类成员对象通过@Autowired、@Value 等方式注入，它们可能涉及复杂的环境配置、依赖第三方接口服务……但是，在单元测试中，为了解除对这些类成员对象的依赖，我们需要对这些类成员对象进行模拟。

```
/** 模拟依赖对象 */
/** 用户DAO */
@Mock
private UserDAO userDAO;
/** 标识生成器 */
@Mock
private IdGenerator idGenerator;
```

3) 注入依赖对象（类成员）

当模拟完这些类成员对象后，我们需要把这些类成员对象注入到被测试类的实例中。以便在调用被测试方法时，可能使用这些类成员对象，而不至于抛出空指针异常。

```
/** 定义被测对象 */
/** 用户服务 */
@InjectMocks
private UserService userService;

/**
 * 在测试之前
 */
@Before
public void beforeTest() {
  // 注入依赖对象
  Whitebox.setInternalState(userService, "canModify", Boolean.TRUE);
}
```

2. 模拟方法阶段

第 2 步是模拟方法阶段，主要包括模拟依赖对象（参数或返回值）、模拟依赖方法 2 大部分。

1) 模拟依赖对象（参数或返回值）

通常，在调用一个方法时，需要先指定方法的参数，然后获取到方法的返回值。所以，在模拟方法之前，需要先模拟该方法的参数和返回值。

```
Long userId = 1L;
```

2) 模拟依赖方法

在模拟完依赖的参数和返回值后，就可以利用 Mockito 和 PowerMock 的功能，进行依赖方法的模拟。如果依赖对象还有方法调用，还需要模拟这些依赖对象的方法。

```
// 模拟依赖方法
// 模拟依赖方法：userDAO.getByName
Mockito.doReturn(null).when(userDAO).getIdByName(Mockito.anyString());
// 模拟依赖方法：idGenerator.next
Mockito.doReturn(userId).when(idGenerator).next();
```

3. 调用方法阶段

第 3 步是调用方法阶段，主要包括模拟依赖对象（参数）、调用被测方法、验证参数对象（返回值）3 步。

1) 模拟依赖对象（参数）

在调用被测方法之前，需要模拟被测方法的参数。如果这些参数还有方法调用，还需要模拟这些参数的方法。

```
String text = ResourceHelper.getResourceAsString(getClass(), "userCreateVO.json");
UserVO userCreate = JSON.parseObject(text, UserVO.class);
```

2) 调用被测方法

在准备好参数对象后，就可以调用被测试方法了。如果被测试方法有返回值，需要定义变量接收返回值；如果被测试方法要抛出异常，需要指定期望的异常。

```
userService.createUser(userCreate)
```

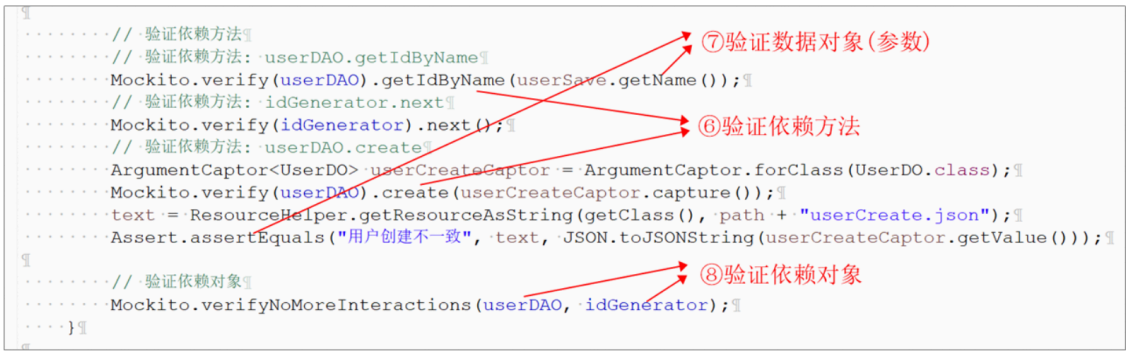
3) 验证数据对象（返回值或异常）

在调用被测试方法后，如果被测试方法有返回值，需要验证这个返回值是否符合预期；如果被测试方法要抛出异常，需要验证这个异常是否满足要求。

```
Assert.assertEquals("用户标识不一致", userId, userService.createUser(userCreate));
```

4. 验证方法阶段

第 4 步是验证方法阶段，主要包括验证依赖方法、验证数据对象（参数）、验证依赖对象 3 步。



1） 验证依赖方法

作为一个完整的测试用例，需要对每一个模拟的依赖方法调用进行验证。

```
// 验证依赖方法
// 验证依赖方法：userDAO.getByName
Mockito.verify(userDAO).getIdByName(userCreate.getName());
// 验证依赖方法：idGenerator.next
Mockito.verify(idGenerator).next();
// 验证依赖方法：userDAO.create
ArgumentCaptor<UserDO> userCreateCaptor = ArgumentCaptor.forClass(UserDO.class);
Mockito.verify(userDAO).create(userCreateCaptor.capture());
```

2） 验证数据对象（参数）

对应一些模拟的依赖方法，有些参数对象是被测试方法内部生成的。为了验证代码逻辑的正确性，就需要对这些参数对象进行验证，看这些参数对象值是否符合预期。

```
text = ResourceHelper.getResourceAsString(getClass(), "userCreateDO.json");
Assert.assertEquals("用户创建不一致", text, JSON.toJSONString(userCreateCaptor.getValue()));
```

3） 验证依赖对象

作为一个完整的测试用例，应该保证每一个模拟的依赖方法调用都进行了验证。正好，Mockito 提供了一套方法，用于验证模拟对象所有方法调用都得到了验证。

```
// 验证依赖对象
Mockito.verifyNoMoreInteractions(idGenerator, userDAO);
```

三、 定义被测对象

在编写单元测试时，首先需要定义被测对象，或直接初始化、或通过 Spy 包装……其实，就是把被测试服务类进行实例化。

1. 直接构建对象

直接构建一个对象，总是简单又直接。

```
UserService userService = new UserService();
```

2. 利用 Mockito.spy 方法

Mockito 提供一个 spy 功能，用于拦截那些尚未实现或不期望被真实调用的方法，默认所有方法都是真实方法，除非主动去模拟对应方法。所以，利用 spy 功能来定义被测对象，适合于需要模拟被测类自身方法的情况，适用于普通类、接口和虚基类。

```
UserService userService = Mockito.spy(new UserService());
UserService userService = Mockito.spy(UserService.class);
AbstractOssService ossService = Mockito.spy(AbstractOssService.class);
```

3. 利用@Spy 注解

@Spy 注解跟 Mockito.spy 方法一样，可以用来定义被测对象，适合于需要模拟被测类自身方法的情况，适用于普通类、接口和虚基类。@Spy 注解需要配合@RunWith 注解使用。

```
@RunWith(PowerMockRunner.class)
public class CompanyServiceTest {
    @Spy
    private UserService userService = new UserService();

    ...
}
```

注意：@Spy 注解对象需要初始化。如果是虚基类或接口，可以用 Mockito.mock 方法实例化。

4. 利用@InjectMocks 注解

@InjectMocks 注解用来创建一个实例，并将其它对象（@Mock、@Spy 或直接定义的对象）注入到该实例中。所以，@InjectMocks 注解本身就可以用来定义被测对象。@InjectMocks 注解需要配合@RunWith 注解使用。

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @InjectMocks
    private UserService userService;
    ...
}
```

四、 模拟依赖对象

在编写单元测试用例时，需要模拟各种依赖对象——类成员、方法参数和方法返回值。

1. 直接构建对象

如果需要构建一个对象，最简单直接的方法就是——定义对象并赋值。

```
Long userId = 1L;
String userName = "admin";
UserDO user = new User();
user.setId(userId);
user.setName(userName);
List<Long> userIdList = Arrays.asList(1L, 2L, 3L);
```

2. 反序列化对象

如果对象字段或层级非常庞大，采用直接构建对象方法，可能会编写大量构建程序代码。这种情况，可以考虑反序列化对象，将会大大减少程序代码。由于 JSON 字符串可读性高，这里就以 JSON 为例，介绍反序列化对象。

反序列化模型对象：

```
String text = ResourceHelper.getResourceAsString(getClass(), "user.json");
UserDO user = JSON.parseObject(text, UserDO.class);
```

反序列化集合对象：

```
String text = ResourceHelper.getResourceAsString(getClass(), "userList.json");
List<UserDO> userList = JSON.parseArray(text, UserDO.class);
```

反序列化映射对象：

```
String text = ResourceHelper.getResourceAsString(getClass(), "userMap.json");
Map<Long, UserDO> userMap = JSON.parseObject(text, new TypeReference<Map<Long, UserDO>>()
{});
```

建议：JSON 文件内容不要格式化，以免占用更多的代码行数。

3. 利用 Mockito.mock 方法

Mockito 提供一个 mock 功能，用于拦截那些尚未实现或不期望被真实调用的方法，默认所有方法都已被模拟——方法为空并返回默认值（null 或 0），除非主动执行 doCallRealMethod 或 thenCallRealMethod 操作，才能够调用真实的方法。

利用 Mockito.mock 方法模拟依赖对象，主要用于以下几种情形：

- 只使用类实例，不使用类属性；
- 类属性太多，但使用其中少量属性（可以 mock 属性返回值）；
- 类是接口或虚基类，并不关心其具体实现类。

```
MockClass mockClass = Mockito.mock(MockClass.class);
List<Long> userIdList = (List<Long>)Mockito.mock(List.class);
```

4. 利用@Mock 注解

@Mock 注解跟 Mockito.mock 方法一样，可以用来模拟依赖对象，适用于普通类、接口和虚基类。@Mock 注解需要配合@RunWith 注解使用。

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Mock
    private UserDAO userDAO;
    ...
}
```

5. 利用 Mockito.spy 方法

Mockito.spy 方法跟 Mockito.mock 方法功能相似，只是 Mockito.spy 方法默认所有方法都是真实方法，除非主动去模拟对应方法。

```
UserService userService = Mockito.spy(new UserService());
UserService userService = Mockito.spy(UserService.class);
AbstractOssService ossService = Mockito.spy(AbstractOssService.class);
```

6. 利用@Spy 注解

@Spy 注解跟 Mockito.spy 方法一样，可以用来模拟依赖对象，适用于普通类、接口和虚基类。@Spy 注解需要配合@RunWith 注解使用。

```
@RunWith(PowerMockRunner.class)
public class CompanyServiceTest {
    @Spy
    private UserService userService = new UserService();
    ...
}
```

注意：@Spy 注解对象需要初始化。如果是虚基类或接口，可以用 Mockito.mock 方法实例化。

五、 注入依赖对象

当模拟完这些类成员对象后，我们需要把这些类成员对象注入到被测试类的实例中。以便在调用被测试方法时，可能使用这些类成员对象，而不至于抛出空指针异常。

1. 利用 Setter 方法注入

如果类定义了 Setter 方法，可以直接调用方法设置字段值。

```
userService.setMaxCount(100);
userService.setUserDAO(userDAO);
```

2. 利用 ReflectionTestUtils.setField 方法注入

JUnit 提供 ReflectionTestUtils.setField 方法设置属性字段值。

```
ReflectionTestUtils.setField(userService, "maxCount", 100);
ReflectionTestUtils.setField(userService, "userDAO", userDAO);
```

3. 利用 Whitebox.setInternalState 方法注入

PowerMock 提供 Whitebox.setInternalState 方法设置属性字段值。

```
Whitebox.setInternalState(userService, "maxCount", 100);
Whitebox.setInternalState(userService, "userDAO", userDAO);
```

4. 利用@InjectMocks 注解注入

@InjectMocks 注解用来创建一个实例，并将其它对象（@Mock、@Spy 或直接定义的对象）注入到该实例中。@InjectMocks 注解需要配合@RunWith 注解使用。

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Mock
    private UserDAO userDAO;
    private Boolean canModify;

    @InjectMocks
    private UserService userService;
    ...
}
```

5. 设置静态常量字段值

有时候，我们需要对静态常量对象进行模拟，然后去验证是否执行了对应分支下的方法。比如：需要模拟 Lombok 的@Slf4j 生成的 log 静态常量。但是，Whitebox.setInternalState 方法和@InjectMocks 注解并不支持设置静态常量，需要自己实现一个设置静态常量的方法：

```
public final class FieldHelper {
    public static void setStaticFinalField(Class<?> clazz, String fieldName, Object
fieldValue) throws NoSuchFieldException, IllegalAccessException {
        Field field = clazz.getDeclaredField(fieldName);
        FieldUtils.removeFinalModifier(field);
        FieldUtils.writeStaticField(field, fieldValue, true);
    }
}
```

具体使用方法如下：

```
FieldHelper.setStaticFinalField(UserService.class, "log", log);
```

注意：经过测试，该方法对于 int、Integer 等基础类型并不生效，应该是编译器常量优化导致。

六、 模拟依赖方法

在模拟完依赖的参数和返回值后，就可以利用 Mockito 和 PowerMock 的功能，进行依赖方法的模拟。如果依赖对象还有方法调用，还需要模拟这些依赖对象的方法。

1. 根据返回模拟方法

1) 模拟无返回值方法

```
Mockito.doNothing().when(userDAO).delete(userId);
```

2) 模拟方法单个返回值

```
Mockito.doReturn(user).when(userDAO).get(userId);
Mockito.when(userDAO.get(userId)).thenReturn(user);
```

3) 模拟方法多个返回值

直接列举出多个返回值：

```
Mockito.doReturn(record0, record1, record2, null).when(recordReader).read();
Mockito.when(recordReader.read()).thenReturn(record0, record1, record2, null);
```

转化列表为多个返回值：

```
List<Record> recordList = ...;
Mockito.doReturn(recordList.get(0), recordList.subList(1,
recordList.size()).toArray()).when(recordReader).read();
Mockito.when(recordReader.read()).thenReturn(recordList.get(0), recordList.subList(1,
recordList.size()).toArray());
```

4) 模拟方法定制返回值

可利用 Answer 定制方法返回值：

```
Map<Long, UserDO> userMap = ...;
Mockito.doAnswer(invocation -> userMap.get(invocation.getArgument(0)))
    .when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong()))
    .thenReturn(invocation -> userMap.get(invocation.getArgument(0)));
Mockito.when(userDAO.get(Mockito.anyLong()))
    .then(invocation -> userMap.get(invocation.getArgument(0)));
```

5) 模拟方法抛出单个异常

指定单个异常类型：

```
Mockito.doThrow(PersistenceException.class).when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong())).thenThrow(PersistenceException.class);
```

指定单个异常对象：

```
Mockito.doThrow(exception).when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong())).thenThrow(exception);
```

6) 模拟方法抛出多个异常

指定多个异常类型：

```
Mockito.doThrow(PersistenceException.class,
RuntimeException.class).when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong())).thenThrow(PersistenceException.class,
RuntimeException.class);
```

指定多个异常对象：

```
Mockito.doThrow(exception1, exception2).when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong())).thenThrow(exception1, exception2);
```

7) 直接调用真实方法

```
Mockito.doCallRealMethod().when(userService).getUser(userId);
Mockito.when(userService.getUser(userId)).thenCallRealMethod();
```

2. 根据参数模拟方法

Mockito 提供 do-when 语句和 when-then 语句模拟方法。

1) 模拟无参数方法

对于无参数的方法模拟：

```
Mockito.doReturn(deleteCount).when(userDAO).deleteAll();
Mockito.when(userDAO.deleteAll()).thenReturn(deleteCount);
```

2) 模拟指定参数方法

对于指定参数的方法模拟：

```
Mockito.doReturn(user).when(userDAO).get(userId);
Mockito.when(userDAO.get(userId)).thenReturn(user);
```

3) 模拟匹配参数方法

在模拟依赖方法时，需要自定义匹配参数值，可以使用 Mockito 参数匹配器的 that 方法。Mockito 提供了 booleanThat、intThat、longThat、doubleThat、argThat 等方法来进行匹配值。

```
Mockito.doReturn(user).when(userDAO).get(Mockito.longThat(userIdSet::contains));
Mockito.when(userDAO.get(userIdSet::contains)).thenReturn(user);
```

4) 模拟任意参数方法

在模拟依赖方法时，有时候并不关心传入参数的具体值，可以使用 Mockito 参数匹配器的 any 方法。Mockito 提供了 anyInt、anyLong、anyString、anyList、anySet、anyMap、any(Class<T> clazz)等方法来表示任意值。

```
Mockito.doReturn(user).when(userDAO).get(Mockito.anyLong());
Mockito.when(userDAO.get(Mockito.anyLong())).thenReturn(user);
```

5) 模拟可空参数方法

Mockito 参数匹配器的 any 具体方法，并不能够匹配 null 对象。而 Mockito 提供一个 nullable 方法，可以匹配包含 null 对象的任意对象。此外，Mockito.any()方法也可以用来匹配可空参数。

```
Mockito.doReturn(user).when(userDAO)
    .queryCompany(Mockito.anyLong(), Mockito.nullable(Long.class));
Mockito.when(userDAO.queryCompany(Mockito.anyLong(), Mockito.<Long>any()))
    .thenReturn(user);
```

6) 拟必空参数方法

同样，如果要匹配 null 对象，可以使用 isNull 方法，或使用 eq(null)。

```
Mockito.doReturn(user).when(userDAO).queryCompany(Mockito.anyLong(), Mockito.isNull());
Mockito.when(userDAO.queryCompany(Mockito.anyLong(), Mockito.eq(null))).thenReturn(user);
```

7) 模拟不同参数方法

Mockito 支持按不同的参数分别模拟同一方法。

```
Mockito.doReturn(user1).when(userDAO).get(1L);
Mockito.doReturn(user2).when(userDAO).get(2L);
...
```

注意：如果一个参数满足多个模拟方法条件，会以最后一个模拟方法为准。

8) 模拟可变参数方法

对于一些变长度参数方法，可以按实际参数个数进行模拟：

```
Mockito.when(userService.delete(Mockito.anyLong()).thenReturn(true);
Mockito.when(userService.delete(1L, 2L, 3L).thenReturn(true);
```

也可以用 Mockito.any()模拟一个通用匹配方法：

```
Mockito.when(userService.delete(Mockito.<Long>any()).thenReturn(true);
```

注意：Mockito.<T>any()并不等于 Mockito.any(Class<T> type),前者可以匹配 null 和类型 T 的可变参数，后者只能匹配 T 必填参数。

3. 模拟其它特殊方法

1) 模拟 final 方法

PowerMock 提供对 final 方法的模拟，方法跟模拟普通方法一样。但是，需要把对应的模拟类添加到@PrepareForTest 注解中。

```
// 添加@PrepareForTest注解
@PrepareForTest({UserService.class})

// 跟模拟普通方法完全一致
Mockito.doReturn(userId).when(idGenerator).next();
Mockito.when(idGenerator.next()).thenReturn(userId);
```

2) 模拟私有方法

PowerMock 提供提对私有方法的模拟，但是需要把私有方法所在的类放在@PrepareForTest 注解中。

```
PowerMockito.doReturn(true).when(UserService.class, "isSuper", userId);
PowerMockito.when(UserService.class, "isSuper", userId).thenReturn(true);
```

3) 模拟构造方法

PowerMock 提供 PowerMockito.whenNew 方法来模拟构造方法，但是需要把使用构造方法的类放在@PrepareForTest 注解中。

```
PowerMockito.whenNew(UserDO.class).withNoArguments().thenReturn(userDO);
PowerMockito.whenNew(UserDO.class).withArguments(userId, userName).thenReturn(userDO);
```

4) 模拟静态方法

PowerMock 提供 PowerMockito.mockStatic 和 PowerMockito.spy 来模拟静态方法类，然后就可以模拟静态方法了。同样，需要把对应的模拟类添加到@PrepareForTest 注解中。

```
// 模拟对应的类
PowerMockito.mockStatic(HttpHelper.class);
PowerMockito.spy(HttpHelper.class);

// 模拟对应的方法
PowerMockito.when(HttpHelper.httpPost(SERVER_URL)).thenReturn(response);
PowerMockito.doReturn(response).when(HttpHelper.class, "httpPost", SERVER_URL);
PowerMockito.when(HttpHelper.class, "httpPost", SERVER_URL).thenReturn(response);
```

注意：第一种方式不适用于 PowerMockito.spy 模拟的静态方法类。

七、 调用被测方法

在准备好参数对象后，就可以调用被测试方法了。

如果把方法按访问权限分类，可以简单地分为有访问权限和无访问权限两种。但实际上，Java 语言中提供了 public、protected、private 和缺失共 4 种权限修饰符，在不同的环境下又对应不同的访问权限。具体映射关系如下：

修饰符	本类	本包	子类	其它
public	有	有	有	有
protected	有	有	有	无
缺省	有	有	无	无
private	有	无	无	无

下面，将根据有访问权限和无访问权限两种情况，来介绍如何调用被测方法。

1. 调用构造方法

1) 调用有访问权限的构造方法

可以直接调用有访问权限的构造方法。

```
UserDO user = new User();
UserDO user = new User(1L, "admin");
```

2) 调用无访问权限的构造方法

调用无访问权限的构造方法，可以使用 PowerMock 提供的 Whitebox.invokeConstructor 方法。

```
Whitebox.invokeConstructor(NumberHelper.class);
Whitebox.invokeConstructor(User.class, 1L, "admin");
```

备注：该方法也可以调用有访问权限的构造方法，但是不建议使用。

2. 调用普通方法

1) 调用有访问权限的普通方法

可以直接调用有访问权限的普通方法。

```
userService.deleteUser(userId);
User user = userService.getUser(userId);
```

2) 调用无权限访问的普通方法

调用无访问权限的普通方法，可以使用 PowerMock 提供的 Whitebox.invokeMethod 方法。

```
User user = (User)Whitebox.invokeMethod(userService, "isSuper", userId);
```

也可以使用 PowerMock 提供 Whitebox.getMethod 方法和 PowerMockito.method 方法，可以直接获取对应类方法对象。然后，通过 Method 的 invoke 方法，可以调用没有访问权限的方法。

```
Method method = Whitebox.getMethod(UserService.class, "isSuper", Long.class);
Method method = PowerMockito.method(UserService.class, "isSuper", Long.class);
User user = (User)method.invoke(userService, userId);
```

备注：该方法也可以调用有访问权限的普通方法，但是不建议使用。

3. 调用静态方法

1) 调用有权限访问的静态方法

可以直接调用有访问权限的静态方法。

```
boolean isPositive = NumberHelper.isPositive(-1);
```

2) 调用无权限访问的静态方法

调用无权限访问的静态方法，可以使用 PowerMock 提供的 Whitebox.invokeMethod 方法。

```
String value = (String)Whitebox.invokeMethod(JSON.class, "toJSONString", object);
```

备注：该方法也可以调用有访问权限的静态方法，但是不建议使用。

八、 验证依赖方法

在单元测试中，验证是确认模拟的依赖方法是否按照预期被调用或未调用的过程。Mockito 提供了许多方法来验证依赖方法调用，给我们编写单元测试用例带来了很大的帮助。

1. 根据参数验证方法调用

1) 验证无参数方法调用

```
Mockito.verify(userDAO).deleteAll();
```

2) 验证指定参数方法调用

```
Mockito.verify(userDAO).delete(userId);
Mockito.verify(userDAO).delete(Mockito.eq(userId));
```

3) 验证匹配参数方法调用

在验证依赖方法时，需要自定义匹配参数值，可以使用 Mockito 参数匹配器的 that 方法。Mockito 提供了 booleanThat、intThat、longThat、doubleThat、argThat 等方法来进行匹配值。

```
Mockito.verify(userDAO).delete(Mockito.longThat(userIdSet::contains));
```

4) 验证任意参数方法调用

在验证依赖方法时，有时候并不关心传入参数的具体值，可以使用 Mockito 参数匹配器的 any 方法。Mockito 提供了 anyInt、anyLong、anyString、anyList、anySet、anyMap、any(Class<T> clazz)等方法来表示任意值。

```
Mockito.verify(userDAO).delete(Mockito.anyLong());
```

5) 验证可空参数方法调用

Mockito 参数匹配器的 any 具体方法，并不能够匹配 null 对象。而 Mockito 提供一个 nullable 方法，可以匹配包含 null 对象的任意对象。此外，Mockito.any()方法也可以用来匹配可空参数。

```
Mockito.verify(userDAO).queryCompany(Mockito.anyLong(), Mockito.nullable(Long.class));
```

6) 验证必空参数方法调用

同样，如果要匹配 null 对象，可以使用 isNull 方法，或使用 eq(null)。

```
Mockito.verify(userDAO).queryCompany(Mockito.anyLong(), Mockito.isNull());
```

7) 验证不同参数方法调用

Mockito 支持按不同的参数分别验证同一方法。

```
Mockito.verify(userDAO).get(1L);
Mockito.verify(userDAO).get(2L);
...
```

8) 验证可变参数方法调用

对于一些变长度参数方法，可以按实际参数个数进行验证：

```
Mockito.verify(userService).delete(Mockito.any(Long.class));
Mockito.verify(userService).delete(1L, 2L, 3L);
```

也可以用 Mockito.any()进行通用验证：

```
Mockito.verify(userService).delete(Mockito.<Long>any());
```

2. 验证方法调用次数

1) 验证方法默认调用 1 次

```
Mockito.verify(userDAO).delete(userId);
```

2) 验证方法从不调用

```
Mockito.verify(userDAO, Mockito.never()).delete(userId);
```

3) 验证方法调用 n 次

```
Mockito.verify(userDAO, Mockito.times(n)).delete(userId);
```

4) 验证方法调用至少 1 次

```
Mockito.verify(userDAO, Mockito.atLeastOnce()).delete(userId);
```

5) 验证方法调用至少 n 次

```
Mockito.verify(userDAO, Mockito.atLeast(n)).delete(userId);
```

6) 验证方法调用最多 1 次

```
Mockito.verify(userDAO, Mockito.atMostOnce()).delete(userId);
```

7) 验证方法调用最多 n 次

```
Mockito.verify(userDAO, Mockito.atMost(n)).delete(userId);
```

8) 验证方法调用指定 n 次

Mockito 允许按顺序进行验证方法调用，未被验证到的方法调用将不会被标记为已验证。

```
Mockito.verify(userDAO, Mockito.call(n)).delete(userId);
```

9) 验证对象及其方法调用 1 次

用于验证对象及其方法调用 1 次，如果该对象还有别的方法被调用或者该方法调用了多次，都将导致验证方法调用失败。

```
Mockito.verify(userDAO, Mockito.only()).delete(userId);
```

相当于：

```
Mockito.verify(userDAO).delete(userId);
Mockito.verifyNoMoreInteractions(userDAO);
```

3. 验证方法调用并捕获参数值

Mockito 提供 ArgumentCaptor 类来捕获参数值，通过调用 forClass(Class<T> clazz) 方法来构建一个 ArgumentCaptor 对象，然后在验证方法调用时来捕获参数，最后获取到捕获的参数值并验证。如果一个方法有多个参数都要捕获并验证，那就需要创建多个 ArgumentCaptor 对象。

ArgumentCaptor 的主要接口方法：

- capture 方法，用于捕获方法参数；
- getValue 方法，用于获取捕获的参数值，如果捕获了多个参数值，该方法只返回最后一个参数值；
- getAllValues 方法，用户获取捕获的所有参数值。

1) 使用 ArgumentCaptor.forClass 方法定义参数捕获器

在测试用例方法中，直接使用 ArgumentCaptor.forClass 方法定义参数捕获器。

```
ArgumentCaptor<UserDO> userCaptor = ArgumentCaptor.forClass(UserDO.class);
Mockito.verify(userDAO).modify(userCaptor.capture());
UserDO user = userCaptor.getValue();
```

注意：定义泛型类的参数捕获器时，存在强制类型转换，会引起编译器警告。

2) 使用@Captor 注解定义参数捕获器

也可以用 Mockito 提供的@Captor 注解，在测试用例类中定义参数捕获器。

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {
    @Captor
    private ArgumentCaptor<UserDO> userCaptor;
    @Test
    public void testModifyUser() {
        ...
        Mockito.verify(userDAO).modify(userCaptor.capture());
        UserDO user = userCaptor.getValue();
    }
}
```

注意：定义泛型类的参数捕获器时，由于是 Mockito 自行初始化，不会引起编译器警告。

3) 捕获多次方法调用的参数值列表

```
ArgumentCaptor<UserDO> userCaptor = ArgumentCaptor.forClass(UserDO.class);
Mockito.verify(userDAO, Mockito.atLeastOnce()).modify(userCaptor.capture());
List<UserDO> userList = userCaptor.getAllValues();
```

4. 验证其它特殊方法

1) 验证 final 方法调用

final 方法的验证跟普通方法类似，这里不再累述。

2) 验证私有方法调用

PowerMockito 提供 verifyPrivate 方法验证私有方法调用。

```
PowerMockito.verifyPrivate(myClass, times(1)).invoke("unload", any(List.class));
```

3) 验证构造方法调用

PowerMockito 提供 verifyNew 方法验证构造方法调用。

```
PowerMockito.verifyNew(MockClass.class).withNoArguments();
PowerMockito.verifyNew(MockClass.class).withArguments(someArgs);
```

4) 验证静态方法调用

PowerMockito 提供 verifyStatic 方法验证静态方法调用。

```
PowerMockito.verifyStatic(StringUtils.class);
StringUtils.isEmpty(string);
```

九、 验证数据对象

JUnit 测试框架中 Assert 类就是断言工具类，主要验证单元测试中实际数据对象与期望数据对象一致。在调用被测方法时，需要对返回值和异常进行验证；在验证方法调用时，也需要对捕获的参数值进行验证。

1. 验证数据对象空值

1) 验证数据对象为空

通过 JUnit 提供的 Assert.assertNull 方法验证数据对象为空。

```
Assert.assertNull("用户标识必须为空", userId);
```

2) 验证数据对象非空

通过 JUnit 提供的 Assert.assertNotNull 方法验证数据对象非空。

```
Assert.assertNotNull("用户标识不能为空", userId);
```

2. 验证数据对象布尔值

1) 验证数据对象为真

通过 JUnit 提供的 Assert.assertTrue 方法验证数据对象为真。

```
Assert.assertTrue("返回值必须为真", NumberHelper.isPositive(1));
```

2) 验证数据对象为假

通过 JUnit 提供的 Assert.assertFalse 方法验证数据对象为假。

```
Assert.assertFalse("返回值必须为假", NumberHelper.isPositive(-1));
```

3. 验证数据对象引用

在单元测试用例中，对于一些参数或返回值对象，不需要验证对象具体取值，只需要验证对象引用是否一致。

1) 验证数据对象一致

JUnit 提供的 Assert.assertSame 方法验证数据对象一致。

```
UserDO expectedUser = ...;
Mockito.doReturn(expectedUser).when(userDAO).get(userId);
UserDO actualUser = userService.getUser(userId);
Assert.assertSame("用户必须一致", expectedUser, actualUser);
```

2) 验证数据对象不一致

JUnit 提供的 Assert.assertNotSame 方法验证数据对象一致。

```
UserDO expectedUser = ...;
Mockito.doReturn(expectedUser).when(userDAO).get(userId);
UserDO actualUser = userService.getUser(otherUserId);
Assert.assertNotSame("用户不能一致", expectedUser, actualUser);
```

4. 验证数据对象值

JUnit 提供 Assert.assertEquals、Assert.assertNotEquals、Assert.assertArrayEquals 方法组，可以用来验证数据对象值是否相等。

1) 验证简单数据对象

对于简单数据对象（比如：基础类型、包装类型、实现了 equals 的数据类型……），可以直接通过 JUnit 的 Assert.assertEquals 和 Assert.assertNotEquals 方法组进行验证。

```
Assert.assertNotEquals("用户名称不一致", "admin", userName);
Assert.assertEquals("账户金额不一致", 10000.0D, accountAmount, 1E-6D);
```

2) 验证简单数组或集合对象

对于简单数组对象（比如：基础类型、包装类型、实现了 equals 的数据类型……），可以直接通过 JUnit 的 Assert.assertArrayEquals 方法组进行验证。对于简单集合对象，也可以通过 Assert.assertEquals 方法验证。

```
Long[] userIds = ...;
Assert.assertArrayEquals("用户标识列表不一致", new Long[] {1L, 2L, 3L}, userIds);

List<Long> userIdList = ...;
Assert.assertEquals("用户标识列表不一致", Arrays.asList(1L, 2L, 3L), userIdList);
```

3) 验证复杂数据对象

对于复杂的 JavaBean 数据对象，需要验证 JavaBean 数据对象的每一个属性字段。

```
UserDO user = ...;
Assert.assertEquals("用户标识不一致", Long.valueOf(1L), user.getId());
Assert.assertEquals("用户名称不一致", "admin", user.getName());
Assert.assertEquals("用户公司标识不一致", Long.valueOf(1L), user.getCompany().getId());
...
```

4) 验证复杂数组或集合对象

对于复杂的 JavaBean 数组和集合对象，需要先展开数组和集合对象中每一个 JavaBean 数据对象，然后验证 JavaBean 数据对象的每一个属性字段。

```
List<UserDO> expectedUserList = ...;
List<UserDO> actualUserList = ...;
Assert.assertEquals("用户列表长度不一致", expectedUserList.size(), actualUserList.size());
UserDO[] expectedUsers = expectedUserList.toArray(new UserDO[0]);
UserDO[] actualUsers = actualUserList.toArray(new UserDO[0]);
for (int i = 0; i < actualUsers.length; i++) {
    Assert.assertEquals(String.format("用户(%s)标识不一致", i), expectedUsers[i].getId(),
actualUsers[i].getId());
    Assert.assertEquals(String.format("用户(%s)名称不一致", i), expectedUsers[i].getName(),
actualUsers[i].getName());
Assert.assertEquals("用户公司标识不一致", expectedUsers[i].getCompany().getId(),
actualUsers[i].getCompany().getId());
    ...
}
```

5) 通过序列化验证数据对象

如上一节例子所示，当数据对象过于复杂时，如果采用 Assert.assertEquals 依次验证每个 JavaBean 对象、验证每一个属性字段，测试用例的代码量将会非常庞大。这里，推荐使用序列化手段简化数据对象的验证，比如利用 JSON.toJSONString 方法把复杂的数据对象转化为字符串，然后再使用 Assert.assertEquals 方法进行验证字符串。但是，序列化值必须具备有序性、一致性和可读性。

```
List<UserDO> userList = ...;
String text = ResourceHelper.getResourceAsString(getClass(), "userList.json");
Assert.assertEquals("用户列表不一致", text, JSON.toJSONString(userList));
```

通常使用 JSON.toJSONString 方法把 Map 对象转化为字符串，其中 key-value 的顺序具有不确定性，无法用于验证两个对象是否一致。这里，JSON 提供序列化选项 SerializerFeature.MapSortField(映射排序字段)，可以用于保证序列化后的 key-value 的有序性。

```
Map<Long, Map<String, Object>> userMap = ...;
String text = ResourceHelper.getResourceAsString(getClass(), "userMap.json");
Assert.assertEquals("用户映射不一致", text, JSON.toJSONString(userMap,
SerializerFeature.MapSortField));
```

建议：JSON 文件内容不要格式化，不但会占用更多的代码行数，还会导致无法进行文本比较。

6) 验证数据对象私有属性字段

有时候，单元测试用例需要对复杂对象的私有属性字段进行验证。而 PowerMockito 提供的 Whitebox.getInternalState 方法，获取轻松地获取到私有属性字段值。

```
MapperScannerConfigurer configurer = myBatisConfiguration.buildMapperScannerConfigurer();
Assert.assertEquals("基础包不一致", "com.alibaba.example",
Whitebox.getInternalState(configurer, "basePackage"));
```

5. 验证异常对象内容

异常作为 Java 语言的重要特性，是 Java 语言健壮性的重要体现。捕获并验证异常数据内容，也是测试用例的一种。

1) 通过@Test 注解验证异常对象

JUnit 的注解@Test 提供了一个 expected 属性，可以指定一个期望的异常类型，用来捕获并验证异常。但是，这种方式只能验证异常类型，并不能验证异常原因和消息。

```
@Test(expected = ExampleException.class)
public void testGetUser() {
    // 模拟依赖方法
    Mockito.doReturn(null).when(userDAO).get(userId);

    // 调用被测方法
    userService.getUser(userId);
}
```

2) 通过@Rule 注解验证异常对象

如果想要验证异常原因和消息，就需求采用@Rule 注解定义 ExpectedException 对象，然后在测试方法的前面声明要捕获的异常类型、原因和消息。

```
@Rule
private ExpectedException exception = ExpectedException.none();
@Test
public void testGetUser() {
    // 模拟依赖方法
    Long userId = 123L;
    Mockito.doReturn(null).when(userDAO).get(userId);

    // 调用被测方法
    exception.expect(ExampleException.class);
    exception.expectMessage(String.format("用户(%s)不存在", userId));
    userService.getUser(userId);
}
```

3) 通过 Assert.assertThrows 验证异常对象

在最新版的 JUnit 中，提供了一个更为简洁的异常验证方式——Assert.assertThrows 方法。

```
@Test
public void testGetUser() {
    // 模拟依赖方法
    Long userId = 123L;
    Mockito.doReturn(null).when(userDAO).get(userId);

    // 调用被测方法
    ExampleException exception = Assert.assertThrows("异常类型不一致", ExampleException.class,
() -> userService.getUser(userId));
    Assert.assertEquals("异常消息不一致", "处理异常", exception.getMessage());
}
```

十、 验证依赖对象

1. 验证模拟对象没有任何方法调用

Mockito 提供了 verifyNoInteractions 方法，可以验证模拟对象在被测试方法中没有任何调用。

```
Mockito.verifyNoInteractions(idGenerator, userDAO);
```

2. 验证模拟对象没有更多方法调用

Mockito 提供了 verifyNoMoreInteractions 方法，在验证模拟对象所有方法调用后使用，可以验证模拟对象所有方法调用是否都得到验证。如果模拟对象存在任何未验证的方法调用，就会抛出 NoInteractionsWanted 异常。

```
Mockito.verifyNoMoreInteractions(idGenerator, userDAO);
```

备注：Mockito 的 verifyZeroInteractions 方法与 verifyNoMoreInteractions 方法功能相同，但是目前前者已经被废弃。

3. 清除模拟对象所有方法调用标记

在编写单元测试用例时，为了减少单元测试用例数和代码量，可以把多组参数定义在同一个单元测试用例中，然后用 for 循环依次执行每一组参数的被测方法调用。为了避免上一次测试的方法调用影响下一次测试的方法调用验证，最好使用 Mockito 提供 clearInvocations 方法清除上一次的方法调用。

```
// 清除所有对象调用
Mockito.clearInvocations();
// 清除指定对象调用
Mockito.clearInvocations(idGenerator, userDAO);
```

十一、 典型案例及解决方案

这里，只收集了几个经典案例，解决了特定环境下的特定问题。

1. 测试框架特性导致问题

在编写单元测试用例时，或多或少会遇到一些问题，大多数是由于对测试框架特性不熟悉导致，比如：

- Mockito 不支持对静态方法、构造方法、final 方法、私有方法的模拟；
- Mockito 的 any 相关的参数匹配方法并不支持可空参数和空参数；

- 采用 Mockito 的参数匹配方法时，其它参数不能直接用常量或变量，必须使用 Mockito 的 eq 方法；

- 使用 when-then 语句模拟 Spy 对象方法会先执行真实方法，应该使用 do-when 语句；

- PowerMock 对静态方法、构造方法、final 方法、私有方法的模拟需要把对应的类添加到@PrepareForTest 注解中；

- PowerMock 模拟 JDK 的静态方法、构造方法、final 方法、私有方法时，需要把使用这些方法的类加入到@PrepareForTest 注解中，从而导致单元测试覆盖率不被统计；

- PowerMock 使用自定义的类加载器来加载类，可能导致系统类加载器认为有类型转换问题；需要加上@PowerMockIgnore({"javax.crypto.*"})注解，来告诉 PowerMock 这个包不要用 PowerMock 的类加载器加载，需要采用系统类加载器来加载。

对于这些问题，可以根据提示信息查阅相关资料解决，这里就不再累述了。

2. 捕获参数值已变更问题

在编写单元测试用例时，通常采用 ArgumentCaptor 进行参数捕获，然后对参数对象值进行验证。如果参数对象值没有变更，这个步骤就没有任何问题。但是，如果参数对象值在后续流程中发生变更，就会导致验证参数值失败。

原始代码：

```
public <T> void readData(RecordReader recordReader, int batchSize, Function<Record, T>
dataParser, Predicate<List<T>> dataStorage) {
    try {
        // 依次读取数据
        Record record;
        boolean isContinue = true;
        List<T> dataList = new ArrayList<>(batchSize);
        while (Objects.nonNull(record = recordReader.read()) && isContinue) {
            // 解析添加数据
            T data = dataParser.apply(record);
            if (Objects.nonNull(data)) {
                dataList.add(data);
            }

            // 批量存储数据
            if (dataList.size() == batchSize) {
                isContinue = dataStorage.test(dataList);
                dataList.clear();
            }
        }

        // 存储剩余数据
        if (CollectionUtils.isNotEmpty(dataList)) {
            dataStorage.test(dataList);
            dataList.clear();
        }
    } catch (IOException e) {
        String message = READ_DATA_EXCEPTION;
        log.warn(message, e);
        throw new ExampleException(message, e);
    }
}
```

测试用例：

```
@Test
public void testReadData() throws Exception {
    // 模拟依赖方法
    // 模拟依赖方法: recordReader.read
    Record record0 = Mockito.mock(Record.class);
    Record record1 = Mockito.mock(Record.class);
    Record record2 = Mockito.mock(Record.class);
    TunnelRecordReader recordReader = Mockito.mock(TunnelRecordReader.class);
    Mockito.doReturn(record0, record1, record2, null).when(recordReader).read();
    // 模拟依赖方法: dataParser.apply
    Object object0 = new Object();
    Object object1 = new Object();
    Object object2 = new Object();
    Function<Record, Object> dataParser = Mockito.mock(Function.class);
    Mockito.doReturn(object0).when(dataParser).apply(record0);
    Mockito.doReturn(object1).when(dataParser).apply(record1);
    Mockito.doReturn(object2).when(dataParser).apply(record2);
    // 模拟依赖方法: dataStorage.test
    Predicate<List<Object>> dataStorage = Mockito.mock(Predicate.class);
    Mockito.doReturn(true).when(dataStorage).test(Mockito.anyList());

    // 调用测试方法
    odpsService.readData(recordReader, 2, dataParser, dataStorage);

    // 验证依赖方法
    // 模拟依赖方法: recordReader.read
    Mockito.verify(recordReader, Mockito.times(4)).read();
    // 模拟依赖方法: dataParser.apply
    Mockito.verify(dataParser, Mockito.times(3)).apply(Mockito.any(Record.class));
    // 验证依赖方法: dataStorage.test
    ArgumentCaptor<List<Object>> recordListCaptor = ArgumentCaptor.forClass(List.class);
    Mockito.verify(dataStorage, Mockito.times(2)).test(recordListCaptor.capture());
    Assert.assertEquals("数据列表不一致", Arrays.asList(Arrays.asList(object0, object1),
Arrays.asList(object2)), recordListCaptor.getAllValues());
}
```

问题现象：

执行单元测试用例失败，抛出以下异常信息：

```
java.lang.AssertionError: 数据列表不一致 expected:<[[java.lang.Object@1e3469df,
java.lang.Object@79499fa], [java.lang.Object@48531d5]]> but was:<[[], []]>
```

问题原因：

由于参数 dataList 在调用 dataStorage.test 方法后，都被主动调用 dataList.clear 方法进行清空。由于 ArgumentCaptor 捕获的是对象引用，所以最后捕获到了同一个空列表。

解决方案：

可以在模拟依赖方法 dataStorage.test 时，保存传入参数的当前值进行验证。代码如下：

```
@Test
public void testReadData() throws Exception {
    // 模拟依赖方法
    ...
    // 模拟依赖方法：dataStorage.test
    List<Object> dataList = new ArrayList<>();
    Predicate<List<Object>> dataStorage = Mockito.mock(Predicate.class);
    Mockito.doAnswer(invocation -> dataList.addAll((List<Object>)invocation.getArgument(0)))
        .when(dataStorage).test(Mockito.anyList());

    // 调用测试方法
    odpsService.readData(recordReader, 2, dataParser, dataStorage);

    // 验证依赖方法
    ...
    // 验证依赖方法：dataStorage.test
    Mockito.verify(dataStorage, Mockito.times(2)).test(Mockito.anyList());
    Assert.assertEquals("数据列表不一致", Arrays.asList(object0, object1, object2), dataList);
}
```

3. 模拟 Lombok 的 log 对象问题

Lombok 的@Slf4j 注解，广泛地应用于 Java 项目中。在某些代码分支里，可能只有 log 记录日志的操作，为了验证这个分支逻辑被正确执行，需要在单元测试用例中对 log 记录日志的操作进行验证。

原始方法：

```
@Slf4j
@Service
public class ExampleService {
    public void recordLog(int code) {
        if (code == 1) {
            log.info("执行分支1");
            return;
        }
        if (code == 2) {
            log.info("执行分支2");
            return;
        }
        log.info("执行默认分支");
    }
    ...
}
```

测试用例：

```
@RunWith(PowerMockRunner.class)
public class ExampleServiceTest {
    @Mock
    private Logger log;
    @InjectMocks
    private ExampleService exampleService;
    @Test
    public void testRecordLog1() {
        exampleService.recordLog(1);
        Mockito.verify(log).info("执行分支1");
    }
}
```

问题现象：

执行单元测试用例失败，抛出以下异常信息：

```
Wanted but not invoked:
logger.info("执行分支1");
```

原因分析：

经过调式跟踪，发现 ExampleService 中的 log 对象并没有被注入。通过编译发现，Lombok 的@Slf4j 注解在 ExampleService 类中生成了一个静态常量 log，而@InjectMocks 注解并不支持静态常量的注入。

解决方案：

采用作者实现的 FieldHelper.setStaticFinalField 方法，可以实现对静态常量的注入模拟对象。

```
@RunWith(PowerMockRunner.class)
public class ExampleServiceTest {
    @Mock
    private Logger log;
    @InjectMocks
    private ExampleService exampleService;
    @Before
    public void beforeTest() throws Exception {
        FieldHelper.setStaticFinalField(ExampleService.class, "log", log);
    }
    @Test
    public void testRecordLog1() {
        exampleService.recordLog(1);
        Mockito.verify(log).info("执行分支1");
    }
}
```

4. 兼容 Pandora 等容器问题

阿里巴巴的很多中间件，都是基于 Pandora 容器的，在编写单元测试用例时，可能会遇到一些坑。

原始方法：

```
@Slf4j
public class MetaqMessageSender {
    @Autowired
    private MetaProducer metaProducer;
    public String sendMetaqMessage(String topicName, String
tagName, String messageKey, String messageBody) {
        try {
            // 组装消息内容
            Message message = new Message();
            message.setTopic(topicName);
            message.setTags(tagName);
            message.setKeys(messageKey);

message.setBody(messageBody.getBytes(StandardCharsets.UTF_8));

            // 发送消息请求
            SendResult sendResult = metaProducer.send(message);
            if (sendResult.getSendStatus() != SendStatus.SEND_OK)
{
                String msg = String.format("发送标签(%s)消息(%s)状态错
误(%s)", tagName, messageKey, sendResult.getSendStatus());
                log.warn(msg);
```

```
                throw new ReconsException(msg);
            }
            log.info(String.format("发送标签(%s)消息(%s)状态成功:%s",
tagName, messageKey, sendResult.getMsgId()));

            // 返回消息标识
            return sendResult.getMsgId();
        } catch (MQClientException | RemotingException |
MQBrokerException | InterruptedException e) {
            // 记录消息异常
            Thread.currentThread().interrupt();
            String message = String.format("发送标签(%s)消息(%s)状态
异常:%s", tagName, messageKey, e.getMessage());
            log.warn(message, e);
            throw new ReconsException(message, e);
        }
    }
}
```

测试用例：

```
@RunWith(PowerMockRunner.class)
public class MetaqMessageSenderTest {
    @Mock
    private MetaProducer metaProducer;
    @InjectMocks
    private MetaqMessageSender metaqMessageSender;
    @Test
    public void testSendMetaqMessage() throws Exception {
        // 模拟依赖方法
        SendResult sendResult = new SendResult();
        sendResult.setMsgId("msgId");
        sendResult.setSendStatus(SendStatus.SEND_OK);
        Mockito.doReturn(sendResult).when(metaProducer).send(Mockito.any(Message.class));

        // 调用测试方法
        String topicName = "topicName";
        String tagName = "tagName";
        String messageKey = "messageKey";
        String messageBody = "messageBody";
        String messageId = metaqMessageSender.sendMetaqMessage(topicName, tagName,
messageKey, messageBody);
        Assert.assertEquals("messageId不一致", sendResult.getMsgId(), messageId);

        // 验证依赖方法
        ArgumentCaptor<Message> messageCaptor = ArgumentCaptor.forClass(Message.class);
        Mockito.verify(metaProducer).send(messageCaptor.capture());
        Message message = messageCaptor.getValue();
        Assert.assertEquals("topicName不一致", topicName, message.getTopic());
        Assert.assertEquals("tagName不一致", tagName, message.getTags());
        Assert.assertEquals("messageKey不一致", messageKey, message.getKeys());
        Assert.assertEquals("messageBody不一致", messageBody, new String(message.getBody()));
    }
}
```

问题现象：

执行单元测试用例失败，抛出以下异常信息：

```
java.lang.RuntimeException: com.alibaba.rocketmq.client.producer.SendResult was loaded by
org.powermock.core.classloader.javassist.JavassistMockClassLoader@5d43661b, it should be
loaded by Pandora Container. Can not load this fake sdk class.
```

原因分析：

基于 Pandora 容器的中间件，需要使用 Pandora 容器加载。在上面测试用例中，使用了 PowerMock 容器加载，从而导致抛出类加载异常。

解决方案：

首先，把 PowerMockRunner 替换为 PandoraBootRunner。其次，为了使@Mock、@InjectMocks 等 Mockito 注解生效，需要加上注解@DelegateTo(MockitoJUnitRunner.class)进行初始化。

```
@RunWith(PandoraBootRunner.class)
@DelegateTo(MockitoJUnitRunner.class)
public class MetaqMessageSenderTest {
  ...
}
```

十二、 消除类型转换警告

在编写测试用例时，特别是泛型类型转换时，很容易产生类型转换警告。常见类型转换警告如下：

```
Type safety: Unchecked cast from Object to List<Object>
Type safety: Unchecked invocation forClass(Class<Map>) of the generic method
forClass(Class<S>) of type ArgumentCaptor
Type safety: The expression of type ArgumentCaptor needs unchecked conversion to conform to
ArgumentCaptor<Map<String,Object>>
```

作为一个有代码洁癖的轻微强迫症程序员，是绝对不容许这些类型转换警告产生的。于是，总结了以下方法来解决这些类型转换警告。

1. 利用注解初始化

Mockito 提供@Mock 注解来模拟类实例，提供@Captor 注解来初始化参数捕获器。由于这些注解实例是通过测试框架进行初始化的，所以不会产生类型转换警告。

问题代码：

```
Map<Long, String> resultMap = Mockito.mock(Map.class);
ArgumentCaptor<Map<String, Object>> parameterMapCaptor = ArgumentCaptor.forClass(Map.class);
```

建议代码：

```
@Mock
private Map<Long, String> resultMap;
@Captor
private ArgumentCaptor<Map<String, Object>> parameterMapCaptor;
```

2. 利用临时类或接口

我们无法获取泛型类或接口的 class 实例，但是很容易获取具体类的 class 实例。这个解决方案的思路是——先定义继承泛型类的具体子类，然后 mock、spy、forClass 以及 any 出这个具体子类的实例，然后把具体子类实例转换为父类泛型实例。

问题代码：

```
Function<Record, Object> dataParser = Mockito.mock(Function.class);
AbstractDynamicValue<Long, Integer> dynamicValue = Mockito.spy(AbstractDynamicValue.class);
ArgumentCaptor<ActionRequest<Void>> requestCaptor =
ArgumentCaptor.forClass(ActionRequest.class);
```

建议代码：

```
/** 定义临时类或接口 */
private interface DataParser extends Function<Record, Object> {};
private static abstract class AbstractTemporaryDynamicValue extends
AbstractDynamicValue<Long, Integer> {};
private static class VoidActionRequest extends ActionRequest<Void> {};

/** 使用临时类或接口 */
Function<Record, Object> dataParser = Mockito.mock(DataParser.class);
AbstractDynamicValue<Long, Integer> dynamicValue =
Mockito.spy(AbstractTemporaryDynamicValue.class);
ArgumentCaptor<ActionRequest<Void>> requestCaptor =
ArgumentCaptor.forClass(VoidActionRequest.class);
```

3. 利用 CastUtils.cast 方法

SpringData 包中提供一个 CastUtils.cast 方法，可以用于类型的强制转换。这个解决方案的思路是——利用 CastUtils.cast 方法屏蔽类型转换警告。

问题代码：

```
Function<Record, Object> dataParser = Mockito.mock(Function.class);
ArgumentCaptor<ActionRequest<Void>> requestCaptor =
ArgumentCaptor.forClass(ActionRequest.class);
Map<Long, Double> scoreMap = (Map<Long, Double>)method.invoke(userService);
```

建议代码：

```
Function<Record, Object> dataParser = CastUtils.cast(Mockito.mock(Function.class));
ArgumentCaptor<ActionRequest<Void>> requestCaptor =
CastUtils.cast(ArgumentCaptor.forClass(ActionRequest.class));
Map<Long, Double> scoreMap = CastUtils.cast(method.invoke(userService));
```

这个解决方案，不需要定义注解，也不需要定义临时类或接口，能够让测试用例代码更为精简，所以作者重点推荐使用。如果不愿意引入 SpringData 包，也可以自己参考实现该方法，只是该方法会产生类型转换警告。

注意：CastUtils.cast 方法本质是——先转换为 Object 类型，再强制转换对应类型，本身不会对类型进行校验。所以，CastUtils.cast 方法好用，但是不要乱用，否则就是大坑（只有执行时才能发现问题）。

4. 利用类型自动转换方法

在 Mockito 中，提供形式如下的方法——泛型类型只跟返回值有关，而跟输入参数无关。这样的方法，可以根据调用方法的参数类型自动转换，而无需手动强制类型转换。如果手动强制类型转换，反而会产生类型转换警告。

```
<T> T getArgument(int index);
public static <T> T any();
public static synchronized <T> T invokeMethod(Object instance, String methodToExecute,
Object... arguments) throws Exception;
```

问题代码：

```
Mockito.doAnswer(invocation -> dataList.addAll((List<Object>)invocation.getArgument(0)))
    .when(dataStorage).test(Mockito.anyList());
Mockito.doThrow(e).when(workflow).beginToPrepare((ActionRequest<Void>)Mockito.any());
Map<Long, Double> scoreMap = (Map<Long, Double>)Whitebox.invokeMethod(userService,
"getScoreMap");
```

建议代码：

```
Mockito.doAnswer(invocation -> dataList.addAll(invocation.getArgument(0)))
    .when(dataStorage).test(Mockito.anyList());
Mockito.doThrow(e).when(workflow).beginToPrepare(Mockito.any());
Map<Long, Double> scoreMap = Whitebox.invokeMethod(userService, "getScoreMap");
```

其实，SpringData 的 CastUtils.cast 方法之所以这么强悍，也是采用了类型自动转化方法。

5. 利用 doReturn-when 语句代替 when-thenReturn 语句

Mockito 的 when-thenReturn 语句需要对返回类型强制校验，而 doReturn-when 语句不会对返回类型强制校验。利用这个特性，可以利用 doReturn-when 语句代替 when-thenReturn 语句解决类型转换警告。

问题代码：

```
List<String> valueList = Mockito.mock(List.class);
Mockito.when(listOperations.range(KEY, start, end)).thenReturn(valueList);
```

建议代码：

```
List<?> valueList = Mockito.mock(List.class);
Mockito.doReturn(valueList).when(listOperations).range(KEY, start, end);
```

6. 利用 Whitebox.invokeMethod 方法代替 Method.invoke 方法

JDK 提供的 Method.invoke 方法返回的是 Object 类型，转化为具体类型时需要强制转换，会产生类型转换警告。而 PowerMock 提供的 Whitebox.invokeMethod 方法返回类型可以自动转化，不会产生类型转换警告。

问题代码：

```
Method method = PowerMockito.method(UserService.class, "getScoreMap");
Map<Long, Double> scoreMap = (Map<Long, Double>)method.invokeMethod(userService);
```

建议代码：

```
Map<Long, Double> scoreMap = Whitebox.invokeMethod(userService, "getScoreMap");
```

7. 利用 instanceof 关键字

在具体类型强制转换时，建议利用 instanceof 关键字先判断类型，否则会产生类型转换警告。

问题代码：

```
JSONArray jsonArray = (JSONArray)object;
...
```

建议代码：

```
if (object instanceof JSONArray) {
    JSONArray jsonArray = (JSONArray)object;
    ...
}
```

8. 利用 Class.cast 方法

在泛型类型强制转换时，会产生类型转换警告。可以采用泛型类的 cast 方法转换，从而避免产生类型转换警告。

问题代码：

```
public static <V> V parseValue(String text, Class<V> clazz) {
    if (Objects.equals(clazz, String.class)) {
        return (V)text;
    }
    return JSON.parseObject(text, clazz);
}
```

建议代码：

```
public static <V> V parseValue(String text, Class<V> clazz) {
    if (Objects.equals(clazz, String.class)) {
        return clazz.cast(text);
    }
    return JSON.parseObject(text, clazz);
}
```

9. 避免不必要的类型转换

有时候，没有必要进行类型转换，就尽量避免类型转换。比如：把 Object 类型转换为具体类型，但又把具体类型当 Object 类型使用，就没有必要进行类型转换。像这种情况，可以合并表达式或定义基类变量，从而避免不必要的类型转化。

问题代码：

```
Boolean isSupper = (Boolean)method.invokeMethod(userService, userId);
Assert.assertEquals("期望值不为真", Boolean.TRUE, isSupper);

List<UserVO> userList = (Map<Long, Double>)method.invokeMethod(userService, companyId);
Assert.assertEquals("期望值不一致", expectedJson, JSON.toJSONString(userList));
```

建议代码：

```
Assert.assertEquals("期望值不为真", Boolean.TRUE, method.invokeMethod(userService, userId));

Object userList = method.invokeMethod(userService, companyId);
Assert.assertEquals("期望值不一致", expectedJson, JSON.toJSONString(userList));
```

后记

登妙峰山记

山高路远车难骑，
精疲力尽人易弃。
多少妙峰登顶者，
又练心境又练力！

骑行的人，一定要沉得住气、要吃得了苦、要耐得住寂寞、要意志坚定不移、要体力够猛够持久……恰好，这也正是技术人所要具备的精神。只要技术人做到了这些，练就了好的“心境”和“体力”，才有可能登上技术的“妙峰山”。

编写技巧：Java 单元测试技巧之 JSON 序列化

前言

《论语》中孔子有言：“工欲善其事，必先利其器。”

今年 7 月，作者希望迎接更大的挑战，从高德地图数据转岗到共享出行后，接手并维护了几个 Java 后端项目。在熟悉业务和代码的过程中，快速地对原有项目进行单元测试用例的补充，使其单元测试覆盖率达到 70%+甚至于 100%。有同事问我：“你写单元测试为什么这么快？”我微微一笑：“工欲善其事，必先利其器。而我快速编写 Java 单元测试用例的技巧就是——JSON 序列化。”

是的，做任何事情，都要讲究方式方法；只要方式方法对了，就会事半功倍。这里，作者系统性地总结了 JSON 序列化在编写 Java 单元测试用例中的使用技巧，希望能够让大家“读有所得、得有所思、思有所获”。

一、 冗长的单元测试代码

在编写单元测试用例的过程中，经常会出现以下冗长的单元测试代码。

1. 冗长的数据模拟代码

1) 模拟类属性值

在模拟类属性值时，会遇到以下的冗长代码：

```
Map<Long, String> languageMap = new HashMap<>(MapHelper.DEFAULT);
languageMap.put(1L, "Java");
languageMap.put(2L, "C++");
languageMap.put(3L, "Python");
languageMap.put(4L, "JavaScript");
... // 约几十行
Whitebox.setInternalState(developmentService, "languageMap", languageMap);
```

2) 模拟方法参数值

在模拟方法参数值时，会遇到以下的冗长代码：

```
List<UserCreateVO> userCreateList = new ArrayList<>();
UserCreateVO userCreate0 = new UserCreateVO();
userCreate0.setName("Changyi");
userCreate0.setTitle("Java Developer");
... // 约几十行
userCreateList.add(userCreate0);
UserCreateVO userCreate1 = new UserCreateVO();
userCreate1.setName("Tester");
userCreate1.setTitle("Java Tester");
... // 约几十行
userCreateList.add(userCreate1);
... // 约几十条
userService.batchCreate(userCreateList);
```

3) 模拟方法返回值

在模拟方法返回值时，会遇到以下的冗长代码：

```
Long companyId = 1L;
List<UserDO> userList = new ArrayList<>();
UserDO user0 = new UserDO();
user0.setId(1L);
user0.setName("Changyi");
user0.setTitle("Java Developer");
... // 约几十行
userList.add(user0);
UserDO user1 = new UserDO();
user1.setId(2L);
user1.setName("Tester");
user1.setTitle("Java Tester");
... // 约几十行
userList.add(user1);
... // 约几十条
Mockito.doReturn(userList).when(userDAO).queryByCompanyId(companyId);
```

2. 冗长的数据验证代码

1) 验证方法返回值

在验证方法返回值时，会遇到以下的冗长代码：

```
Long companyId = 1L;
List<UserVO> userList = userService.queryByCompanyId(companyId);
UserVO user0 = userList.get(0);
Assert.assertEquals("name不一致", "Changyi", user0.getName());
Assert.assertEquals("title不一致", "Java Developer", user0.getTitle());
... // 约几十行
UserVO user1 = userList.get(1);
Assert.assertEquals("name不一致", "Tester", user1.getName());
Assert.assertEquals("title不一致", "Java Tester", user1.getTitle());
... // 约几十行
... // 约几十条
```

2) 验证方法参数值

在验证方法参数值时，会遇到以下的冗长代码：

```
ArgumentCaptor<List<UserDO>> userCreateListCaptor =
CastUtils.cast(ArgumentCaptor.forClass(List.class));
Mockito.verify(userDAO).batchCreate(userCreateListCaptor.capture());
List<UserDO> userCreateList = userCreateListCaptor.getValue();
UserDO userCreate0 = userCreateList.get(0);
Assert.assertEquals("name不一致", "Changyi", userCreate0.getName());
Assert.assertEquals("title不一致", "Java Developer", userCreate0.getTitle());
... // 约几十行
UserDO userCreate1 = userCreateList.get(1);
Assert.assertEquals("name不一致", "Tester", userCreate1.getName());
Assert.assertEquals("title不一致", "Java Tester", userCreate1.getTitle());
... // 约几十行
... // 约几十条
```

二、 采用 JSON 序列化简化

常言道：“眼见为实，耳听为虚。”下面，就通过 JSON 序列化来简化上面的单元测试用例代码，让大家先睹为快。

1. 简化数据模拟代码

对于数据模拟，首先需要先加载 JSON 资源文件为字符串，然后通过 JSON 反序列化字符串为数据对象，最后用于模拟类属性值、方法参数值和方法返回值。这样，就精简了原来冗长的赋值语句。

1) 模拟类属性值

利用 JSON 反序列化，简化模拟类属性值代码如下：

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "languageMap.json");
Map<Long, String> languageMap = JSON.parseObject(text, new TypeReference<Map<Long, String>>
() {});
Whitebox.setInternalState(mobilePhoneService, "languageMap", languageMap);
```

其中，JSON 资源文件 languageMap.json 的内容如下：

```
{1:"Java",2:"C++",3:"Python",4:"JavaScript"...}
```

2) 模拟方法参数值

利用 JSON 反序列化，简化模拟方法参数值代码如下：

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateList.json");
List<UserCreateVO> userCreateList = JSON.parseArray(text, UserCreateVO.class);
userService.batchCreate(userCreateList);
```

其中，JSON 资源文件 userCreateList.json 的内容如下：

```
[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]
```

3) 模拟方法返回值

利用 JSON 反序列化，简化模拟方法返回值代码如下：

```
Long companyId = 1L;
String text = ResourceHelper.getResourceAsString(getClass(), path + "userList.json");
List<UserDO> userList = JSON.parseArray(text, UserDO.class);
Mockito.doReturn(userList).when(userDAO).queryByCompanyId(companyId);
```

其中，JSON 资源文件 userList.json 的内容如下：

```
[{"id":1,"name":"Changyi","title":"Java Developer"...},{"id":2,"name":"Tester","title":"Java
Tester"...},...]
```

2. 简化数据验证代码

对于数据验证，首先需要先加载 JSON 资源文件为字符串，然后通过 JSON 序列化数据对象为字符串，最后验证两字符串是否一致。这样，就精简了原来冗长的验证语句。

1) 验证方法返回值

利用 JSON 序列化，简化验证方法返回值代码如下：

```
Long companyId = 1L;
List<UserVO> userList = userService.queryByCompanyId(companyId);
String text = ResourceHelper.getResourceAsString(getClass(), path + "userList.json");
Assert.assertEquals("用户列表不一致", text, JSON.toJSONString(userList));
```

其中，JSON 资源文件 userList.json 的内容如下：

```
[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]
```

2) 验证方法参数值

利用 JSON 序列化，简化验证方法参数值代码如下：

```
ArgumentCaptor<List<UserDO>> userCreateListCaptor =
CastUtils.cast(ArgumentCaptor.forClass(List.class));
Mockito.verify(userDAO).batchCreate(userCreateListCaptor.capture());
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateList.json");
Assert.assertEquals("用户创建列表不一致", text,
JSON.toJSONString(userCreateListCaptor.getValue()));
```

其中，JSON 资源文件 userCreateList.json 的内容如下：

```
[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]
```

三、 测试用例及资源命名

俗话说：“没有规矩，不成方圆。”所以，为了更好地利用 JSON 序列化技巧，首先对测试用例和资源文件进行规范化命名。

1. 测试类命名

按照行业惯例，测试类的命名应以被测试类名开头并以 Test 结尾。比如：UserService(用户服务类)的测试类需要命名为 UserServiceTest(用户服务测试类)。

单元测试类应该放在被测试类的同一工程的"src/test/java"目录下,并且要放在被测试类的同一包下。注意，单元测试类不允许写在业务代码目录下，否则在编译时没法过滤这些测试用例。

2. 测试方法命名

按照行业规范，测试方法命名应以 test 开头并以被测试方法结尾。比如：batchCreate(批量创建)的测试方法需要命名为 testBatchCreate(测试：批量创建)，queryByCompanyId(根据公司标识查询)的测试方法需要命名为 testQueryByCompanyId(测试：根据公司标识查询)。

当一个方法对应多个测试用例时，就需要创建多个测试方法，原有测试方法命名已经不能满足需求了。有人建议在原有的测试方法命名的基础上，添加 123 等序号表示不同的用例。比如：testBatchCreate1(测试：批量创建 1)、testBatchCreate2(测试：批量创建 2).....但是，这种方法不能明确每个单元测试的用意。

这里，作者建议在原有的测试方法命名的基础上，添加”With+条件“来表达不同的测试用例方法。

1) 按照结果命名

- testBatchCreateWithSuccess（测试：批量创建-成功）
- testBatchCreateWithFailure（测试：批量创建-失败）
- testBatchCreateWithException（测试：批量创建-异常）

2) 按照参数命名

- testBatchCreateWithListNull（测试：批量创建-列表为 NULL）
- testBatchCreateWithListEmpty（测试：批量创建-列表为空）
- testBatchCreateWithListNotEmpty（测试：批量创建-列表不为空）

3) 按照意图命名

- testBatchCreateWithNormal（测试：批量创建-正常）
- testBatchCreateWithGray（测试：批量创建-灰度）
- testBatchCreateWithException（测试：批量创建-异常）

当然,还有形成其它的测试方法命名方式,也可以把不同的测试方法命名方式混用,只要能清楚地表达出这个测试用例的涵义即可。

3. 测试类资源目录命名

这里,作者建议的资源目录命名方式为——以 test 开头且以被测试类名结尾。比如：UserService(用户服务类)的测试资源目录可以命名为 testUserService。

那么，这个资源目录应该放在哪儿了？作者提供了 2 个选择：

- 放在“src/test/java”目录下,跟测试类放在同一目录下——这是作者最喜欢的方式；
- 放在“src/test/resources”目录下,跟测试类放在同一目录下——建议 IDEA 用户采用这种方式。

4. 测试方法资源目录命名

在前面的小节中，我们针对测试方法进行了规范命名。这里，我们可以直接拿来使用——即用测试方法名称来命名测试目录。当然，这些测试方法资源目录应该放在测试类资源目录下。比如：测试类 UserServiceTest（用户服务测试类）的测试方法 testBatchCreateWithSuccess（测试：批量创建-成功）的测试资源目录就是 testUserService/testBatchCreateWithSuccess。

另外，也可以采用“测试方法名称”+“测试条件名称”二级目录的命名方式。比如：测试类 UserServiceTest（用户服务测试类）的测试方法 testBatchCreateWithSuccess（测试：批量创建-成功）的测试资源目录就是 testUserService/testBatchCreate/success。

这里，作者首推的是第一种方式，因为测试方法名称和资源目录名称能够保持一致。

5. 测试资源文件命名

在被测试代码中，所有参数、变量都已经有了命名。所以，建议优先使用这些参数和变量的名称，并加后缀“.json”标识文件格式。如果这些资源文件名称冲突，可以添加前缀以示区分。比如：userCreateList 的资源文件名称为"userCreateList.json"。

另外，在测试用例代码中，把这些测试资源文件加载后，反序列化为对应的数据对象，这些数据对象的变量名称也应该跟资源文件名称保持一致。

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateList.json");
List<UserCreateVO> userCreateList = JSON.parseArray(text, UserCreateVO.class);
userService.batchCreate(userCreateList);
```

6. 测试资源文件存储

在测试资源目录和名称定义好之后，就需要存入测试资源文件了。存储方式总结如下：

- 如果是测试类下所有测试用例共用的资源文件，建议存储在测试类资源目录下，比如：testUserService；
- 如果是测试用例独有的资源文件，建议存储在测试方法资源目录下，比如：testUserService/testBatchCreateWithSuccess；
- 如果是某一被测方法所有的测试用例共用的资源文件，建议存储在不带任何修饰的测试方法资源目录下，比如：testUserService/testBatchCreate；
- 如果测试类资源目录下只有一个测试方法资源目录，可以去掉这个测试方法资源目录，把所有资源文件存储在测试类资源目录下。

注意：这里的资源文件不光是 JSON 资源文件，但也可以是其它类型的资源文件。

7. Git 文件名称过长

由于资源目录名称较长（大概超过 50 个字符），可能会导致 git 检出代码时出现以下错误：

```
git checkout develop
error: xxx/xxx: Filename too long
```

或者，在添加文件时出现以下错误：

```
git add .
error: open("xxx/xxx"): Filename too long
error: unable to index file 'xxx/xxx'
fatal: adding files failed
```

可以通过以下 git 设置参数解决：

```
git config --system core.longpaths true
```

当然，测试用例名称和资源目录名称没必要太长，可以进行一些精简使其小于等于 50 个字符。

8. JSON 资源文件格式

关于 JSON 资源文件是否格式化的建议：不要格式化 JSON 资源文件内容，否则会占用更多的代码行数，还会导致无法直接进行文本比较。

四、 测试资源使用案例

在上一章中，讲了测试用例和资源的命名规则以及存放方式。但是，只是文字的描述，没有什么体感。所有，这一章将举例一个完整的案例来实际说明。

1. 被测案例代码

以 UserService 的 createUser 方法为例说明：

```
/**
 * 用户服务类
 */
@Service
public class UserService {

    /** 服务相关 */
    /** 用户 DAO */
    @Autowired
    private UserDAO userDAO;
    /** 标识生成器 */
    @Autowired
    private IdGenerator idGenerator;

    /** 参数相关 */
    /** 可以修改 */
    @Value("${userService.canModify}")
    private Boolean canModify;

    /**
     * 创建用户
     *
     * @param userCreate 用户创建
     * @return 用户标识
     */
    public Long createUser(UserVO userCreate) {
        // 获取用户标识
        Long userId = userDAO.getIdByName(userCreate.getName());

        // 根据存在处理
        // 根据存在处理：不存在则创建
        if (Objects.isNull(userId)) {
            userId = idGenerator.next();
            UserDO userCreateDO = new UserDO();
            userCreateDO.setId(userId);
            userCreateDO.setName(userCreate.getName());
            userDAO.create(userCreateDO);
        }
        // 根据存在处理：已存在可修改
        else if (Boolean.TRUE.equals(canModify)) {
```

```
            UserDO userModifyDO = new UserDO();
            userModifyDO.setId(userId);
            userModifyDO.setName(userCreate.getName());
            userDAO.modify(userModifyDO);
        }
        // 根据存在处理：已存在禁修改
        else {
            throw new UnsupportedOperationException("不支持修改");
        }

        // 返回用户标识
        return userId;
    }

}
```

2. 测试用例代码

编写完整的测试用例如下：

```
/**
 * 用户服务测试类
 */
@RunWith(PowerMockRunner.class)
public class UserServiceTest {

    /** 模拟依赖对象 */
    /** 用户 DAO */
    @Mock
    private UserDAO userDAO;
    /** 标识生成器 */
    @Mock
    private IdGenerator idGenerator;

    /** 定义测试对象 */
    /** 用户服务 */
    @InjectMocks
    private UserService userService;

    /** 定义静态常量 */
    /** 资源路径 */
    private static final String RESOURCE_PATH =
"testUserService/";
```

```
    /**
     * 在测试之前
     */
    @Before
    public void beforeTest() {
        // 注入依赖对象
        Whitebox.setInternalState(userService, "canModify",
Boolean.TRUE);
    }

    /**
     * 测试：创建用户-创建
     */
    @Test
    public void testCreateUserWithCreate() {
        // 模拟依赖方法
        // 模拟依赖方法：userDAO.getByName

Mockito.doReturn(null).when(userDAO).getIdByName(Mockito.anyStri
ng());
        // 模拟依赖方法：idGenerator.next
        Long userId = 1L;
        Mockito.doReturn(userId).when(idGenerator).next();

        // 调用测试方法
        String path = RESOURCE_PATH +
"testCreateUserWithCreate/";
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"userCreateVO.json");
        UserVO userCreate = JSON.parseObject(text, UserVO.class);
        Assert.assertEquals("用户标识不一致", userId,
userService.createUser(userCreate));

        // 验证依赖方法
        // 验证依赖方法：userDAO.getByName

Mockito.verify(userDAO).getIdByName(userCreate.getName());
        // 验证依赖方法：idGenerator.next
        Mockito.verify(idGenerator).next();
        // 验证依赖方法：userDAO.create
        ArgumentCaptor<UserDO> userCreateCaptor =
ArgumentCaptor.forClass(UserDO.class);

Mockito.verify(userDAO).create(userCreateCaptor.capture());
```

```
        text = ResourceHelper.getResourceAsString(getClass(),
path + "userCreateDO.json");
        Assert.assertEquals("用户创建不一致", text,
JSON.toJSONString(userCreateCaptor.getValue()));

        // 验证依赖对象
        // 验证依赖对象：idGenerator, userDAO
        Mockito.verifyNoMoreInteractions(idGenerator, userDAO);
    }

    /**
     * 测试：创建用户-修改
     */
    @Test
    public void testCreateUserWithModify() {
        // 模拟依赖方法
        // 模拟依赖方法：userDAO.getByName
        Long userId = 1L;

Mockito.doReturn(userId).when(userDAO).getIdByName(Mockito.anySt
ring());

        // 调用测试方法
        String path = RESOURCE_PATH +
"testCreateUserWithModify/";
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"userCreateVO.json");
        UserVO userCreate = JSON.parseObject(text, UserVO.class);
        Assert.assertEquals("用户标识不一致", userId,
userService.createUser(userCreate));

        // 验证依赖方法
        // 验证依赖方法：userDAO.getByName

Mockito.verify(userDAO).getIdByName(userCreate.getName());
        // 验证依赖方法：userDAO.modify
        ArgumentCaptor<UserDO> userModifyCaptor =
ArgumentCaptor.forClass(UserDO.class);

Mockito.verify(userDAO).modify(userModifyCaptor.capture());
        text = ResourceHelper.getResourceAsString(getClass(),
path + "userModifyDO.json");
        Assert.assertEquals("用户修改不一致", text,
JSON.toJSONString(userModifyCaptor.getValue()));
```

```
        // 验证依赖对象
        // 验证依赖对象：idGenerator
        Mockito.verifyZeroInteractions(idGenerator);
        // 验证依赖对象：userDAO
        Mockito.verifyNoMoreInteractions(userDAO);
    }

    /**
     * 测试：创建用户-异常
     */
    @Test
    public void testCreateUserWithException() {
        // 注入依赖对象
        Whitebox.setInternalState(userService, "canModify",
Boolean.FALSE);

        // 模拟依赖方法
        // 模拟依赖方法：userDAO.getByName
        Long userId = 1L;

Mockito.doReturn(userId).when(userDAO).getIdByName(Mockito.anySt
ring());

        // 调用测试方法
        String path = RESOURCE_PATH +
"testCreateUserWithException/";
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"userCreateVO.json");
        UserVO userCreate = JSON.parseObject(text, UserVO.class);
        UnsupportedOperationException exception =
Assert.assertThrows("返回异常不一致",
            UnsupportedOperationException.class, () ->
userService.createUser(userCreate));
        Assert.assertEquals("异常消息不一致", "不支持修改",
exception.getMessage());

        // 验证依赖方法
        // 验证依赖方法：userDAO.getByName

Mockito.verify(userDAO).getIdByName(userCreate.getName());

        // 验证依赖对象
        // 验证依赖对象：idGenerator
```

```
        Mockito.verifyZeroInteractions(idGenerator);
        // 验证依赖对象：userDAO
        Mockito.verifyNoMoreInteractions(userDAO);
    }

}
```

3. 资源文件目录

测试用例所涉及的资源文件目录如下：

其中，资源文件内容比较简单，这里就不再累述了。

4. POM 文件配置

根项目的 pom.xml 文件需要做以下配置：

```
<?xml version="1.0" encoding="UTF-8" ?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
    ...

    <!-- 属性管理 -->
    <properties>
        ...
```

```
    <junit.version>4.13.1</junit.version>
    <mockito.version>3.3.3</mockito.version>
    <powermock.version>2.0.9</powermock.version>
</properties>

<!-- 依赖管理 -->
<dependencyManagement>
    <dependencies>
        ...
        <!-- PowerMock -->
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.mockito</groupId>
            <artifactId>mockito-core</artifactId>
            <version>${mockito.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.powermock</groupId>
            <artifactId>powermock-module-junit4</artifactId>
            <version>${powermock.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.powermock</groupId>
            <artifactId>powermock-api-mockito2</artifactId>
            <version>${powermock.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</dependencyManagement>

<!-- 构建管理 -->
<build>
    <pluginManagement>
        <plugins>
            ...
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-resources-plugin</artifactId>
```

```
                <version>2.6</version>
                <executions>
                    ...
                    <execution>
                        <id>copy-test-resources</id>
                        <phase>compile</phase>
                        <goals>
                            <goal>copy-resources</goal>
                        </goals>
                        <configuration>
                            <encoding>UTF-8</encoding>

<outputDirectory>${project.build.directory}/test-
classes</outputDirectory>
                            <resources>
                                <resource>

<directory>src/test/java</directory>
                                    <includes>
                                        <include>**/*.txt</include>
                                        <include>**/*.csv</include>

<include>**/*.json</include>

<include>**/*.properties</include>
                                    </includes>
                                </resource>
                                <resource>

<directory>src/test/resources</directory>
                                    <includes>
                                        <include>**/*.txt</include>
                                        <include>**/*.csv</include>

<include>**/*.json</include>

<include>**/*.properties</include>
                                    </includes>
                                </resource>
                            </resources>
                        </configuration>
                    </execution>
                </executions>
            </plugin>
        </plugins>
```

```
        </pluginManagement>
    </build>
</project>
```

简要说明如下：

- 在属性配置中，配置了单元测试所依赖的包版本；
- 在依赖配置中，配置了单元测试所依赖的包名称；
- 在构建配置中，配置了编译时需要拷贝目录下的资源文件（如果有其它的资源文件格式，需要在 pom 中配置添加）。

5. 工具类代码

在上面单元测试用例中，需要使用到一个工具类 ResourceHelper（资源赋值类），代码如下：

```
/**
 * 资源辅助类
 */
public final class ResourceHelper {

    /**
     * 构造方法
     */
    private ResourceHelper() {
        throw new UnsupportedOperationException();
    }

    /**
     * 以字符串方式获取资源
     *
     * @param clazz 类
     * @param name 资源名称
     * @return 字符串
     */
    public static <T> String getResourceAsString(Class<T> clazz, String name) {
        try (InputStream is = clazz.getResourceAsStream(name)) {
            return IOUtils.toString(is, StandardCharsets.UTF_8);
        } catch (IOException e) {
            throw new RuntimeException(String.format("以字符串方式获取资源(%s)异常", name), e);
        }
    }

}
```

如果在集团内部，也可以直接导入作者提供的二方工具包：

```
<dependency>
  <groupId>com.amap</groupId>
  <artifactId>aostools-util</artifactId>
  <version>1.0.1</version>
</dependency>
```

五、 JSON 资源文件的来源

JSON 资源文件来源方式很多，作者根据实际操作经验，总结出以下几种以供大家参考。

1. 来源于自己组装

直接利用 JSON 编辑器或者纯文本编辑器，自己一个字段一个字段地编写 JSON 资源数据。

```
[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]
```

注意：这种方式容易出现 JSON 格式错误及字符串转义问题。

2. 来源于代码生成

做为程序员，能够用程序生成 JSON 资源数据，就绝不手工组装 JSON 资源数据。下面，便是利用 Fastjson 的 JSON.toJSONString 方法生成 JSON 资源数据。

```
public static void main(String[] args) {
    List<UserCreateVO> userCreateList = new ArrayList<>();
    UserCreateVO userCreate0 = new UserCreateVO();
    userCreate0.setName("Changyi");
    userCreate0.setTitle("Java Developer");
    ... // 约几十行
    userCreateList.add(userCreate0);
    UserCreateVO userCreate1 = new UserCreateVO();
    userCreate1.setName("Tester");
    userCreate1.setTitle("Java Tester");
    ... // 约几十行
    userCreateList.add(userCreate1);
    ... // 约几十条
    System.out.println(JSON.toJSONString(userCreateList));
}
```

执行该程序后，生成的 JSON 资源数据如下：

```
[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]
```

注意：这种方式能够避免 JSON 格式错误及字符串转义问题。

3. 来源于线上日志

如果是事后补充单元测试，首先想到的就是利用线上日志。比如：

```
2021-08-31 18:55:40,867 INFO [UserService.java:34] - 根据公司标识(1)查询所有用户:
[{"id":1,"name":"Changyi","title":"Java Developer"...},{"id":2,"name":"Tester","title":"Java
Tester"...},...]
```

从上面的日志中，我们可以得到方法 userDAO.queryByCompanyId 的请求参数 companyId 取值为 "1"，返回结果为 “[{"id":1,"name":"Changyi","title":"Java Developer"...},{"id":2,"name":"Tester","title":"Java Tester"...},...]” 。

注意：要想得到现成的 JSON 资源数据，就必须输出完整的 JSON 数据内容。但是，由于 JSON 数据内容过大，一般不建议全部输出。所以，从线上日志中也不一定能够拿到现成的 JSON 资源数据。

4. 来源于集成测试

集成测试，就是把整个或部分项目环境运行起来，能够连接数据库、Redis、MetaQ、HSF 等所依赖的第三方服务环境，然后测试某一个方法的功能是否能够达到预期。

```
/**
 * 用户DAO测试类
 */
@Slf4j
@RunWith(PandoraBootRunner.class)
@DelegateTo(SpringJUnit4ClassRunner.class)
@SpringBootTest(classes = {ExampleApplication.class})
public class UserDaoTest {

    /** 用户DAO */
    @Resource
    private UserDAO userDAO;

    /**
     * 测试：根据公司标识查询
     */
    @Test
    public void testQueryByCompanyId() {
        Long companyId = 1L;
        List<UserDO> userList = userDAO.queryByCompanyId(companyId);
        log.info("userList={}", JSON.toJSONString(userList));
    }

}
```

执行上面集成测试用例，输出的日志内容如下：

```
2021-08-31 18:55:40,867 INFO [UserDaoTest.java:24] - userList=
[{"id":1,"name":"Changyi","title":"Java Developer"...},{"id":2,"name":"Tester","title":"Java
Tester"...},...]
```

上面日志中，userList 后面的就是我们需要的 JSON 资源数据。

我们也可以用集成测试得到方法内部的方法调用的参数值和返回值，具体方法如下：

- 首先，在源代码中添加日志输出语句；
- 然后，执行单元测试用例，得到对应的方法调用参数值和返回值；
- 最后，删除源代码中日志输出语句，恢复源代码为原来的样子。

5. 来源于测试过程

有一些数据，是由被测方法生成的，比如：方法返回值和调用参数。针对这类数据，可以在测试过程中生成，然后逐一进行数据核对，最后整理成 JSON 资源文件。

被测方法：

```
public void batchCreate(List<UserCreate> createList) {
    List<UserDO> userList = createList.stream()
        .map(UserService::convertUser).collect(Collectors.toList());
    userDAO.batchCreate(userList);
}
```

测试用例：

```
@Test
public void testBatchCreate() {
    // 调用测试方法
    List<UserCreate> createList = ...;
    userService.batchCreate(createList);

    // 验证测试方法
    ArgumentCaptor<List<UserDO>> userListCaptor =
CastUtils.cast(ArgumentCaptor.forClass(List.class));
    Mockito.verify(userDAO).batchCreate(userListCaptor.capture());
    Assert.assertEquals("用户列表不一致", "", JSON.toJSONString(userListCaptor.getValue()));
}
```

执行单元测试后，提示以下问题：

```
org.junit.ComparisonFailure: 用户列表不一致 expected:<[]> but was:
<[[{"name":"Changyi","title":"Java Developer"...},{"name":"Tester","title":"Java
Tester"...},...]]>
```

上面的错误信息中，后面括号中的就是我们需要需要的 JSON 资源数据。

注意：一定要进行数据核对，这有可能是错误代码生成的错误数据。用错误数据去验证生成它的代码，当然不会测试出其中的问题。

六、 JSON 序列化技巧

这里以 Fastjson 为例，介绍一些 JSON 序列化技巧。

1. 序列化对象

利用 JSON.toJSONString 方法序列化对象：

```
UserVO user = ...;
String text = JSON.toJSONString(user);
```

2. 序列化数组

利用 JSON.toJSONString 方法序列化数组：

```
UserVO[] users = ...;
String text = JSON.toJSONString(users);
```

3. 序列化集合

利用 JSON.toJSONString 方法序列化集合（继承至 Collection，比如 List、Set 等集合）：

```
List<UserVO> userList = ...;
String text = JSON.toJSONString(userList);
```

4. 序列化映射

利用 JSON.toJSONString 方法序列化映射：

```
Map<Long, UserVO> userMap = ...;
String text = JSON.toJSONString(userMap, SerializerFeature.MapSortField);
```

其中，为了保证每次序列化的映射字符串一致，需要指定序列化参数 MapSortField 进行排序。

5. 序列化模板对象

利用 JSON.toJSONString 方法序列化模板对象：

```
Result<UserVO> result = ...;
String text = JSON.toJSONString(result);
```

6. 序列化指定属性字段

利用 JSON.toJSONString 方法序列化指定属性字段，主要通过设置属性预过滤器（SimplePropertyPreFilter）的包含属性字段列表（includes）实现。主要应用于只想验证某些字段的情况，比如只验证跟测试用例有关的字段。

1) 指定所有类的属性字段

利用 JSON.toJSONString 方法序列化指定所有类的属性字段：

```
UserVO user = ...;
SimplePropertyPreFilter filter = new SimplePropertyPreFilter();
filter.getIncludes().addAll(Arrays.asList("id", "name"));
String text = JSON.toJSONString(user, filter);
```

2) 指定单个类的属性字段

利用 JSON.toJSONString 方法序列化指定单个类的属性字段：

```
List<UserVO> userList = ...;
SimplePropertyPreFilter filter = new SimplePropertyPreFilter(UserVO.class);
filter.getIncludes().addAll(Arrays.asList("id", "name"));
String text = JSON.toJSONString(userList, filter);
```

3) 指定多个类的属性字段

利用 JSON.toJSONString 方法序列化指定多个类的属性字段：

```
Pair<UserVO, CompanyVO> userCompanyPair = ...;
SimplePropertyPreFilter userFilter = new SimplePropertyPreFilter(UserVO.class);
userFilter.getUncludes().addAll(Arrays.asList("id", "name"));
SimplePropertyPreFilter companyFilter = new SimplePropertyPreFilter(CompanyVO.class);
companyFilter.getIncludes().addAll(Arrays.asList("id", "name"));
String text = JSON.toJSONString(userCompanyPair, new SerializeFilter[]{userFilter,
companyFilter});
```

7. 序列化字段排除属性字段

利用 JSON.toJSONString 方法序列化过滤属性字段，主要通过设置属性预过滤器（SimplePropertyPreFilter）的排除属性字段列表（excludes）实现。主要应用于不想验证某些字段的情况，比如排除无法验证的随机属性字段。

1) 排除所有类的属性字段

利用 JSON.toJSONString 方法序列化排除所有类的属性字段：

```
UserVO user = ...;
SimplePropertyPreFilter filter = new SimplePropertyPreFilter();
filter.getExcludes().addAll(Arrays.asList("gmtCreate", "gmtModified"));
String text = JSON.toJSONString(user, filter);
```

2) 排除单个类的属性字段

利用 JSON.toJSONString 方法序列化排除单个类的属性字段：

```
List<UserVO> userList = ...;
SimplePropertyPreFilter filter = new SimplePropertyPreFilter(UserVO.class);
filter.getExcludes().addAll(Arrays.asList("gmtCreate", "gmtModified"));
String text = JSON.toJSONString(userList, filter);
```

3) 排除多个类的属性字段

利用 JSON.toJSONString 方法序列化排除多个类的属性字段：

```
Pair<UserVO, CompanyVO> userCompanyPair = ...;
SimplePropertyPreFilter userFilter = new SimplePropertyPreFilter(UserVO.class);
userFilter.getExcludes().addAll(Arrays.asList("gmtCreate", "gmtModified"));
SimplePropertyPreFilter companyFilter = new SimplePropertyPreFilter(CompanyVO.class);
companyFilter.getExcludes().addAll(Arrays.asList("createTime", "modifyTime"));
String text = JSON.toJSONString(userCompanyPair, new SerializeFilter[]{userFilter,
companyFilter});
```

8. 自定义序列化

对应一些类对象，需要序列化为特殊格式文本，就必须自定义序列化器。比如：Geometry 序列化文本，通常采用 WKT(Well-known text)表示，便于用户快速阅读理解。

1) 全局配置序列化器

通过 JSON 序列化全局配置指定类序列化器：

```
Geometry geometry = ...;
SerializeConfig.getGlobalInstance().put(Geometry.class, new GeometrySerializer());
String text = JSON.toJSONString(geometry);
```

注意：这种方式不支持类继承，必须指定到具体类。比如要序列化 Point 对象，就必须配置 Point 类的序列化器。

2) 特定配置序列化器

通过 JSON 序列化特定配置指定类序列化器：

```
Geometry geometry = ...;
SerializeConfig config = new SerializeConfig();
config.put(Geometry.class, new GeometrySerializer());
String text = JSON.toJSONString(geometry, config);
```

注意：这种方式不支持类继承，必须指定到具体类。比如要序列化 Point 对象，就必须配置 Point 类的序列化器。

3) 注解配置序列化器

通过 JSON 序列化注解配置指定类序列化器：

```
public class User {
    ...
    @JSONField(serializeUsing = GeometrySerializer.class)
    private Geometry location;
    ...
}

User user = ...;
String text = JSON.toJSONString(user);
```

其中：GeometrySerializer 为自定义类，这里就不贴出具体实现了。

9. 禁用对象自引用检测

如果实体类中存在对象自引用（即多个属性字段指向同一个对象引用），在利用 JSON.toJSONString 方法序列化会出现如下内容：

```
{"userInfo":{"additions":[...],...},"addition":{"$ref":"$.userInfo.additions[0]"}}
```

其中，属性字段 addition 指向了属性字段 userInfo 的属性字段 additions 的第 1 个对象。

但是，这样的 JSON 字符串，可能无法与我们提供的标准 JSON 字符串对比。这里，Fastjson 提供了序列化参数 SerializerFeature.DisableCircularReferenceDetect（禁用循环引用检测）来禁用这个功能。

```
UserWapper userWapper = ...;
String text = JSON.toJSONString(userWapper,
SerializerFeature.DisableCircularReferenceDetect);
```

注意：实体对象中千万不要存在对象循环引用的情况。

七、 JSON 反序列化技巧

这里以 Fastjson 为例，介绍一些 JSON 反序列化技巧。

1. 反序列化对象

利用 JSON.parseObject 方法反序列化对象：

```
String text = ...;
UserVO user = JSON.parseObject(text, UserVO.class);
```

2. 反序列化数组

利用 JSON.parseObject 方法反序列化数组：

```
String text = ...;
UserVO[] users = JSON.parseObject(text, UserVO[].class);
```

3. 反序列化集合

利用 JSON.parseArray 方法反序列化列表：

```
String text = ...;
List<UserVO> userList = JSON.parseArray(text, UserVO.class);
```

利用 JSON.parseObject 方法反序列化集合：

```
String text = ...;
Set<UserVO> userSet = JSON.parseObject(text, new TypeReference<Set<UserVO>>() {});
```

4. 反序列化映射

利用 JSON.parseObject 方法反序列化映射：

```
String text = ...;
Map<Long, UserVO> userList = JSON.parseObject(text, new TypeReference<Map<Long, UserVO>>()
{});
```

注意：如果映射的 key 是复杂类型，这种方法反序列会报格式错误，需要自定义反序列化器。

5. 反序列化模板对象

利用 JSON.parseObject 方法反序列化模板对象：

```
String text = ...;
Result<UserVO> result = JSON.parseArray(text, new TypeReference<Result<UserVO>>() {});
```

6. 反序列化非公有字段

由于某些属性字段没有公有设置方法，或者以字段名称作为公有设置方法。当需要反序列化这些属性字段时，需要指定 SupportNonPublicField（支持非公有字段）反序列化参数。

```
String text = ...;
UserVO user = JSON.parseObject(text, UserVO.class, Feature.SupportNonPublicField);
```

7. 反序列化 Builder 模式类

有些同学喜欢用 Builder 模式，导致实体类并没有公有构造方法。当利用 Fastjson 反序列化这些类是，就会出现以下问题：

```
com.alibaba.Fastjson.JSONException: default constructor not found. class com.example.User
```

只要对应的 Builder 类有默认构造方法，就可以采用下面的方式序列化。

```
String text = ...;
User user = JSON.parseObject(text, User.UserBuilder.class,
Feature.SupportNonPublicField).build();
```

首先通过 JSON.parseObject 方法+SupportNonPublicField 参数反序列化 Builder 对象，然后通过 Builder 对象的 build 方法来构造实体对象。

如果对应的 Builder 类没有默认构造方法，或者需要反序列化模板对象时，需要自定义 JSON 反序列化器。

8. 反序列化丢失字段值

Fastjson 支持没有默认构造方法的类的反序列化，但存在丢失字段值的问题。

```
@Getter
@Setter
@ToString
class User {
    private Long id;
    private String name;
    public User(Long id) {
        this.id = id;
    }
}

String text = "{\"id\":123,\"name\":\"test\"}";
User user = JSON.parseObject(text, User.class); // 会丢失name值
```

咨询过 Fastjson 维护人员，目前还没有解决这个 bug，有待后续版本中解决。如果要反序列化这种类，可以考虑添加默认构造方法或自定义反序列化器。

9. 自定义反序列化器

对应一些类对象，需要把特殊格式文本反序列化为对象，就必须自定义反序列化器。比如：Geometry 序列化文本，通常采用 WKT(Well-known text)表示，便于用户快速阅读理解。

1) 全局配置反序列化器

通过 JSON 序列化全局配置指定类反序列化器：

```
String text = ...;
ParserConfig.getGlobalInstance().putDeserializer(Geometry.class, new
GeometryDeserializer());
Geometry geometry = JSON.parseObject(text, Geometry.class);
```

注意：这种方式不支持类继承，必须指定到具体类。比如要序列化 Point 对象，就必须配置 Point 类的反序列化器。

2) 特定配置反序列化器

通过 JSON 序列化特定配置指定类反序列化器：

```
String text = ...;
ParserConfig config = new ParserConfig();
config.putDeserializer(Geometry.class, new GeometryDeserializer());
Geometry geometry = JSON.parseObject(text, Geometry.class, config);
```

注意：这种方式不支持类继承，必须指定到具体类。比如要序列化 Point 对象，就必须配置 Point 类的反序列化器。

3) 注解配置反序列化器

通过 JSON 序列化注解配置指定类反序列化器：

```
public class User {
    ...
    @JSONField(deserializeUsing = GeometryDeserializer.class)
    private Geometry location;
    ...
}

String text = ...;
User user = JSON.parseObject(text, User.class);
```

其中：GeometryDeserializer 为自定义类，这里就不贴出具体实现了。

八、 不必要的 JSON 序列化

以上章节，都是说 JSON 资源文件在单元测试中如何运用，如何利用 JSON 资源文件把单元测试编写得更优雅。有时候，任何手段都有两面性，过渡依赖 JSON 资源文件测试，也会把单元测试复杂化。这里，作者总结了几个例子以示说明。

1. 完全透传的对象

1) 完全透传的参数对象

在测试方法中，有些参数没有被任何修改，只是完全被透传而已。

被测方法：

```
public void batchCreate(List<UserCreate> createList) {
    userDAO.batchCreate(createList);
}
```

测试用例：

```
@Test
public void testBatchCreate() {
    // 调用测试方法
    List<UserCreate> createList = new ArrayList<>();
    userService.batchCreate(createList);

    // 验证测试方法
    Mockito.verify(userDAO).batchCreate(createList);
}
```

其中，不需要 ArgumentCaptor 去捕获 userDAO.batchCreate 的参数并验证参数值，这里只需要验证 createList 是不是同一个对象即可。

2) 完全透传的返回对象

在测试方法中，有些返回值没有被任何修改，只是完全被透传而已。

被测方法：

```
public List<UserVO> queryByCompanyId(Long companyId) {
    return userDAO.queryByCompanyId(companyId);
}
```

测试用例：

```
@Test
public void testQueryByCondition() {
    // 模拟依赖方法
    Long companyId = 1L;
    List<UserVO> userList = new ArrayList<>();
    Mockito.doReturn(userList).when(userDAO).queryByCompanyId(companyId);

    // 调用测试方法
    Assert.assertEquals("用户列表不一致", userList, userService.queryByCompanyId(companyId));
}
```

其中，userList 对象不需要构造数据，只需要验证是不是同一个对象即可。

2. 完全透传的属性

1) 完全透传的参数值属性

在测试方法中，有些参数值属性没有被任何修改，只是完全被透传而已。

被测方法：

```
public void handleResult(Result<UserVO> result) {
    if (!result.isSuccess()) {
        metaProducer.sendCouponMessage(result.getData());
    }
}
```

测试用例：

```
@Test
public void testHandleResultWithSuccess() {
    // 调用测试方法
    UserVO user = new UserVO();
    Result<UserVO> result = Result.success(user);
    userService.handleResult(result);

    // 验证依赖方法
    Mockito.verify(metaProducer).sendCouponMessage(user);
}
```

其中，user 对象不需要构造数据，只需要验证是不是同一个对象即可。

2) 完全透传的返回值属性

在测试方法中，有些返回值属性没有被任何修改，只是完全被透传而已。

被测方法：

```
public UserVO get(Long userId) {
    Result<UserVO> result = userHsfService.get(userId);
    if (!result.isSuccess()) {
        throw new ExmapleException(String.format("获取用户(%s)失败:%s", userId,
result.getMessage()));
    }
    return result.getData();
}
```

测试用例：

```
@Test
public void testGetWithSuccess() {
    // 模拟依赖方法
    Long userId = 123L;
    UserVO user = UserVO();
    Mockito.doReturn(Result.success(user)).when(userHsfService).get(userId);

    // 调用测试方法
    Assert.assertEquals("用户信息不一致", user, userService.get(userId));
}
```

其中，user 对象不需要构造数据，只需要验证是不是同一个对象即可。

3. 仅用少数字段的对象

1) 仅用少数字段的参数值对象

在测试方法中，有些参数值对象字段虽多，但只会用到其中少数字段。

被测方法：

```
public void create(UserCreate userCreate) {
    Boolean exist = userDAO.existByName(userCreate.getName());
    if (Boolean.TRUE.equals(exist)) {
        throw new ExmapleException(String.format("用户(%s)已存在", userCreate.getName()));
    }
    userDAO.create(userCreate);
}
```

测试用例：

```
@Test
public void testCreateWithException() {
    UserCreate userCreate = new UserCreate();
    userCreate.setName("changyi");
    ExmapleException exception = Assert.assertThrows("异常类型不一致", ExmapleException.class,
() -> userService.create(userCreate));
    Assert.assertEquals("异常消息不一致", String.format("用户(%s)已存在", userCreate.getName()),
exception.getMessage());
}
```

其中，不需要构造参数值 userCreate 的所有属性字段，只需构造使用到的 name 属性字段即可。

2) 仅用少数字段的返回值对象

在测试方法中，有些返回值对象字段虽多，但只会用到其中少数字段。

被测方法：

```
public boolean isVip(Long userId) {
    UserDO user = userDAO.get(userId);
    return VIP_ROLE_ID_SET.contains(user.getRoleId());
}
```

测试用例：

```
@Test
public void testIsVipWithTrue() {
    // 模拟依赖方法
    Long userId = 123L;
    UserDO user = new UserDO();
    user.setRoleId(VIP_ROLE_ID);
    Mockito.doReturn(user).when(userDAO).get(userId);

    // 调用测试方法
    Assert.assertTrue("返回值不为真", userService.isVip());
}
```

其中，不需要构造返回值 user 的所有属性字段，只需构造使用到的 roleId 属性字段即可。

4. 使用 new 还是 mock 初始化对象？

在上面案例中，我们都采用 new 来初始化对象并采用 set 来模拟属性值的。有些同学会问，为什么不采用 mock 来初始化对象、用 doReturn-when 来模拟属性值？我想说，都是一样的效果，只是前者显得更简洁而已。

关于使用 new 还是 mock 初始化对象，这个问题在网上一直有争论，双方都各有自己的理由。

这里，按照作者的个人使用习惯，进行了简单的归纳总结如下：

使用情形	采用new初始化对象	采用mock初始化对象
实体类	首选	可以
接口类	不可以	首选
虚基类	不可以	首选
简单方法	首选	可以
复杂方法	可以	首选

九、 JSON 结合 Mockito 妙用

上面已经介绍过，JSON 序列化在编写 Java 单元测试用例时最大的妙用有两点：

- JSON 反序列化字符串为数据对象，大大减少了数据对象的模拟代码；
- JSON 序列化数据对象为字符串，把数据对象验证简化为字符串验证，大大减少了数据对象的验证代码。

除此之外，JSON 序列化结合 Mockito，往往会起到意想不到的效果，能产生一些非常巧妙有效的用法。

1. 模拟方法返回多个值

当一个方法需要多次调用，但返回值跟输入参数无关，只跟调用顺序有关的时，可以用数组来模拟方法返回值。先加载一个列表 JSON 资源文件，通过 JSON.parseObject 方法转化为数组，然后利用 Mockito 的 doReturn-when 或 when-thenReturn 语法来模拟方法返回多个值。

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "recordList.json");
Record[] records = JSON.parseObject(text, Record[].class);
Mockito.doReturn(records[0], ArrayUtils.subarray(records, 1,
records.length)).when(recordReader).read();
```

2. 模拟方法返回对应值

当一个方法需要多次调用，但返回值跟调用顺序无关，只能调输入参数有关的时，可以用映射来模拟方法返回值。先加载一个映射 JSON 资源文件，通过 JSON.parseObject 方法转化为映射，然后利用 Mockito 的 doAnswer-when 或

when-thenAnswer 语法来模拟方法返回对应值（根据指定参数返回映射中的对应值）。

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "roleMap.json");
Map<Long, String> roleIdMap = JSON.parseObject(text, new TypeReference<Map<Long, String>>()
{});
Mockito.doAnswer(invocation ->
userMap.get(invocation.getArgument(0))).when(roleService).get(roleId);
```

3. 验证多次方法调用参数

当验证一个方法调用参数时，需要用 ArgumentCaptor 来捕获这个参数，然后通过 getValue 方法验证这个参数。如果这个方法被多次调用，就没有必要依次验证了，可以通过 getAllValues 方法获取一个列表，然后通过 JSON.toJSONString 转化为 JSON 字符串，然后跟 JSON 资源文件进行统一验证。

```
ArgumentCaptor<UserCreateVO> userCreateCaptor = ArgumentCaptor.forClass(UserCreateVO.class);
Mockito.verify(userDAO, Mockito.atLeastOnce()).create(userCreateCaptor.capture());
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateList.json");
Assert.assertEquals("用户创建列表不一致", text,
JSON.toJSONString(userCreateCaptor.getAllValues()));
```

当然，二者结合的妙用不仅限于此，还有更多作者没有总结到的。

后记

闲暇之余，填词一首自娱自乐，借以致敬广大的码农们：

《卜算子·致码农》

业务要精通，
眼界需开阔。
谈笑风生略几回，
草案心头过。
编写守规则，
测试知结果。
奋笔疾书数万行，
代码无一错。

其中，第二段便说明了代码规范和单元测试的重要性——只有做到“编写守规则，测试知结果”，才能保障“奋笔疾书数万行，代码无一错”。

简化技巧：Java 编程技巧之单元测试用例简化方法

前言

清代谴责小说家吴趼人在《痛史》中写道：“卷帙浩繁，望而生畏。”

意思是：“一部书的篇幅太长，让人看见就害怕。”编写单元测试用例也是如此，如果单元测试用例写起来又长又复杂，自然而然地会让人“望而生畏”，于是开始反感甚至于最终放弃。为了便于 Java 单元测试的推广，作者总结了十余种测试用例的简化方法，希望能够让大家编写单元测试用例时——“化繁为简、下笔如神”。

一、 简化模拟数据对象

1. 利用 JSON 反序列化简化数据对象赋值语句

利用 JSON 反序列化，可以简化大量的数据对象赋值语句。首先，加载 JSON 资源文件为 JSON 字符串；然后，通过 JSON 反序列化 JSON 字符串为数据对象；最后，用该数据对象来模拟类属性值、方法参数值和方法返回值。

原始用例：

```
List<UserCreateVO> userCreateList = new ArrayList<>();
UserCreateVO userCreate0 = new UserCreateVO();
userCreate0.setName("Changyi");
... // 约几十行
userCreateList.add(userCreate0);
UserCreateVO userCreate1 = new UserCreateVO();
userCreate1.setName("Tester");
... // 约几十行
userCreateList.add(userCreate1);
... // 约几十条
userService.batchCreate(userCreateList);
```

简化用例：

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateList.json");
List<UserCreateVO> userCreateList = JSON.parseArray(text, UserCreateVO.class);
userService.batchCreate(userCreateList);
```

2. 利用虚拟数据对象简化返回值模拟语句

有时候，模拟的方法返回值在测试方法内部并不发生修改，只是起到一个透传的作用而已。对于这种情况，我们只需要模拟一个对象实例，但并不关心其内部具体内容。所以，可以直接用虚拟数据对象替换真实数据对象，从而简化了返回值模拟语句。

被测代码：

```
@GetMapping("/get")
public ExampleResult<UserVO> getUser(@RequestParam(value = "userId", required = true) Long
userId) {
  UserVO user = userService.getUser(userId);
  return ExampleResult.success(user);
}
```

原始用例：

```
// 模拟依赖方法
String path = RESOURCE_PATH + "testGetUser/";
String text = ResourceHelper.getResourceAsString(getClass(), path + "user.json");
UserVO user = JSON.parseObject(text, UserVO.class);
Mockito.doReturn(user).when(userService).getUser(user.getId());

// 调用测试方法
ExampleResult<UserVO> result = userController.getUser(user.getId());
Assert.assertEquals("结果编码不一致", ResultCode.SUCCESS.getCode(), result.getCode());
Assert.assertEquals("结果数据不一致", user, result.getData());
```

简化用例：

```
// 模拟依赖方法
Long userId = 12345L;
UserVO user = Mockito.mock(UserVO.class); // 也可以使用new UserVO()
Mockito.doReturn(user).when(userService).getUser(userId);

// 调用测试方法
ExampleResult<UserVO> result = userController.getUser(userId);
Assert.assertEquals("结果编码不一致", ResultCode.SUCCESS.getCode(), result.getCode());
Assert.assertSame("结果数据不一致", user, result.getData());
```

3. 利用虚拟数据对象简化参数值模拟语句

有时候，模拟的方法参数值在测试方法内部并不发生修改，只是起到一个透传的作用而已。对于这种情况，我们只需要模拟一个对象实例，但并不关心其内部具体内容。所以，可以直接用虚拟数据对象替换真实数据对象，从而简化参数值模拟语句。

被测代码：

```
@GetMapping("/create")
public ExampleResult<Void> createUser(@Valid @RequestBody UserCreateVO userCreate) {
  userService.createUser(userCreate);
  return ExampleResult.success();
}
```

原始用例：

```
// 调用测试方法
String path = RESOURCE_PATH + "testCreateUser/";
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreate.json");
UserCreateVO userCreate = JSON.parseObject(text, UserCreateVO.class);
ExampleResult<Void> result = userController.createUser(userCreate);
Assert.assertEquals("结果编码不一致", ResultCode.SUCCESS.getCode(), result.getCode());

// 验证依赖方法
Mockito.verify(userService).createUser(userCreate);
```

简化用例：

```
// 调用测试方法
UserCreateVO userCreate = Mockito.mock(UserCreateVO.class); // 也可以使用new UserCreateVO()
ExampleResult<Void> result = userController.createUser(userCreate);
Assert.assertEquals("结果编码不一致", ResultCode.SUCCESS.getCode(), result.getCode());

// 验证依赖方法
Mockito.verify(userService).createUser(userCreate);
```

二、 简化模拟依赖方法

1. 利用默认返回值简化模拟依赖方法

模拟对象的方法是具有默认返回值的：当方法返回类型为基础类型时，默认返回值是 0 或 false；当方法返回类型为对象类型时，默认返回值是 null。在测试用例中，

当需要模拟方法返回值为上述默认值时，我们可以省略这些模拟方法语句。当然，显式地写上这些模拟方法语句，可以让测试用例变得更便于理解。

原始用例：

```
Mockito.doReturn(false).when(userDAO).existName(userName);
Mockito.doReturn(0L).when(userDAO).countByCompany(companyId);
Mockito.doReturn(null).when(userDAO).queryByCompany(companyId, startIndex, pageSize);
```

简化用例：

可以把以上模拟方法语句直接删除。

2. 利用任意匹配参数简化模拟依赖方法

在模拟依赖方法时，有些参数需要使用到后面加载的数据对象，比如下面案例中的 UserCreateVO 的 name 属性值。这样，我们就需要提前加载 UserCreateVO 对象，既让模拟方法语句看起来比较繁琐，又让加载 UserCreateVO 对象语句和使用 UserCreateVO 对象语句分离（优秀的代码，变量定义和初始化一般紧挨着变量使用代码）。

利用任意匹配参数就可以解决这些问题，使测试用例变得更简洁更便于维护。但是要注意，验证该方法时，不能再用任意匹配参数去验证，必须使用真实的值去验证。

原始用例：

```
// 模拟依赖方法
String path = RESOURCE_PATH + "testCreateUserWithSuccess/";
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateVO.json");
UserCreateVO userCreateVO = JSON.parseObject(text, UserCreateVO.class);
Mockito.doReturn(false).when(userDAO).existName(userCreateVO.getName());
...

// 调用测试方法
Assert.assertEquals("用户标识不一致", userId, userService.createUser(userCreateVO));

// 验证依赖方法
Mockito.verify(userDAO).existName(userCreateVO.getName());
...
```

简化用例：

```
// 模拟依赖方法
Mockito.doReturn(false).when(userDAO).existName(Mockito.anyString());
...

// 调用测试方法
String path = RESOURCE_PATH + "testCreateUserWithSuccess/";
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateVO.json");
UserCreateVO userCreateVO = JSON.parseObject(text, UserCreateVO.class);
Assert.assertEquals("用户标识不一致", userId, userService.createUser(userCreateVO));

// 验证依赖方法
Mockito.verify(userDAO).existName(userCreateVO.getName());
...
```

3. 利用 do/thenAnswer 简化模拟依赖方法

当一个方法需要调用多次，但返回值跟调用顺序无关，只跟输入参数有关的时，可以用映射来模拟方法不同返回值。先加载一个映射 JSON 资源文件，通过 JSON.parseObject 方法转化为映射，然后利用 Mockito 的 doAnswer-when 或 when-thenAnswer 语法来模拟方法返回对应值（根据指定参数返回映射中的对应值）。

原始用例：

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "user1.json");
UserDO user1 = JSON.parseObject(text, UserDO.class);
Mockito.doReturn(user1).when(userDAO).get(user1.getId());
text = ResourceHelper.getResourceAsString(getClass(), path + "user2.json");
UserDO user2 = JSON.parseObject(text, UserDO.class);
Mockito.doReturn(user2).when(userDAO).get(user2.getId());
...
```

简化用例：

```
String text = ResourceHelper.getResourceAsString(getClass(), path + "userMap.json");
Map<Long, UserDO> userMap = JSON.parseObject(text, new TypeReference<Map<Long, UserDO>>()
{});
Mockito.doAnswer(invocation ->
userMap.get(invocation.getArgument(0))).when(userDAO).get(Mockito.anyLong());
```

4. 利用 Mock 参数简化模拟链式调用方法

在日常编码过程中，很多人都喜欢使用链式调用，这样可以让代码变得更简洁。对于链式调用，Mockito 提供了更加简便的单元测试方法——提供 Mockito.RETURNS_DEEP_STUBS 参数，实现链式调用的对象自动 mock。

被测代码：

```
public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/**")
        .allowedOrigins("*")
        .allowedMethods("GET", "HEAD", "POST", "PUT", "DELETE", "OPTIONS")
        .allowCredentials(true)
        .maxAge(MAX_AGE)
        .allowedHeaders("*");
}
```

原始用例：

正常情况下，每一个依赖对象及其调用方法都要 mock，编写的代码如下：

```
@Test
public void testAddCorsMappings() {
    // 模拟依赖方法
    CorsRegistry registry = Mockito.mock(CorsRegistry.class);
    CorsRegistration registration = Mockito.mock(CorsRegistration.class);
    Mockito.doReturn(registration).when(registry).addMapping(Mockito.anyString());
    Mockito.doReturn(registration).when(registration).allowedOrigins(Mockito.any());
    Mockito.doReturn(registration).when(registration).allowedMethods(Mockito.any());

Mockito.doReturn(registration).when(registration).allowCredentials(Mockito.anyBoolean());
    Mockito.doReturn(registration).when(registration).maxAge(Mockito.anyLong());
    Mockito.doReturn(registration).when(registration).allowedHeaders(Mockito.any());

    // 调用测试方法
    webAuthInterceptConfig.addCorsMappings(registry);

    // 验证依赖方法
    Mockito.verify(registry).addMapping("/**");
    Mockito.verify(registration).allowedOrigins("*");
    Mockito.verify(registration).allowedMethods("GET", "HEAD", "POST", "PUT", "DELETE",
"OPTIONS");
    Mockito.verify(registration).allowCredentials(true);
    Mockito.verify(registration).maxAge(3600L);
    Mockito.verify(registration).allowedHeaders("*");
}
```

简化用例：

利用 Mockito.RETURNS_SELF 参数编写的测试用例如下：

```
@Test
public void testAddCorsMapping() {
    // 模拟依赖方法
    CorsRegistry registry = Mockito.mock(CorsRegistry.class);
    CorsRegistration registration = Mockito.mock(CorsRegistration.class,
Answers.RETURNS_SELF);
    Mockito.doReturn(registration).when(registry).addMapping(Mockito.anyString());

    // 调用测试方法
    webAuthInterceptConfig.addCorsMappings(registry);

    // 验证依赖方法
    Mockito.verify(registry).addMapping("/**");
    Mockito.verify(registration).allowedOrigins("*");
    Mockito.verify(registration).allowedMethods("GET", "HEAD", "POST", "PUT", "DELETE",
"OPTIONS");
    Mockito.verify(registration).allowCredentials(true);
    Mockito.verify(registration).maxAge(3600L);
    Mockito.verify(registration).allowedHeaders("*");
}
```

代码说明：

- 在 mock 对象时，对于自返回对象，需要指定 Mockito.RETURNS_SELF 参数；
- 在 mock 方法时，无需对自返回对象进行 mock 方法，因为框架已经 mock 方法返回了自身；
- 在 verify 方法时，可以像普通测试法一样优美地验证所有方法调用。

参数说明：

在 mock 参数中，有 2 个参数适合于 mock 链式调用：

- RETURNS_SELF 参数：mock 调用方法语句最少，适合于链式调用返回相同值；
- RETURNS_DEEP_STUBS 参数：mock 调用方法语句较少，适合于链式调用返回不同值。

三、 简化验证数据对象

1. 利用 JSON 序列化简化数据对象验证语句

利用 JSON 反序列化，可以简化大量的数据对象验证语句。首先，加载 JSON 资源文件为 JSON 字符串；然后，通过 JSON 序列化数据对象（方法返回值或方法参数值）为 JSON 字符串；最后，再验证两个 JSON 字符串是否一致。

原始用例：

```
List<UserVO> userList = userService.queryByCompanyId(companyId);
UserVO user0 = userList.get(0);
Assert.assertEquals("name不一致", "Changyi", user0.getName());
... // 约几十行
UserVO user1 = userList.get(1);
Assert.assertEquals("name不一致", "Tester", user1.getName());
... // 约几十行
... // 约几十条
```

简化用例：

```
List<UserVO> userList = userService.queryByCompanyId(companyId);
String text = ResourceHelper.getResourceAsString(getClass(), path + "userList.json");
Assert.assertEquals("用户列表不一致", text, JSON.toJSONString(userList));
```

小知识点：

- 如果数据对象中存在 Map 对象，为了保证序列化后的字段顺序一致，需要添加 SerializerFeature.MapSortField 特征。

```
JSON.toJSONString(userMap, SerializerFeature.MapSortField);
```

- 如果数据对象中存在随机对象，比如时间、随机数等，需要使用过滤器过滤这些字段。

排除所有类的属性字段：

```
List<UserVO> userList = ...;
SimplePropertyPreFilter filter = new SimplePropertyPreFilter();
filter.getExcludes().addAll(Arrays.asList("gmtCreate", "gmtModified"));
Assert.assertEquals("用户信息不一致", text, JSON.toJSONString(user, filter));
```

排除单个类的属性字段：

```
List<UserVO> userList = ...;
SimplePropertyPreFilter filter = new SimplePropertyPreFilter(UserVO.class);
filter.getExcludes().addAll(Arrays.asList("gmtCreate", "gmtModified"));
Assert.assertEquals("用户信息不一致", text, JSON.toJSONString(user, filter));
```

排除多个类的属性字段：

```
Pair<UserVO, CompanyVO> userCompanyPair = ...;
SimplePropertyPreFilter userFilter = new SimplePropertyPreFilter(UserVO.class);
userFilter.getExcludes().addAll(Arrays.asList("gmtCreate", "gmtModified"));
SimplePropertyPreFilter companyFilter = new SimplePropertyPreFilter(CompanyVO.class);
companyFilter.getExcludes().addAll(Arrays.asList("createTime", "modifyTime"));
Assert.assertEquals("用户公司对不一致", text, JSON.toJSONString(userCompanyPair, new
SerializeFilter[]{userFilter, companyFilter});
```

2. 利用数据对象相等简化返回值验证语句

在用 Assert.assertEquals 方法验证返回值时，可以直接指定基础类型值或数据对象实例。当数据对象实例不一致时，只要其数据对象相等（equals 比较返回 true），Assert.assertEquals 方法也会认为其是相等的。所以，可以利用数据对象相等来替换 JSON 字符串验证，从而简化测试方法返回值验证语句。

原始用例：

```
List<Long> userIdList = userService.getAllUserIds(companyId);
String text = JSON.toJSONString(Arrays.asList(1L, 2L, 3L));
Assert.assertEquals("用户标识列表不一致", text, JSON.toJSONString(userIdList));
```

简化用例：

```
List<Long> userIdList = userService.getAllUserIds(companyId);
Assert.assertEquals("用户标识列表不一致", Arrays.asList(1L, 2L, 3L), userIdList);
```

小知识点：

- Assert.assertSame 用于相同类实例验证——类实例相同；
- Assert.assertEquals 用于相等类实例验证——类实例相同或相等（equals 为 true）。

注意：

这里不建议为了使用这个功能而重载 equals 方法，只建议针对相同或已重载 equals 方法的类实例使用。而针对未重载 equals 方法的类实例，建议转化为 JSON 字符串后再验证。

3. 利用数据对象相等简化参数值验证语句

在用 Mockito.verify 方法验证依赖方法参数时，可以直接指定基础类型值或数据对象实例。当数据对象实例不一致时，只要其数据对象相等（equals 比较返回 true），Mockito.verify 方法也会认为其是相等的。所以，可以利用数据对象相等来替换 ArgumentCaptor 参数捕获，从而简化依赖方法参数值验证语句。

原始用例：

```
ArgumentCaptor<List<Long>> userIdListCaptor =
CastHelper.cast(ArgumentCaptor.forClass(List.class));
Mockito.verify(userDAO).batchDelete(userIdListCaptor.capture());
Assert.assertEquals("用户标识列表不一致", Arrays.asList(1L, 2L, 3L),
userIdListCaptor.getValue());
```

简化用例：

```
Mockito.verify(userDAO).batchDelete(Arrays.asList(1L, 2L, 3L));
```

注意：

这里不建议为了使用这个功能而重载 equals 方法，只建议针对相同或已重载 equals 方法的类实例使用。而针对未重载 equals 方法的类实例，建议先捕获参数转化为 JSON 字符串后再验证。

四、 简化验证依赖方法

1. 利用 ArgumentCaptor 简化验证依赖方法

当一个模拟方法被多次调用时，需要对每一次模拟方法调用进行验证，就显得模拟方法验证代码过于繁琐。这里，可以通过 ArgumentCaptor 来捕获参数值，然后通过 getAllValues 方法来获取参数值列表，最后对参数值列表进行统一验证即可。其中，即验证了方法调用次数（列表长度），又验证了方法参数值（列表数据）。

原始用例：

```
Mockito.verify(userDAO).get(user1.getId());
Mockito.verify(userDAO).get(user2.getId());
...
```

简化用例：

```
ArgumentCaptor<Long> userIdCaptor = ArgumentCaptor.forClass(Long.class);
Mockito.verify(userDAO, Mockito.atLeastOnce()).get(userIdCaptor.capture());
text = ResourceHelper.getResourceAsString(getClass(), path + "userIdList.json");
Assert.assertEquals("用户标识列表不一致", text,
JSON.toJSONString(userIdCaptor.getAllValues()));
```

五、 简化单元测试用例

1. 利用直接测试私有方法简化单元测试用例

习惯性地，我们通过构造共有方法的测试用例，来覆盖公有方法及其私有方法的所有分支。这种方式没有问题，但有时候显得测试用例比较繁琐。我们可以直接测试私有方法，单独对私有方法进行全覆盖，从而减少对公有方法的测试用例。

被测代码：

```
public UserVO getUser(Long userId) {
    // 获取用户信息
    UserDO userDO = userDAO.get(userId);
    if (Objects.isNull(userDO)) {
        throw new ExampleException(ErrorCode.OBJECT_NONEXIST, "用户不存在");
    }

    // 返回用户信息
    UserVO userVO = new UserVO();
    userVO.setId(userDO.getId());
    userVO.setName(userDO.getName());
    userVO.setVip(isVip(userDO.getRoleIdList()));
    ...
    return userVO;
}
private static boolean isVip(List<Long> roleIdList) {
    for (Long roleId : roleIdList) {
        if (VIP_ROLE_ID_SET.contains(roleId)) {
            return true;
        }
    }
    return false;
}
```

原始用例：

```
@Test
public void testGetUserWithVip() {
    // 模拟依赖方法
    String path = RESOURCE_PATH + "testGetUserWithVip/";
    String text = ResourceHelper.getResourceAsString(getClass(), path + "userDO.json");
    UserDO userDO = JSON.parseObject(text, UserDO.class);
    Mockito.doReturn(userDO).when(userDAO).get(userDO.getId());

    // 调用测试方法
    UserVO userVO = userService.getUser(userDO.getId());
    text = ResourceHelper.getResourceAsString(getClass(), path + "userVO.json");
    Assert.assertEquals("用户信息不一致", text, JSON.toJSONString(userVO));

    // 验证依赖方法
    Mockito.verify(userDAO).get(userDO.getId());
}
@Test
public void testGetUserWithNotVip() {
    ... // 代码跟testGetUserWithVip一致，只是测试数据不同而已
}
```

简化用例：

```
@Test
public void testGetUserWithNormal() {
    ... // 代码跟原testGetUserWithVip一致
}
@Test
public void testIsVipWithTrue() throws Exception {
    List<Long> roleIdList = ...; // 包含VIP角色标识
    Assert.assertTrue("返回值不为真", Whitebox.invokeMethod(UserService.class, "isVip",
roleIdList));
}
@Test
public void testIsVipWithFalse() throws Exception {
    List<Long> roleIdList = ...; // 不含VIP角色标识
    Assert.assertFalse("返回值不为假", Whitebox.invokeMethod(UserService.class, "isVip",
roleIdList));
}
```

2. 利用 JUnit 的参数化测试简化单元测试用例

有时候我们会发现，同一方法的在不同场景下的单元测试，除了加载的数据不同之外，单元测试用例的代码基本完全一致。我们可以这样分析：虽然单元测试用例的场景不一样——执行代码的分支不一样，调用方法方法的顺序、次数、返回值不一样；但是，其调用的依赖方法的数量是完全一致的；所以，最终写出来的单元测试用例的代码也是完全一致的。这时，我们就可以采用 JUnit 的参数化测试来简化单元测试用例。

被测代码：

同上一章的测试代码。

原始用例：

同上一章的原始用例。

简化用例：

```
@ParameterizedTest
@ValueSource(strings = { "vip/", "notVip/"})
public void testGetUserWithNormal(String dir) {
    // 模拟依赖方法
    String path = RESOURCE_PATH + "testGetUserWithNormal/" + dir;
    String text = ResourceHelper.getResourceAsString(getClass(), path + "userDO.json");
    UserDO userDO = JSON.parseObject(text, UserDO.class);
    Mockito.doReturn(userDO).when(userDAO).get(userDO.getId());

    // 调用测试方法
    UserVO userVO = userService.getUser(userDO.getId());
    text = ResourceHelper.getResourceAsString(getClass(), path + "userVO.json");
    Assert.assertEquals("用户信息不一致", text, JSON.toJSONString(userVO));

    // 验证依赖方法
    Mockito.verify(userDAO).get(userDO.getId());
}
```

如上简化用例所示：在资源目录 testGetUserWithNormal 中创建了两个目录 vip 和 notVip，用于存储相同名称的 JSON 文件 userDO.json 和 userVO.json，但是其文件内容根据场景又有所不同。

备注：本案例采用 JUnit5.0 的参数化测试新特性。

后记

《走向卓越工程实践》是一本面向集团一线研发工程师/二层组织技术负责人/TLM/Tech Lead 的实践指南，意在引导集团技术朝着更卓越先进的水平持续优化和演进，以高质量带动高效率，为研发工程师和技术管理者指明从基础迈向卓越工程的道路。这里，填写了一首词《满江红·卓越工程》，谨此献给卓越工程小组。

《满江红·卓越工程》

夜色朦胧，高楼里、灯如白昼。
揉睡眼，仰天长吼，又熬一宿。
美好生活仍未见，称心工作何时有？
累成狗、秃了少年头，人消瘦。

编单测，纠错谬。核代码，查疏漏。
讲方法，打破加班魔咒。

卓越工程提效率，日常实践争优秀。

从此后、阔步向前行，精神抖。

无效单测：那些年，我们写过的无效单元测试

前言

那些年，为了学分，我们学会了面向过程编程；
那些年，为了就业，我们学会了面向对象编程；
那些年，为了生活，我们学会了面向工资编程；
那些年，为了升职加薪，我们学会了面向领导编程；
那些年，为了完成指标，我们学会了面向指标编程；
.....
那些年，我们学会了敷衍地编程；
那些年，我们编程只是为了敷衍。

现在，领导要响应集团提高代码质量的号召，需要提升单元测试的代码覆盖率。当然，我们不能让领导失望，那就加班加点地补充单元测试用例，努力提高单元测试的代码覆盖率。至于单元测试用例的有效性，我们大抵是不用关心的，因为我们只是面向指标编程。

我曾经阅读过一个 Java 服务项目，单元测试的代码覆盖率非常高，但是通篇没有一个依赖方法验证（Mockito.verify）、满纸仅存几个数据对象断言（Assert.assertNotNull）。我说，这些都是无效的单元测试用例，根本起不到测试代码 BUG 和回归验证代码的作用。后来，在一个月黑风高的夜里，一个新增的方法调用，引起了一场血雨腥风。

编写单元测试用例的目的，并不是为了追求单元测试代码覆盖率，而是为了利用单元测试验证回归代码——试图找出代码中潜藏着的 BUG。所以，我们应该具备工匠精神、怀着一颗敬畏心，编写出有效的单元测试用例。在这篇文章里，作者通过日常的单元测试实践，系统地总结出一套避免编写无效单元测试用例的方法和原则。

一、 单元测试简介

1. 单元测试概念

在维基百科中是这样描述的：

在计算机编程中，单元测试又称为模块测试，是针对程序模块来进行正确性检验的测试工作。程序单元是应用的最小可测试部件。在过程化编程中，一个单元就是单个程序、函数、过程等；对于面向对象编程，最小单元就是方法，包括基类、抽象类、或者派生类中的方法。

2. 单元测试案例

首先，通过一个简单的服务代码案例，让我们认识一下集成测试和单元测试。

1) 服务代码案例

这里，以用户服务（UserService）的分页查询用户（queryUser）为例说明。

```
@Service
public class UserService {
    /** 定义依赖对象 */
    /** 用户DAO */
    @Autowired
    private UserDAO userDAO;

    /**
     * 查询用户
     *
     * @param companyId 公司标识
     * @param startIndex 开始序号
     * @param pageSize 分页大小
     * @return 用户分页数据
     */
    public PageDataVO<UserVO> queryUser(Long companyId, Long startIndex, Integer pageSize) {
        // 查询用户数据
        // 查询用户数据：总共数量
        Long totalSize = userDAO.countByCompany(companyId);
        // 查询接口数据：数据列表
        List<UserVO> dataList = null;
        if (NumberHelper.isPositive(totalSize)) {
            dataList = userDAO.queryByCompany(companyId, startIndex, pageSize);
        }

        // 返回分页数据
        return new PageDataVO<>(totalSize, dataList);
    }
}
```

2) 集成测试用例

很多人认为，凡是用到 JUnit 测试框架的测试用例都是单元测试用例，于是就写出了下面的集成测试用例。

```
@Slf4j
@RunWith(PandoraBootRunner.class)
@DelegateTo(SpringJUnit4ClassRunner.class)
@SpringBootTest(classes = {ExampleApplication.class})
public class UserServiceTest {
    /** 用户服务 */
    @Autowired
    private UserService userService;

    /**
     * 测试：查询用户
     */
    @Test
    public void testQueryUser() {
        Long companyId = 123L;
        Long startIndex = 90L;
        Integer pageSize = 10;
        PageDataVO<UserVO> pageData = userService.queryUser(companyId, startIndex,
pageSize);
        log.info("testQueryUser: pageData={}", JSON.toJSONString(pageData));
    }
}
```

集成测试用例主要有以下特点：

- 依赖外部环境和数据；
- 需要启动应用并初始化测试对象；
- 直接使用`@Autowired`注入测试对象；
- 有时候无法验证不确定的返回值，只能靠打印日志来人工核对。

3) 单元测试用例

采用 JUnit+Mockito 编写的单元测试用例如下：

```
@Slf4j
@RunWith(MockitoJUnitRunner.class)
public class UserServiceTest {
    /** 定义静态常量 */
    /** 资源路径 */
    private static final String RESOURCE_PATH =
"testUserService/";

    /** 模拟依赖对象 */
    /** 用户 DAO */
    @Mock
    private UserDAO userDAO;
```

```
    /** 定义测试对象 */
    /** 用户服务 */
    @InjectMocks
    private UserService userService;

    /**
     * 测试：查询用户-无数据
     */
    @Test
    public void testQueryUserWithoutData() {
        // 模拟依赖方法
        // 模拟依赖方法：userDAO.countByCompany
        Long companyId = 123L;
        Long startIndex = 90L;
        Integer pageSize = 10;

Mockito.doReturn(0L).when(userDAO).countByCompany(companyId);

        // 调用测试方法
        String path = RESOURCE_PATH +
"testQueryUserWithoutData/";
        PageDataVO<UserVO> pageData =
userService.queryUser(companyId, startIndex, pageSize);
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"pageData.json");
        Assert.assertEquals("分页数据不一致", text,
JSON.toJSONString(pageData));

        // 验证依赖方法
        // 验证依赖方法：userDAO.countByCompany
        Mockito.verify(userDAO).countByCompany(companyId);

        // 验证依赖对象
        Mockito.verifyNoMoreInteractions(userDAO);
    }

    /**
     * 测试：查询用户-有数据
     */
    @Test
    public void testQueryUserWithData() {
        // 模拟依赖方法
        String path = RESOURCE_PATH + "testQueryUserWithData/";
```

```
        // 模拟依赖方法：userDAO.countByCompany
        Long companyId = 123L;

Mockito.doReturn(91L).when(userDAO).countByCompany(companyId);
        // 模拟依赖方法：userDAO.queryByCompany
        Long startIndex = 90L;
        Integer pageSize = 10;
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"dataList.json");
        List<UserVO> dataList = JSON.parseArray(text,
UserVO.class);

Mockito.doReturn(dataList).when(userDAO).queryByCompany(companyI
d, startIndex, pageSize);

        // 调用测试方法
        PageDataVO<UserVO> pageData =
userService.queryUser(companyId, startIndex, pageSize);
        text = ResourceHelper.getResourceAsString(getClass(),
path + "pageData.json");
        Assert.assertEquals("分页数据不一致", text,
JSON.toJSONString(pageData));

        // 验证依赖方法
        // 验证依赖方法：userDAO.countByCompany
        Mockito.verify(userDAO).countByCompany(companyId);
        // 验证依赖方法：userDAO.queryByCompany
        Mockito.verify(userDAO).queryByCompany(companyId,
startIndex, pageSize);

        // 验证依赖对象
        Mockito.verifyNoMoreInteractions(userDAO);
    }
}
```

单元测试用例主要有以下特点：

- 不依赖外部环境和数据；
- 不需要启动应用和初始化对象；
- 需要用@Mock 来初始化依赖对象，用@InjectMocks 来初始化测试对象；
- 需要自己模拟依赖方法，指定什么参数返回什么值或异常；
- 因为测试方法返回值确定，可以直接用 Assert 相关方法进行断言；

- 可以验证依赖方法的调用次数和参数值，还可以验证依赖对象的方法调用是否验证完毕。

二、 单元测试原则

为什么集成测试不算单元测试呢？我们可以从单元测试原则上来判断。在业界，常见的单元测试原则有 AIR 原则和 FIRST 原则。

1. AIR 原则

AIR 原则内容如下：

- **Automatic（自动的）**

单元测试应该是全自动执行的，并且非交互式的。测试用例通常是被定期执行的，执行过程必须完全自动化才有意义。输出结果需要人工检查的测试不是一个好的单元测试。单元测试中不准使用`System.out`来进行人肉验证，必须使用 assert 来验证。

- **I-Independent（独立的）**

单元测试应该保持的独立性。为了保证单元测试稳定可靠且便于维护，单元测试用例之间决不能互相调用，也不能对外部资源有所依赖。

- **R-Repeatable（可重复的）**

单元测试是可以重复执行的，不能受到外界环境的影响。单元测试通常会被放入持续集成中，每次有代码提交时单元测试都会被执行。

2. FIRST 原则

FIRST 原则内容如下：

- **F-Fast（快速的）**

单元测试应该是可以快速运行的，在各种测试方法中，单元测试的运行速度是最快的，大型项目的单元测试通常应该在几分钟内运行完毕。

- **I-Independent（独立的）**

单元测试应该是可以独立运行的，单元测试用例互相之间无依赖，且对外部资源也无任何依赖。

- **R-Repeatable（可重复的）**

单元测试应该可以稳定重复的运行，并且每次运行的结果都是稳定可靠的。

- **S-SelfValidating（自我验证的）**

单元测试应该是用例自动进行验证的，不能依赖人工验证。

- **T-Timely（及时的）**

单元测试必须及时进行编写，更新和维护，以保证用例可以随着业务代码的变化动态的保障质量。

3. ASCII 原则

阿里的夕华先生也提出了一条 ASCII 原则：

- **Automatic（自动的）**

单元测试应该是全自动执行的，并且非交互式的。

- **S-SelfValidating（自我验证的）**

单元测试中必须使用断言方式来进行正确性验证，而不能根据输出进行人肉验证。

- **C-Consistent（一致的）**

单元测试的参数和结果是确定且一致的。

- **I-Independent（独立的）**

单元测试之间不能互相调用，也不能依赖执行的先后次序。

- **I-Isolated（隔离的）**

单元测试需要是隔离的，不要依赖外部资源。

4. 对比集测和单测

根据上节中的单元测试原则，我们可以对比集成测试和单元测试的满足情况如下：

原则名称	原则项目	集成测试	单元测试
AIR原则	Automatic（自动的）	不一定支持	支持
	Independent（独立的）	不一定支持	支持
	Repeatable（可重复的）	不一定支持	支持
FIRST原则	Fast（快速的）	不一定支持	支持
	Independent（独立的）	不一定支持	支持
	Repeatable（可重复的）	不一定支持	支持
	SelfValidating（自我验证的）	不一定支持	支持
	Timely（及时的）	-	-
ASCII原则	Automatic（自动的）	不一定支持	支持
	SelfValidating（自我验证的）	不一定支持	支持
	Consistent（一致的）	不一定支持	支持
	Independent（独立的）	不一定支持	支持
	Isolated（隔离的）	不一定支持	支持

通过上面表格的对比，可以得出以下结论：

- 集成测试基本上不一定满足所有单元测试原则；
- 单元测试基本上一定都满足所有单元测试原则。

所以，根据这些单元测试原则，可以看出集成测试具有很大的不确定性，不能也不可能完全代替单元测试。另外，集成测试始终是集成测试，即便用于代替单元测试也还是集成测试，比如：利用 H2 内存数据库测试 DAO 方法。

三、 无效单元测试

要想识别无效单元测试，就必须站在对方的角度思考——如何在保障单元测试覆盖率的前提下，能够更少地编写单元测试代码。那么，就必须从单元测试编写流程入手，看哪一阶段哪一方法可以偷工减料。

1. 单元测试覆盖率

在维基百科中是这样描述的：代码覆盖（Code Coverage）是软件测试中的一种度量，描述程序中源代码被测试的比例和程度，所得比例称为代码覆盖率。

常用的单元测试覆盖率指标有：

- **行覆盖（Line Coverage）**

用于度量被测代码中每一行执行语句是否都被测试到了。

- **分支覆盖（Branch Coverage）**

用于度量被测代码中每一个代码分支是否都被测试到了。

- **条件覆盖（Condition Coverage）**

用于度量被测代码的条件中每一个子表达式（true 和 false）是否都被测试到了。

- **路径覆盖（Path Coverage）**

用于度量被测代码中的每一个代码分支组合是否都被测试到了。

除此之外，还有方法覆盖（Method Coverage）、类覆盖（Class Coverage）等单元测试覆盖率指标。

下面，用一个简单方法来分析各个单元测试覆盖率指标：

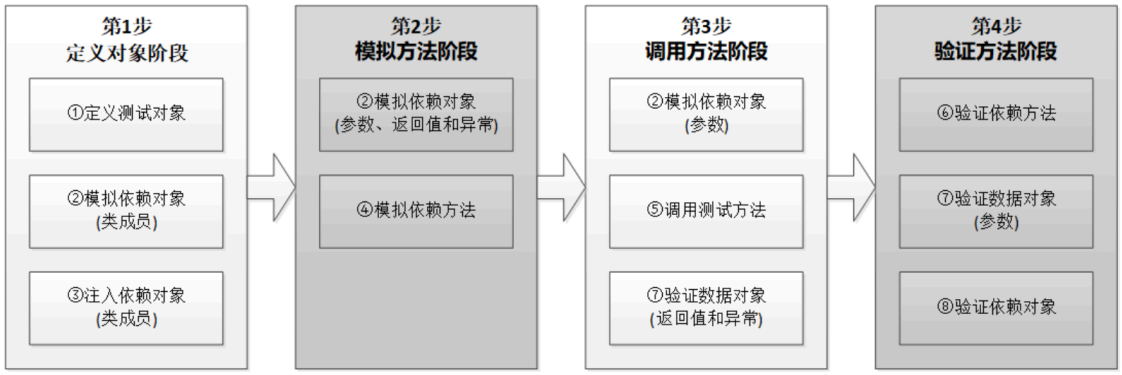
```
public static byte combine(boolean b0, boolean b1) {
    byte b = 0;
    if (b0) {
        b |= 0b01;
    }
    if (b1) {
        b |= 0b10;
    }
    return b;
}
```

覆盖指标	测试用例	覆盖率	备注信息
行覆盖(Line Coverage)	combine(true, true)	100%	每一行执行语句都被执行到
分支覆盖(Branch Coverage)	combine(false, false) combine(true, true)	100%	每一个代码分支都被执行到
条件覆盖(Condition Coverage)	combine(false, true) combine(true, false)	100%	每一个条件子表达式都被执行到
路径覆盖(Path Coverage)	combine(false, false) combine(false, true) combine(true, false) combine(true, true)	100%	每一个代码分支组合都被执行到

单元测试覆盖率，只能代表被测代码的类、方法、执行语句、代码分支、条件子表达式等是否被执行，但是并不能代表这些代码是否被正确地执行并返回了正确的结果。所以，只看单元测试覆盖率，而不看单元测试有效性，是没有任何意义的。

2. 单元测试编写流程

首先，介绍一下作者总结的单元测试编写流程：



1) 定义对象阶段

定义对象阶段主要包括：定义被测对象、模拟依赖对象（类成员）、注入依赖对象（类成员）。

```
@RunWith(PowerMockRunner.class)
public class UserServiceTest {

    /** 模拟依赖对象 */
    /** 用户DAO */
    @Mock
    private UserDAO userDAO;
    /** 标识生成器 */
    @Mock
    private IdGenerator idGenerator;

    /** 定义测试对象 */
    /** 用户服务 */
    @InjectMocks
    private UserService userService;

    /** 定义静态常量 */
    /** 资源路径 */
    private static final String RESOURCE_PATH = "testUserService/";

    /**
     * 在测试之前
     */
    @Before
    public void beforeTest() {
        // 注入依赖对象
        Whitebox.setInternalState(userService, "canModify", Boolean.TRUE);
    }
```

2) 模拟方法阶段

模拟方法阶段主要包括：模拟依赖对象（参数、返回值和异常）、模拟依赖方法。

```
    @Test
    public void testCreateUserWithCreate() {
        // 模拟依赖方法
        // 模拟依赖方法：userDAO.getByName
        Mockito.doReturn(null).when(userDAO).getIdByName(Mockito.anyString());
        // 模拟依赖方法：idGenerator.next
        Long userId = 1L;
        Mockito.doReturn(userId).when(idGenerator).next();

```

3) 调用方法阶段

调用方法阶段主要包括：模拟依赖对象（参数）、调用被测方法、验证参数对象（返回值和异常）。

4) 验证方法阶段

验证方法阶段主要包括：验证依赖方法、验证数据对象（参数）、验证依赖对象 。

3. 是否可以偷工减料

针对单元测试编写流程的阶段和方法，在不影响单元测试覆盖率的情况，我们是否可以进行一些偷工减料。

测试阶段	测试方法	可否偷减	主要原因
1.定义对象阶段	①定义测试对象	不可以	不定义测试对象，根本无法进行测试
	②定义依赖对象（类成员）	不可以	不定义依赖对象（类成员），测试时会抛出空指针或无法进入期望分支
	③注入依赖对象（类成员）	不可以	不注入依赖对象（类成员），测试会抛出空指针异常或无法进入期望分支
2.模拟方法阶段	②模拟依赖对象（参数、返回值和异常）	不可以	不模拟依赖对象（参数、返回值和异常），无法进入期望分支
	④模拟依赖方法	不可以	不模拟模拟依赖方法，无法进入期望分支
3.调用方法阶段	②模拟依赖对象（参数）	不可以	不模拟依赖对象（参数），无法进入期望分支
	⑤调用测试方法	不可以	不执行调用测试方法，根本无法进行测试
	⑦验证数据对象（返回值和异常）	可以	不验证验证数据对象（返回值和异常），对单元测试覆盖率无影响
4.验证方法阶段	⑥验证依赖方法	可以	不验证依赖方法，对单元测试覆盖率无影响
	⑦验证数据对象（参数）	可以	不验证数据对象（参数），对单元测试覆盖率无影响
	⑧验证依赖对象	可以	不验证验证依赖对象，对单元测试覆盖率无影响

4. 最终可以得出结论

通过上表格，可以得出结论，偷工减料主要集中在**验证阶段**：

- **调用方法阶段**
  - 验证数据对象（返回值和异常）
- **验证方法阶段**
  - 验证依赖方法
  - 验证数据对象（参数）
  - 验证依赖对象

通过一些合并和拆分，后续将从以下三部分展开：

- 验证数据对象（包括属性、参数和返回值）；
- 验证抛出异常；
- 验证依赖方法（包括依赖方法和依赖对象）。

四、 验证数据对象

在单元测试中，验证数据对象是为了验证是否传入了期望的参数值、返回了期望的返回值、设置了期望的属性值。

1. 数据对象来源方式

在单元测试中，需要验证的数据对象主要有以下几种来源。

1) 来源于被测方法的返回值

数据对象来源于调用被测方法的返回值，例如：

```
PageDataVO<UserVO> pageData = userService.queryUser(companyId, startIndex, pageSize);
```

2) 来源于依赖方法的参数捕获

数据对象来源于验证依赖方法的参数捕获，例如：

```
ArgumentCaptor<UserDO> userCreateCaptor = ArgumentCaptor.forClass(UserDO.class);
Mockito.verify(userDAO).create(userCreateCaptor.capture());
UserDO userCreate = userCreateCaptor.getValue();
```

3) 来源于被测对象的属性值

数据对象来源于获取被测对象的属性值，例如：

```
userService.loadRoleMap();
Map<Long, String> roleMap = Whitebox.getInternalState(userService, "roleMap");
```

4) 来源于请求参数的属性值

数据对象来源于获取请求参数的属性值，例如：

```
OrderContext orderContext = new OrderContext();
orderContext.setOrderId(12345L);
orderService.supplyProducts(orderContext);
List<ProductDO> productList = orderContext.getProductList();
```

当然，数据对象还有其它来源方式，这里就不再一一举例了。

2. 数据对象验证方式

在调用被测方法时，需要对返回值和异常进行验证；在验证方法调用时，也需要对捕获的参数值进行验证。

1) 验证数据对象空值

JUnit 提供 Assert.assertNull 和 Assert.assertNotNull 方法来验证数据对象空值。

```
// 1. 验证数据对象为空
Assert.assertNull("用户标识必须为空", userId);

// 2. 验证数据对象非空
Assert.assertNotNull("用户标识不能为空", userId);
```

2) 验证数据对象布尔值

JUnit 提供 Assert.assertTrue 和 Assert.assertFalse 方法来验证数据对象布尔值的真假。

```
// 1. 验证数据对象为真
Assert.assertTrue("返回值必须为真", NumberHelper.isPositive(1));

// 2. 验证数据对象为假
Assert.assertFalse("返回值必须为假", NumberHelper.isPositive(-1));
```

3) 验证数据对象引用

JUnit 提供 Assert.assertSame 和 Assert.assertNotSame 方法来验证数据对象引用是否一致。

```
// 1. 验证数据对象一致
Assert.assertSame("用户必须一致", expectedUser, actualUser);

// 2. 验证数据对象不一致
Assert.assertNotSame("用户不能一致", expectedUser, actualUser);
```

4) 验证数据对象取值

JUnit 提供 Assert.assertEquals、Assert.assertNotEquals、Assert.assertArrayEquals 方法组，可以用来验证数据对象值是否相等。

```
// 1. 验证简单数据对象
Assert.assertNotEquals("用户名称不一致", "admin", userName);
Assert.assertEquals("账户金额不一致", 10000.0D, accountAmount, 1E-6D);

// 2. 验证简单集合对象
Assert.assertArrayEquals("用户标识列表不一致", new Long[] {1L, 2L, 3L}, userIds);
Assert.assertEquals("用户标识列表不一致", Arrays.asList(1L, 2L, 3L), userIdList);

// 3. 验证复杂数据对象
Assert.assertEquals("用户标识不一致", Long.valueOf(1L), user.getId());
Assert.assertEquals("用户名称不一致", "admin", user.getName());
...

// 4. 验证复杂集合对象
Assert.assertEquals("用户列表长度不一致", expectedUserList.size(), actualUserList.size());
UserDO[] expectedUsers = expectedUserList.toArray(new UserDO[0]);
UserDO[] actualUsers = actualUserList.toArray(new UserDO[0]);
for (int i = 0; i < actualUsers.length; i++) {
     Assert.assertEquals(String.format("用户 (%s) 标识不一致", i), expectedUsers[i].getId(),
actualUsers[i].getId());
     Assert.assertEquals(String.format("用户 (%s) 名称不一致", i), expectedUsers[i].getName(),
actualUsers[i].getName());
     ...
};

// 5. 通过序列化验证数据对象
String text = ResourceHelper.getResourceAsString(getClass(), "userList.json");
Assert.assertEquals("用户列表不一致", text, JSON.toJSONString(userList));;

// 6. 验证数据对象私有属性字段
Assert.assertEquals("基础包不一致", "com.alibaba.example",
Whitebox.getInternalState(configurer, "basePackage"));
```

当然，数据对象还有其它验证方法，这里就不再一一举例了。

3. 验证数据对象问题

这里，以分页查询公司用户为例，来说明验证数据对象时所存在的问题。

代码案例：

```
public PageDataVO<UserVO> queryUser(Long companyId, Long startIndex, Integer pageSize) {
  // 查询用户数据
  // 查询用户数据：总共数量
  Long totalSize = userDAO.countByCompany(companyId);
  // 查询接口数据：数据列表
  List<UserVO> dataList = null;
  if (NumberHelper.isPositive(totalSize)) {
    List<UserDO> userList = userDAO.queryByCompany(companyId, startIndex, pageSize);
        dataList = userList.stream().map(UserService::convertUser)
            .collect(Collectors.toList());
  }

  // 返回分页数据
  return new PageDataVO<>(totalSize, dataList);
}
private static UserVO convertUser(UserDO userDO) {
    UserVO userVO = new UserVO();
    userVO.setId(userDO.getId());
    userVO.setName(userDO.getName());
    userVO.setDesc(userDO.getDesc());
    ...
    return userVO;
}
```

1) 不验证数据对象

反面案例：

很多人为了偷懒，对数据对象不进行任何验证。

```
// 调用测试方法
userService.queryUser(companyId, startIndex, pageSize);
```

存在问题：

无法验证数据对象是否正确，比如被测代码进行了以下修改：

```
// 返回分页数据
return null;
```

2) 验证数据对象非空

反面案例：

既然不验证数据对象有问题，那么我就简单地验证一下数据对象非空。

```
// 调用测试方法
PageDataVO<UserVO> pageData = userService.queryUser(companyId, startIndex, pageSize);
Assert.assertNotNull("分页数据不为空", pageData);
```

存在问题：

无法验证数据对象是否正确，比如被测代码进行了以下修改：

```
// 返回分页数据
return new PageDataVO<>();
```

3) 验证数据对象部分属性

反面案例：

既然简单地验证数据对象非空不行，那么我就验证数据对象的部分属性。

```
// 调用测试方法
PageDataVO<UserVO> pageData = userService.queryUser(companyId, startIndex, pageSize);
Assert.assertEquals("数据总量不为空", totalSize, pageData.getTotalSize());
```

存在问题：

无法验证数据对象是否正确，比如被测代码进行了以下修改：

```
// 返回分页数据
return new PageDataVO<>(totalSize, null);
```

4) 验证数据对象全部属性

反面案例：

验证数据对象部分属性也不行，那我验证数据对象所有属性总行了吧。

```
// 调用测试方法
PageDataVO<UserVO> pageData = userService.queryUser(companyId);
Assert.assertEquals("数据总量不为空", totalSize, pageData.getTotalSize());
Assert.assertEquals("数据列表不为空", dataList, pageData.getDataList());
```

存在问题：

上面的代码看起来很完美，验证了 PageDataVO 中两个属性值 totalSize 和 dataList。但是，如果有一天在 PageDataVO 中添加了 startIndex 和 pageSize，就无法验证这两个新属性是否赋值正确。代码如下：

```
// 返回分页数据
return new PageDataVO<>(startIndex, pageSize, totalSize, dataList);
```

备注：本方法仅适用于属性字段不可变的数据对象

5) 完美地验证数据对象

对于数据对象属性字段新增，有没有完美的验证方案？有的！答案就是利用 JSON 序列化，然后比较 JSON 文本内容。如果数据对象新增了属性字段，必然会提示 JSON 字符串不一致。

完美案例：

```
// 调用测试方法
PageDataVO<UserVO> pageData = userService.queryUser(companyId, startIndex, pageSize);
text = ResourceHelper.getResourceAsString(getClass(), path + "pageData.json");
Assert.assertEquals("分页数据不一致", text, JSON.toJSONString(pageData));
```

备注：本方法仅适用于属性字段可变的数据对象。

4. 模拟数据对象准则

由于没有模拟数据对象章节,这里在验证数据对象章节中插入了模拟数据对象准则。

1) 除触发条件分支外，模拟对象所有属性值不能为空

在上一节中，我们展示了如何完美地验证数据对象。但是，这种方法真正完美吗？答案是否定。

比如：我们把 userDAO.queryByCompany 方法返回的 uesrList 的所有 UserDO 对象的属性值 name 和 desc 赋值为空，再把 convertUser 方法的 name 和 desc 赋值做一下交换，上面的单元测试用例是无法验证出来的。

```
private static UserVO convertUser(UserDO userDO) {
    UserVO userVO = new UserVO();
    userVO.setId(userDO.getId());
    userVO.setName(userDO.getDesc());
    userVO.setDesc(userDO.getName());
    ...
    return userVO;
}
```

所以，在单元测试中，除触发条件分支外，模拟对象所有属性值不能为空。

2) 新增数据类属性字段时，必须模拟数据对象的属性值

在上面的案例中，如果 UserDO 和 UserVO 新增了属性字段 age（用户年龄），且新增了赋值语句如下：

```
userVO.setAge(userDO.getAge());
```

如果还是用原有的数据对象执行单元测试，我们会发现单元测试用例执行通过。这是因为，由于属性字段 age 为空，赋值不赋值没有任何差别。所以，新增属性类属性字段是，必须模拟数据对象的属性值。

注意：如果用 JSON 字符串对比，且设置输出空字段，是可以触发单元测试用例执行失败的。

5. 验证数据对象准则

1) 必须验证所有数据对象

在单元测试中，必须验证所有数据对象：

- 来源于被测方法的返回值
- 来源于依赖方法的参数捕获
- 来源于被测对象的属性值
- 来源于请求参数的属性值。

具体案例可以参考《数据对象来源方式》章节。

2) 必须使用明确语义的断言

在使用断言验证数据对象时，必须使用确定语义的断言，不能使用不明确语义的断言。

正例：

```
Assert.assertTrue("返回值不为真", NumberHelper.isPositive(1));
Assert.assertEquals("用户不一致", user, userService.getUser(userId));
```

反例：

```
Assert.assertNotNull("用户不能为空", userService.getUser(userId));
Assert.assertNotEquals("用户不能一致", user, userService.getUser(userId));
```

谨防一些试图绕过本条准则的案例，试图用明确语义的断言去做不明确语义的判断。

```
Assert.assertTrue("用户不能为空", Objects.nonNull(userService.getUser(userId)));
```

3) 尽量采用整体验证方式

如果一个模型类，会根据业务需要新增字段。那么，针对这个模型类所对应的数据对象，尽量采用整体验证方式。

正例：

```
UserVO user = userService.getUser(userId);
String text = ResourceHelper.getResourceAsString(getClass(), path + "user.json");
Assert.assertEquals("用户不一致", text, JSON.toJSONString(user));
```

反例：

```
UserVO user = userService.getUser(userId);
Assert.assertEquals("用户标识不一致", Long.valueOf(123L), user.getId());
Assert.assertEquals("用户名称不一致", "changyi", user.getName());
...
```

上面这种数据验证方式，如果模型类删除了属性字段，是可以验证出来的。但是，如果模型类添加了字段，是无法验证出来的。所以，如果采用了这种验证方式，在新增了模型类属性字段后，需要梳理并补全测试用例。否则，在使用单元测试用例回归代码时，它将会告诉你这里没有任何问题。

五、 验证抛出异常

异常作为 Java 语言的重要特性，是 Java 语言健壮性的重要体现。捕获并验证抛出异常，也是测试用例的一种。所以，在单元测试中，也需要对抛出异常进行验证。

1. 抛出异常来源方式

1) 来源于属性字段的判断

判断属性字段是否非法，否则抛出异常。

```
private Map<String, MessageHandler> messageHandlerMap = ...;
public void handleMessage(Message message) {
    ...
    // 判断处理器映射非空
    if (CollectionUtils.isEmpty(messageHandlerMap)) {
        throw new ExampleException("消息处理器映射不能为空");
    }
    ...
}
```

2) 来源于输入参数的判断

判断输入参数是否合法，否则抛出异常。

```
public void handleMessage(Message message) {
    ...
    // 判断获取处理器非空
    MessageHandler messageHandler = messageHandlerMap.get(message.getType());
    if (CollectionUtils.isEmpty(messageHandler)) {
        throw new ExampleException("获取消息处理器不能为空");
    }
    ...
}
```

注意：这里采用的是 Spring 框架提供的 Assert 类，跟 if-throw 语句的效果一样。

3) 来源于返回值的判断

判断返回值是否合法，否则抛出异常。

```
public void handleMessage(Message message) {
    ...
    // 进行消息处理器处理
    boolean result = messageHandler.handleMessage(message);
    if (!reuslt) {
        throw new ExampleException("处理消息异常");
    }
    ...
}
```

4) 来源于模拟方法的调用

调用模拟的依赖方法时，可能模拟的依赖方法会抛出异常。

```
public void handleMessage(Message message) {
    ...
    // 进行消息处理器处理
    boolean result = messageHandler.handleMessage(message); // 直接抛出异常
    ...
}
```

这里，可以进行异常捕获处理，或打印输出日志，或继续抛出异常。

5) 来源于静态方法的调用

有时候，静态方法调用也有可能抛出异常。

```
// 可能会抛出IOException
String response = HttpHelper.httpGet(url, parameterMap);
```

除此之外，还有别的抛出异常来源方式，这里不再累述。

2. 抛出异常验证方式

在单元测试中，通常存在四种验证抛出异常方法。

1) 通过 try-catch 语句验证抛出异常

Java 单元测试用例中，最简单直接的异常捕获方式就是使用 try-catch 语句。

```
@Test
public void testCreateUserWithException() {
    // 模拟依赖方法
    Mockito.doReturn(true).when(userDAO).existName(Mockito.anyString());

    // 调用测试方法
    UserCreateVO userCreate = new UserCreateVO();
    try {
        userCreate.setName("changyi");
        userCreate.setDescription("Java Programmer");
        userService.createUser(userCreate);
    } catch (ExampleException e) {
        Assert.assertEquals("异常编码不一致", ErrorCode.OBJECT_EXIST, e.getCode());
        Assert.assertEquals("异常消息不一致", "用户已存在", e.getMessage());
    }

    // 验证依赖方法
    Mockito.verify(userDAO).existName(userCreate.getName());
}
```

2) 通过@Test 注解验证抛出异常

JUnit 的@Test 注解提供了一个 expected 属性，可以指定一个期望的异常类型，用来捕获并验证异常。

```
@Test(expected = ExampleException.class)
public void testCreateUserWithException() {
    // 模拟依赖方法
    Mockito.doReturn(true).when(userDAO).existName(Mockito.anyString());

    // 调用测试方法
    UserCreateVO userCreate = new UserCreateVO();
    userCreate.setName("changyi");
    userCreate.setDescription("Java Programmer");
    userService.createUser(userCreate);

    // 验证依赖方法(不会执行)
    Mockito.verify(userDAO).existName(userCreate.getName());
}
```

注意：测试用例在执行到 userService.createUser 方法后将跳出方法，导致后续验证语句无法执行。所以，这种方式无法验证异常编码、消息、原因等内容，也无法验证依赖方法及其参数。

3) 通过@Rule 注解验证抛出异常

如果想要验证异常原因和消息，就需求采用@Rule 注解定义 ExpectedException 对象，然后在测试方法的前面声明要捕获的异常类型、原因和消息。

```
@Rule
public ExpectedException exception = ExpectedException.none();
@Test
public void testCreateUserWithException1() {
    // 模拟依赖方法
    Mockito.doReturn(true).when(userDAO).existName(Mockito.anyString());

    // 调用测试方法
    UserCreateVO userCreate = new UserCreateVO();
    userCreate.setName("changyi");
    userCreate.setDescription("Java Programmer");
    exception.expect(ExampleException.class);
    exception.expectMessage("用户已存在");
    userService.createUser(userCreate);

    // 验证依赖方法(不会执行)
    Mockito.verify(userDAO).existName(userCreate.getName());
}
```

注意：测试用例在执行到 userService.createUser 方法后将跳出方法，导致后续验证语句无法执行。所以，这种方式无法验证依赖方法及其参数。由于 ExpectedException 的验证方法只支持验证异常类型、原因和消息，无法验证异常的自定义属性字段值。目前，JUnit 官方建议使用 Assert.assertThrows 替换。

4) 通过 Assert.assertThrows 方法验证抛出异常

在最新版的 JUnit 中，提供了一个更为简洁的异常验证方式——Assert.assertThrows 方法。

```
@Test
public void testCreateUserWithException() {
    // 模拟依赖方法
    Mockito.doReturn(true).when(userDAO).existName(Mockito.anyString());

    // 调用测试方法
    UserCreateVO userCreate = new UserCreateVO();
    userCreate.setName("changyi");
    userCreate.setDescription("Java Programmer");
    ExampleException exception = Assert.assertThrows("异常类型不一致", ExampleException.class,
() -> userService.createUser(userCreate));
    Assert.assertEquals("异常编码不一致", ErrorCode.OBJECT_EXIST, exception.getCode());
    Assert.assertEquals("异常消息不一致", "用户已存在", exception.getMessage());

    // 验证依赖方法
    Mockito.verify(userDAO).existName(userCreate.getName());
}
```

5) 四种抛出异常验证方式对比

根据不同的验证异常功能项，对四种抛出异常验证方式对比。结果如下：

对比内容	try-catch语句	@Test注解	@Rule注解	Assert.assertThrows方法
验证异常类型	支持	支持	支持	支持
验证异常消息	支持	不支持	支持	支持
验证异常原因	支持	不支持	支持	支持
验证自定义属性	支持	不支持	不支持	支持
验证依赖方法及其参数	支持	不支持	不支持	支持
单元测试代码优雅性	不优雅	优雅	不优雅	优雅
JUnit官方推荐使用	不推荐	推荐	不推荐	推荐

综上所述，采用 Assert.assertThrows 方法验证抛出异常是最佳的，也是 JUnit 官方推荐使用的。

3. 验证抛出异常问题

这里，以创建用户时抛出异常为例，来说明验证抛出异常时所存在的问题。

代码案例：

```
private UserDAO userDAO;
public void createUser(@Valid UserCreateVO userCreateVO) {
    try {
        UserDO userCreateDO = new UserDO();
        userCreateDO.setName(userCreateVO.getName());
        userCreateDO.setDesc(userCreateVO.getDesc());
        userDAO.create(userCreateDO);
    } catch (RuntimeException e) {
        log.error("创建用户异常：userName={}", userName, e)
        throw new ExampleException(ErrorCode.DATABASE_ERROR, "创建用户异常", e);
    }
}
```

1） 不验证抛出异常类型

反面案例：

在验证抛出异常时，很多人使用@Test 注解的 expected 属性，并且指定取值为 Exception.class，主要原因是：

- 单元测试用例的代码简洁，只有一行@Test 注解；
- 不管抛出什么异常，都能保证单元测试用例通过。

```
@Test(expected = Exception.class)
public void testCreateUserWithException() {
    // 模拟依赖方法
    Throwable e = new RuntimeException();
    Mockito.doThrow(e).when(userDAO).create(Mockito.any(UserCreateVO.class));

    // 调用测试方法
    UserCreateVO userCreateVO = ...;
  userService.createUser(userCreate);
}
```

存在问题：

上面用例指定了通用异常类型，没有对抛出异常类型进行验证。所以，如果把 ExampleException 异常改为 RuntimeException 异常，该单元测试用例是无法验证出来的。

```
throw new RuntimeException("创建用户异常", e);
```

2) 不验证抛出异常属性

反面案例：

既然需要验证异常类型，简单地指定@Test 注解的 expected 属性为 ExampleException.class 即可。

```
@Test(expected = ExampleException.class)
public void testCreateUserWithException() {
    // 模拟依赖方法
    Throwable e = new RuntimeException();
    Mockito.doThrow(e).when(userDAO).create(Mockito.any(UserCreateVO.class));

    // 调用测试方法
    UserCreateVO userCreateVO = ...;
  userService.createUser(userCreate);
}
```

存在问题：

上面用例只验证了异常类型，没有对抛出异常属性字段（异常消息、异常原因、错误编码等）进行验证。所以，如果把错误编码 DATABASE_ERROR（数据库错误）改为 PARAMETER_ERROR（参数错误），该单元测试用例是无法验证出来的。

```
throw new ExampleException(ErrorCode.PARAMETER_ERROR, "创建用户异常", e);
```

3) 只验证抛出异常部分属性

反面案例：

如果要验证异常属性，就必须用 Assert.assertThrows 方法捕获异常，并对异常的常用属性进行验证。但是，有些人为了偷懒，只验证抛出异常部分属性。

```
// 模拟依赖方法
Throwable e = new RuntimeException();
Mockito.doThrow(e).when(userDAO).create(Mockito.any(UserCreateVO.class));

// 调用测试方法
UserCreateVO userCreateVO = ...;
ExampleException exception = Assert.assertThrows("异常类型不一致", ExampleException.class, ()
-> userService.createUser(userCreateVO));
Assert.assertEquals("异常编码不一致", ErrorCode.DATABASE_ERROR, exception.getCode());
```

存在问题：

上面用例只验证了异常类型和错误编码，如果把错误消息“创建用户异常”改为“创建用户错误”，该单元测试用例是无法验证出来的。

```
throw new ExampleException(ErrorCode.DATABASE_ERROR, "创建用户错误", e);
```

4) 不验证抛出异常原因

反面案例：

先捕获抛出异常，再验证异常编码和异常消息，看起来很完美了。

```
// 模拟依赖方法
Throwable e = new RuntimeException();
Mockito.doThrow(e).when(userDAO).create(Mockito.any(UserCreateVO.class));

// 调用测试方法
UserCreateVO userCreateVO = ...;
ExampleException exception = Assert.assertThrows("异常类型不一致", ExampleException.class, ()
-> userService.createUser(userCreateVO));
Assert.assertEquals("异常编码不一致", ErrorCode.OBJECT_EXIST, exception.getCode());
Assert.assertEquals("异常消息不一致", “创建用户异常”, exception.getMessage());
```

存在问题：

通过代码可以看出，在抛出 ExampleException 异常时，最后一个参数 e 是我们模拟的 userService.createUser 方法抛出的 RuntimeException 异常。但是，我们没有对抛出异常原因进行验证。如果修改代码，把最后一个参数 e 去掉，上面的单元测试用例是无法验证出来的。

```
throw new ExampleException(ErrorCode.DATABASE_ERROR, "创建用户异常");
```

5) 不验证相关方法调用

反面案例：

很多人认为，验证抛出异常就只验证抛出异常，验证依赖方法调用不是必须的。

```
// 模拟依赖方法
Throwable e = new RuntimeException();
Mockito.doThrow(e).when(userDAO).create(Mockito.any(UserCreateVO.class));

// 调用测试方法
UserCreateVO userCreateVO = ...;
ExampleException exception = Assert.assertThrows("异常类型不一致", ExampleException.class, ()
-> userService.createUser(userCreateVO));
Assert.assertEquals("异常编码不一致", ErrorCode.OBJECT_EXIST, exception.getCode());
Assert.assertEquals("异常消息不一致", “创建用户异常”, exception.getMessage());
Assert.assertEquals("异常原因不一致", e, exception.getCause());
```

存在问题：

如果不验证相关方法调用，如何能证明代码走过这个分支？比如：我们在创建用户之前，检查用户名称无效并抛出异常。

```
// 检查用户名称有效
String userName = userCreateVO.getName();
if (StringUtils.length(userName) < USER_NAME_LENGTH) {
    throw new ExampleException(ErrorCode.INVALID_USERNAME, "无效用户名称");
}
```

6) 完美地验证抛出异常

一个完美的异常验证，除对异常类型、异常属性、异常原因等进行验证外，还需对抛出异常前的依赖方法调用进行验证。

完美案例：

```
// 模拟依赖方法
Throwable e = new RuntimeException();
Mockito.doThrow(e).when(userDAO).create(Mockito.any(UserCreateVO.class));

// 调用测试方法
String text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateVO.json");
UserCreateVO userCreateVO = JSON.parseObject(text, UserCreateVO.class);
ExampleException exception = Assert.assertThrows("异常类型不一致", ExampleException.class, ()
-> userService.createUser(userCreateVO));
Assert.assertEquals("异常编码不一致", ErrorCode.OBJECT_EXIST, exception.getCode());
Assert.assertEquals("异常消息不一致", “创建用户异常”, exception.getMessage());
Assert.assertEquals("异常原因不一致", e, exception.getCause());

// 验证依赖方法
ArgumentCaptor<UserDO> userCreateCaptor = ArgumentCaptor.forClass(UserDO.class);
Mockito.verify(userDAO).create(userCreateCaptor.capture());
text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateDO.json");
Assert.assertEquals("用户创建不一致", text, JSON.toJSONString(userCreateCaptor.getValue()));
```

4. 验证抛出异常准则

1) 必须验证所有抛出异常

在单元测试中，必须验证所有抛出异常：

- 来源于属性字段的判断
- 来源于输入参数的判断
- 来源于返回值的判断
- 来源于模拟方法的调用
- 来源于静态方法的调用

具体内容可以参考《抛出异常来源方式》章节。

2) 必须验证异常类型、异常属性、异常原因

在验证抛出异常时，必须验证异常类型、异常属性、异常原因等。

正例：

```
ExampleException exception = Assert.assertThrows("异常类型不一致", ExampleException.class, ()
-> userService.createUser(userCreateVO));
Assert.assertEquals("异常编码不一致", ErrorCode.OBJECT_EXIST, exception.getCode());
Assert.assertEquals("异常消息不一致", "用户已存在", exception.getMessage());
Assert.assertEquals("异常原因不一致", e, exception.getCause());
```

反例：

```
@Test(expected = ExampleException.class)
public void testCreateUserWithException() {
    ...
    userService.createUser(userCreateVO);
}
```

3) 验证抛出异常后，必须验证相关方法调用

在验证抛出异常后，必须验证相关方法调用，来保证单元测试用例走的是期望分支。

正例：

```
// 调用测试方法
...

// 验证依赖方法
ArgumentCaptor<UserDO> userCreateCaptor = ArgumentCaptor.forClass(UserDO.class);
Mockito.verify(userDAO).create(userCreateCaptor.capture());
text = ResourceHelper.getResourceAsString(getClass(), path + "userCreateDO.json");
Assert.assertEquals("用户创建不一致", text, JSON.toJSONString(userCreateCaptor.getValue()));
```

六、 验证方法调用

在单元测试中，验证方法调用是为了验证依赖方法的调用次数和顺序以及是否传入了期望的参数值。

1. 方法调用来源方式

1) 来源于注入对象的方法调用

最常见的方法调用就是对注入依赖对象的方法调用。

```
private UserDAO userDAO;
public UserVO getUser(Long userId) {
    UserDO user = userDAO.get(userId); // 方法调用
    return convertUser(user);
}
```

2) 来源于输入参数的方法调用

有时候，也可以通过输入参数传入依赖对象，然后调用依赖对象的方法。

```
public <T> List<T> executeQuery(String sql, DataParser<T> dataParser) {
    List<T> dataList = new ArrayList<>();
    List<Record> recordList = SQLTask.getResult(sql);
    for (Record record : recordList) {
        T data = dataParser.parse(record); // 方法调用
        if (Objects.nonNull(data)) {
            dataList.add(data);
        }
    }
    return dataList;
}
```

3) 来源于返回值的方法调用

```
private UserHsfService userHsfService;
public User getUser(Long userId) {
    Result<User> result = userHsfService.getUser(userId);
    if (!result.isSuccess()) { // 方法调用1
        throw new ExampleException("获取用户异常");
    }
    return result.getData(); // 方法调用2
}
```

4) 来源于静态方法的调用

在 Java 中，静态方法是指被 static 修饰的成员方法，不需要通过对象实例就可以被调用。在日常代码中，静态方法调用一直占有一定的比例。

```
String text = JSON.toJSONString(user); // 方法调用
```

2. 方法调用验证方式

在单元测试中，验证依赖方法调用是确认模拟对象的依赖方法是否被按照预期调用的过程。

1) 验证依赖方法的调用参数

```
// 1.验证无参数依赖方法调用
Mockito.verify(userDAO).deleteAll();

// 2.验证指定参数依赖方法调用
Mockito.verify(userDAO).delete(userId);

// 3.验证任意参数依赖方法调用
Mockito.verify(userDAO).delete(Mockito.anyLong());

// 4.验证可空参数依赖方法调用
Mockito.verify(userDAO).queryCompany(Mockito.anyLong(), Mockito.nullable(Long.class));

// 5.验证必空参数依赖方法调用
Mockito.verify(userDAO).queryCompany(Mockito.anyLong(), Mockito.isNull());

// 6.验证可变参数依赖方法调用
Mockito.verify(userService).delete(1L, 2L, 3L);
Mockito.verify(userService).delete(Mockito.any(Long.class));  // 匹配一个
Mockito.verify(userService).delete(Mockito.<Long>any()); // 匹配多个
```

2) 验证依赖方法的调用次数

```
// 1.验证依赖方法默认调用1次
Mockito.verify(userDAO).delete(userId);

// 2.验证依赖方法从不调用
Mockito.verify(userDAO, Mockito.never()).delete(userId);

// 3.验证依赖方法调用n次
Mockito.verify(userDAO, Mockito.times(n)).delete(userId);

// 4.验证依赖方法调用至少1次
Mockito.verify(userDAO, Mockito.atLeastOnce()).delete(userId);

// 5.验证依赖方法调用至少n次
Mockito.verify(userDAO, Mockito.atLeast(n)).delete(userId);

// 6.验证依赖方法调用最多1次
Mockito.verify(userDAO, Mockito.atMostOnce()).delete(userId);

// 7.验证依赖方法调用最多n次
Mockito.verify(userDAO, Mockito.atMost(n)).delete(userId);

// 8.验证依赖方法调用指定n次
Mockito.verify(userDAO, Mockito.call(n)).delete(userId); // 不会被标记为已验证

// 9.验证依赖对象及其方法仅调用1次
Mockito.verify(userDAO, Mockito.only()).delete(userId);
```

3) 验证依赖方法并捕获参数值

```
// 1.使用ArgumentCaptor.forClass方法定义参数捕获器
ArgumentCaptor<UserDO> userCaptor = ArgumentCaptor.forClass(UserDO.class);
Mockito.verify(userDAO).modify(userCaptor.capture());
UserDO user = userCaptor.getValue();

// 2.使用@Captor注解定义参数捕获器
@Captor
private ArgumentCaptor<UserDO> userCaptor;

// 3.捕获多次方法调用的参数值列表
ArgumentCaptor<UserDO> userCaptor = ArgumentCaptor.forClass(UserDO.class);
Mockito.verify(userDAO, Mockito.atLeastOnce()).modify(userCaptor.capture());
List<UserDO> userList = userCaptor.getAllValues();
```

4) 验证其它类型的依赖方法调用

```
// 1.验证 final 方法调用
final方法的验证跟普通方法类似。

// 2.验证私有方法调用
PowerMockito.verifyPrivate(mockClass, times(1)).invoke("unload", any(List.class));

// 3.验证构造方法调用
PowerMockito.verifyNew(MockClass.class).withNoArguments();
PowerMockito.verifyNew(MockClass.class).withArguments(someArgs);

// 4.验证静态方法调用
PowerMockito.verifyStatic(StringUtils.class);
StringUtils.isEmpty(string);
```

5) 验证依赖对象没有更多方法调用

```
// 1.验证模拟对象没有任何方法调用
Mockito.verifyNoInteractions(idGenerator, userDAO);

// 2.验证模拟对象没有更多方法调用
Mockito.verifyNoMoreInteractions(idGenerator, userDAO);
```

3. 验证依赖方法问题

这里，以 cacheUser（缓存用户）为例，来说明验证依赖方法时所存在的问题。

代码案例：

```
private UserCache userCache;
public boolean cacheUser(List<User> userList) {
    boolean result = true;
    for (User user : userList) {
        result = result && userCache.set(user.getId(), user);
    }
    return result;
}
```

1) 不验证依赖方法调用

反面案例：

有些人觉得，既然已经模拟了依赖方法，并且被测方法已经按照预期返回了值，就没有必要对依赖方法进行验证。

```
// 模拟依赖方法
Mockito.doReturn(true).when(userCache).set(Mockito.anyLong(), Mockito.any(User.class));

// 调用测试方法
List<User> userList = ...;
Assert.assertTrue("处理结果不为真", userService.cacheUser(userList));

// 不验证依赖方法
```

存在问题：

模拟了依赖方法，并且被测方法已经按照预期返回了值，并不代表这个依赖方法被调用或者被正确地调用。

比如：在 for 循环之前，把 userList 置为空列表，这个单元测试用例是无法验证出来的。

```
// 清除用户列表
userList = Collections.emptyList();
```

2) 不验证依赖方法调用次数

反面案例：

有些很喜欢用 Mockito.verify 的验证至少一次和任意参数的组合，因为它可以适用于任何依赖方法调用的验证。

```
// 验证依赖方法
Mockito.verify(userCache, Mockito.atLeastOnce()).set(Mockito.anyLong(),
Mockito.any(User.class));
```

存在问题：

这种方法虽然适用于任何依赖方法调用的验证，但是基本上没有任何实质作用。

比如：我们不小心，把缓存语句写了两次，这个单元测试用例是无法验证出来的。

```
// 写了两次缓存
result = result && userCache.set(user.getId(), user);
result = result && userCache.set(user.getId(), user);
```

3) 不验证依赖方法调用参数

反面案例：

既然说验证至少一次有问题，那我就指定一下验证次数。

```
// 验证依赖方法
Mockito.verify(userCache, Mockito.times(userList.size())).set(Mockito.anyLong(),
Mockito.any(User.class));
```

存在问题：

验证方法次数的问题虽然解决了，但是验证方法参数的问题任然存在。

比如：我们不小心，把循环缓存每一个用户写成循环缓存第一个用户，这个单元测试用例是无法验证出来的。

```
User user = userList.get(0);
for (int i = 0; i < userList.size(); i++) {
    result = result && userCache.set(user.getId(), user);
}
```

4) 不验证所有依赖方法调用

反面案例：

不能用任意参数验证方法，那只好用实际参数验证方法了。但是，验证所有依赖方法调用代码太多，所以验证一两个依赖方法调用意思意思就行了。

```
Mockito.verify(userCache).set(user1.getId(), user1);
Mockito.verify(userCache).set(user2.getId(), user2);
```

存在问题：

如果只验证了一两个方法调用，只能保障这一两个方法调用没有问题。

比如：我们不小心，在 for 循环之后，还进行了一个用户缓存。

```
// 缓存最后一个用户
User user = userList.get(userList.size() - 1);
userCache.set(user.getId(), user);
```

5) 验证所有依赖方法调用

反面案例：

既然不验证所有方法调用有问题，那我就把所有方法调用验证了吧。

```
for (User user : userList) {
    Mockito.verify(userCache).set(user.getId(), user);
}
```

存在问题：

所有方法调用都被验证了，看起来应该没有问题了。但是，如果缓存用户方法中，存在别的方法调用。

比如：我们在进入缓存用户方法之前，新增了清除所有用户缓存，这个单元测试用是无法验证的。

```
// 删除所有用户缓存
userCache.clearAll();
```

6) 完美地验证依赖方法调用

验证所有的方法调用，只能保证现在的逻辑没有问题。如果涉及新增方法调用，这个单元测试用例是无法验证出来的。所有，我们需要验证所有依赖对象没有更多方法调用。

完美案例：

```
// 验证依赖方法
ArgumentCaptor<Long> userIdCaptor = ArgumentCaptor.forClass(Long.class);
ArgumentCaptor<User> userCaptor = ArgumentCaptor.forClass(User.class);
Mockito.verify(userCache, Mockito.atLeastOnce()).set(userIdCaptor.capture(),
userCaptor.capture());
Assert.assertEquals("用户标识列表不一致", userIdList, userIdCaptor.getAllValues());
Assert.assertEquals("用户信息列表不一致", userList, userCaptor.getAllValues());

// 验证依赖对象
Mockito.verifyNoMoreInteractions(userCache);
```

注意：利用 ArgumentCaptor（参数捕获器），不但可以验证参数，还可以验证调用次数和顺序。

4. 验证方法调用准则

1) 必须验证所有的模拟方法调用

在单元测试中，涉及到的所有模拟方法都要被验证：

- 来源于注入对象的方法调用
- 来源于输入参数的方法调用
- 来源于返回值的方法调用
- 来源于静态方法的调用

具体案例可以参考《方法调用来源方式》章节。

2) 必须验证所有的模拟对象没有更多方法调用

在单元测试中，为了防止被测方法中存在或新增别的方法调用，必须验证所有的模拟对象没有更多方法调用。

正例：

```
// 验证依赖对象
Mockito.verifyNoMoreInteractions(userDAO, userCache);
```

备注：

作者喜欢在@After 方法中对所有模拟对象进行验证，这样就不必在每个单元测试用例中验证模拟对象。

```
@After
public void afterTest() {
    Mockito.verifyNoMoreInteractions(userDAO, userCache);
}
```

可惜 Mockito.verifyNoMoreInteractions 不支持无参数就验证所有模拟对象的功能，否则这段代码会变得更简洁。

3) 必须使用明确语义的参数值或匹配器

验证依赖方法时，必须使用明确语义的参数值或匹配器，不能使用任何不明确语义的匹配器，比如：any 系列参数匹配器。

正例：

```
Mockito.verify(userDAO).get(userId);
Mockito.verify(userDAO).query(Mockito.eq(companyId), Mockito.isNull());
```

反例：

```
Mockito.verify(userDAO).get(Mockito.anyLong());
Mockito.verify(userDAO).query(Mockito.anyLong(), Mockito.isNotNull());
```

后记

最后，根据本文所表达的观点，即兴赋诗七言绝句一首：

《单元测试》

单元测试分真假，
工匠精神贯始终。
覆盖追求非目的，
回归验证显奇功。

意思是：

- 一定要知道如何去分辨单元测试的真假，
- 一定要把工匠精神贯彻单元测试的始终。
- 追求单测覆盖率并不是单元测试的目的，
- 回归验证代码才能彰显单元测试的功效。

案例集锦：Java 单元测试典型案例集锦

前言

近期，阿里巴巴 CTO 线卓越工程小组举办了阿里巴巴第一届单元测试比赛《这！就是单测》并取得了圆满成功。本人有幸作为评委，在仔细地阅读了各个小组的单元测试用例后，发现了两大单元测试问题：

- **无效验证问题：**

不进行有效地验证数据对象、抛出异常和调用方法。

- **测试方法问题：**

不知道如何测试某些典型案例，要么错误地测试、要么不进行测试、要么利用集成测试来保证覆盖率。比如：

  - 错误地测试：利用测试返回节点占比来测试随机负载均衡策略；
  - 不进行测试：没有人针对虚基类进行单独地测试；
  - 利用集成测试：很多案例中，直接注入真实依赖对象，然后一起进行集成测试。

针对无效验证问题，在我的 ATA 文章《那些年，我们写过的无效单元测试》中，介绍了如何识别和解决单元测试无效验证问题，这里就不再累述了。在本文中，作者收集了一些的 Java 单元测试典型案例，主要是为了解决这个测试方法问题。

一、 如何测试不可达代码

在程序代码中，由于无法满足进入条件，永远都不会执行到的代码，我们称之为“不可达代码”。不可达代码的危害主要有：复杂了代码逻辑，增加了代码运行和维护成本。不可达代码是可以由单元测试检测出来的——不管如何构造单元测试用例，都无法覆盖到不可达代码。

1. 案例代码

在下面的案例代码中，就存在一段不可达代码。

```
/**
 * 交易订单服务类
 */
@Service
public class TradeOrderService {
    /** 注入依赖对象 */
    /** 交易订单 DAO */
    @Autowired
    private TradeOrderDAO tradeOrderDAO;

    /**
     * 查询交易订单
     *
     * @param orderQuery 订单查询
     * @return 交易订单分页
     */
    public PageDataVO<TradeOrderVO>
queryTradeOrder(TradeOrderQueryVO orderQuery) {
        // 查询交易订单
        // 查询交易订单：总共数量
        Long totalSize =
tradeOrderDAO.countByCondition(orderQuery);
        // 查询交易订单：数据列表
        List<TradeOrderVO> dataList = null;
        if (NumberHelper.isPositive(totalSize)) {
            List<TradeOrderDO> tradeOrderList =
tradeOrderDAO.queryByCondition(orderQuery);
            if (CollectionUtils.isNotEmpty(tradeOrderList)) {
                dataList = convertTradeOrders(tradeOrderList);
            }
        }

        // 返回分页数据
        return new PageDataVO<>(totalSize, dataList);
    }

    /**
     * 转化交易订单列表
     *
     * @param tradeOrderList 交易订单 DO 列表
```

```
     * @return 交易订单 VO 列表
     */
    private static List<TradeOrderVO>
convertTradeOrders(List<TradeOrderDO> tradeOrderList) {
        // 检查订单列表
        if (CollectionUtils.isEmpty(tradeOrderList)) {
            return Collections.emptyList();
        }

        // 转化订单列表
        return
tradeOrderList.stream().map(TradeOrderService::convertTradeOrder
)
                .collect(Collectors.toList());
    }

    /**
     * 转化交易订单
     *
     * @param tradeOrder 交易订单 DO
     * @return 交易订单 VO
     */
    private static TradeOrderVO convertTradeOrder(TradeOrderDO
tradeOrder) {
        TradeOrderVO tradeOrderVO = new TradeOrderVO();
        tradeOrderVO.setId(tradeOrder.getId());
        // ...
        return tradeOrderVO;
    }
}
```

由于方法 convertTradeOrders（转化交易订单列表）传入的参数 tradeOrderList（交易订单列表）不可能为空，所以“检查订单列表”这段代码是不可达代码。

```
        // 检查订单列表
        if (CollectionUtils.isEmpty(tradeOrderList)) {
            return Collections.emptyList();
        }
```

2. 方案 1：删除不可达代码（推荐）

最简单的方法，就是删除方法 convertTradeOrders（转化交易订单列表）中的不可达代码。

```
/**
 * 转化交易订单列表
 *
 * @param tradeOrderList 交易订单DO列表
 * @return 交易订单VO列表
 */
private static List<TradeOrderVO> convertTradeOrders(List<TradeOrderDO> tradeOrderList) {
    return tradeOrderList.stream().map(TradeOrderService2::convertTradeOrder)
        .collect(Collectors.toList());
}
```

3. 方案 2：利用不可达代码（推荐）

还有一种方法，把不可达代码利用起来，可以降低方法 queryTradeOrder（查询交易订单）的代码复杂度。

```
/**
 * 查询交易订单
 *
 * @param orderQuery 订单查询
 * @return 交易订单分页
 */
public PageDataVO<TradeOrderVO> queryTradeOrder(TradeOrderQueryVO orderQuery) {
    // 查询交易订单
    // 查询交易订单：总共数量
    Long totalSize = tradeOrderDAO.countByCondition(orderQuery);
    // 查询交易订单：数据列表
    List<TradeOrderVO> dataList = null;
    if (NumberHelper.isPositive(totalSize)) {
        List<TradeOrderDO> tradeOrderList = tradeOrderDAO.queryByCondition(orderQuery);
        dataList = convertTradeOrders(tradeOrderList);
    }

    // 返回分页数据
    return new PageDataVO<>(totalSize, dataList);
}
```

4. 方案 3：测试不可达代码（不推荐）

对于一些祖传代码，有些小伙伴不敢删除代码。在某些情况下，可以针对不可达代码进行单独测试。

```
/**
 * 测试：转化交易订单列表-交易订单列表为空
 *
 * @throws Exception 异常信息
 */
@Test
public void testConvertTradeOrdersWithTradeOrderListEmpty() throws Exception {
    List<TradeOrderDO> tradeOrderList = null;
    Assert.assertSame("交易订单列表不为空", Collections.emptyList(),
        Whitebox.invokeMethod(TradeOrderService1.class, "convertTradeOrders",
tradeOrderList));
}
```

二、 如何测试内部的构造方法

在这次单元测试总决赛中，有一个随机负载均衡策略，需要针对 Random（随机数）进行单元测试。

1. 代码案例

按照题目要求，编写了一个简单的随机负载均衡策略。

```
/**
 * 随机负载均衡策略类
 */
public class RandomLoadBalanceStrategy implements LoadBalanceStrategy {
    /**
     * 选择服务节点
     *
     * @param serverNodeList 服务节点列表
     * @param clientRequest 客户请求
     * @return 服务节点
     */
    @Override
    public ServerNode selectNode(List<ServerNode> serverNodeList, ClientRequest
clientRequest) {
        // 检查节点列表
        if (CollectionUtils.isEmpty(serverNodeList)) {
            return null;
        }

        // 计算随机序号
        int totalWeight = serverNodeList.stream().mapToInt(ServerNode::getWeight).sum();
        int randomIndex = new Random().nextInt(totalWeight);

        // 查找对应节点
        for (ServerNode serverNode : serverNodeList) {
            int currentWeight = serverNode.getWeight();
            if (currentWeight > randomIndex) {
                return serverNode;
            }
            randomIndex -= currentWeight;
        }
        return null;
    }
}
```

2. 方法 1：直接测试法（不推荐）

有些参赛选手，不知道如何测试随机数（主要原因是因为不知道如何 Mock 构造方法），所以直接利用测试返回节点占比来测试随机负载均衡策略。

```
/**
 * 随机负载均衡策略测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class RandomLoadBalanceStrategyTest {
    /** 定义测试对象 */
    /** 随机负载均衡策略 */
    @InjectMocks
    private RandomLoadBalanceStrategy randomLoadBalanceStrategy;

    /**
     * 测试：选择服务节点-随机
     *
     * @throws Exception 异常信息
     */
    @Test
    public void testSelectNodeWithRandom() throws Exception {
        int nodeCount1 = 0;
        int nodeCount2 = 0;
        int nodeCount3 = 0;
        ServerNode serverNode1 = new ServerNode(1L, 10);
        ServerNode serverNode2 = new ServerNode(2L, 20);
        ServerNode serverNode3 = new ServerNode(3L, 30);
        List<ServerNode> serverNodeList =
Arrays.asList(serverNode1, serverNode2, serverNode3);
        ClientRequest clientRequest = new ClientRequest();
        for (int i = 0; i < 1000; i++) {
            ServerNode serviceNode =
randomLoadBalanceStrategy.selectNode(serverNodeList,
clientRequest);
            if (serviceNode == serverNode1) {
                nodeCount1++;
            } else if (serviceNode == serverNode2) {
                nodeCount2++;
            } else if (serviceNode == serverNode3) {
                nodeCount3++;
            }
        }
```

```
        Assert.assertEquals("节点 1 占比不一致",
serverNode1.getWeight() / 60.0D, nodeCount1 / 1000.0D, 1E-3D);
        Assert.assertEquals("节点 2 占比不一致",
serverNode2.getWeight() / 60.0D, nodeCount2 / 1000.0D, 1E-3D);
        Assert.assertEquals("节点 3 占比不一致",
serverNode3.getWeight() / 60.0D, nodeCount3 / 1000.0D, 1E-3D);
    }
}
```

这个测试用例主要存在 3 个问题：

- 执行时间长：被测方法需要被执行 1000 遍；
- 不一定通过：由于随机数是随机，并不一定保证比例，所以导致测试用例并不一定通过；
- 测试目标变更：单测测试的测试目标应该是负载均衡逻辑，现在感觉测试目标变成了 Random 方法。

3. 方法 2：直接 mock 法（不推荐）

用过 PowerMockito 高级功能的，知道如何去 Mock 构造方法。

```
/**
 * 随机负载均衡策略测试类
 */
@RunWith(PowerMockRunner.class)
@PrepareForTest(RandomLoadBalanceStrategy.class)
public class RandomLoadBalanceStrategyTest {
    /** 定义测试对象 */
    /** 随机负载均衡策略 */
    @InjectMocks
    private RandomLoadBalanceStrategy randomLoadBalanceStrategy;

    /**
     * 测试：选择服务节点-第一个节点
     *
     * @throws Exception 异常信息
     */
    @Test
    public void testSelectNodeWithFirstNode() throws Exception {
        // 模拟依赖方法
        Random random = Mockito.mock(Random.class);
```

```
Mockito.doReturn(9).when(random).nextInt(Mockito.anyInt());

PowerMockito.whenNew(Random.class).withNoArguments().thenReturn(
random);

        // 调用测试方法
        ServerNode serverNode1 = new ServerNode(1L, 10);
        ServerNode serverNode2 = new ServerNode(2L, 20);
        ServerNode serverNode3 = new ServerNode(3L, 30);
        List<ServerNode> serverNodeList =
Arrays.asList(serverNode1, serverNode2, serverNode3);
        ClientRequest clientRequest = new ClientRequest();
        ServerNode serviceNode =
randomLoadBalanceStrategy.selectNode(serverNodeList,
clientRequest);
        Assert.assertEquals("服务节点不一致", serverNode1,
serviceNode);

        // 验证依赖方法
        int totalWeight =
serverNodeList.stream().mapToInt(ServerNode::getWeight).sum();
        Mockito.verify(random).nextInt(totalWeight);
    }
}
```

但是，这个测试用例也存在问题：需要把 RandomLoadBalanceStrategy 加到 @PrepareForTest 注解中，导致 Jacoco 无法统计单元测试的覆盖率。

4. 方法 3：工具方法法（推荐）

其实，随机数生成，还有很多工具方法，我们可以利用工具方法 RandomUtils.nextInt 代替构造方法。

1) 重构代码

```
/**
 * 随机负载均衡策略类
 */
public class RandomLoadBalanceStrategy implements LoadBalanceStrategy {
    /**
     * 选择服务节点
     *
     * @param serverNodeList 服务节点列表
     * @param clientRequest 客户请求
     * @return 服务节点
     */
    @Override
    public ServerNode selectNode(List<ServerNode> serverNodeList, ClientRequest
clientRequest) {
        // 检查节点列表
        if (CollectionUtils.isEmpty(serverNodeList)) {
            return null;
        }

        // 计算随机序号
        int totalWeight = serverNodeList.stream().mapToInt(ServerNode::getWeight).sum();
        int randomIndex = RandomUtils.nextInt(0, totalWeight);

        // 查找对应节点
        for (ServerNode serverNode : serverNodeList) {
            int currentWeight = serverNode.getWeight();
            if (currentWeight > randomIndex) {
                return serverNode;
            }
            randomIndex -= currentWeight;
        }
        return null;
    }
}
```

2) 测试用例

```
/**
 * 随机负载均衡策略测试类
 */
@RunWith(PowerMockRunner.class)
@PrepareForTest(RandomUtils.class)
public class RandomLoadBalanceStrategyTest {
    /** 定义测试对象 */
    /** 随机负载均衡策略 */
    @InjectMocks
    private RandomLoadBalanceStrategy randomLoadBalanceStrategy;

    /**
     * 测试：选择服务节点-第一个节点
```

```
     */
    @Test
    public void testSelectNodeWithFirstNode() {
        // 模拟依赖方法
        PowerMockito.mockStatic(RandomUtils.class);
        PowerMockito.when(RandomUtils.nextInt(Mockito.eq(0),
Mockito.anyInt())).thenReturn(9);

        // 调用测试方法
        ServerNode serverNode1 = new ServerNode(1L, 10);
        ServerNode serverNode2 = new ServerNode(2L, 20);
        ServerNode serverNode3 = new ServerNode(3L, 30);
        List<ServerNode> serverNodeList =
Arrays.asList(serverNode1, serverNode2, serverNode3);
        ClientRequest clientRequest = new ClientRequest();
        ServerNode serviceNode =
randomLoadBalanceStrategy.selectNode(serverNodeList,
clientRequest);
        Assert.assertEquals("服务节点不一致", serverNode1,
serviceNode);

        // 验证依赖方法
        PowerMockito.verifyStatic(RandomUtils.class);
        int totalWeight =
serverNodeList.stream().mapToInt(ServerNode::getWeight).sum();
        RandomUtils.nextInt(0, totalWeight);
    }
}
```

5. 方法 4：注入对象法（推荐）

如果不愿意使用工具方法，也可以注入依赖对象，我们可以利用 RandomProvider（随机数提供者）来代替构造方法。

1) 重构代码

```
/**
 * 随机负载均衡策略类
 */
public class RandomLoadBalanceStrategy implements
LoadBalanceStrategy {
    /** 注入依赖对象 */
    /** 随机数提供者 */
```

```
    @Autowired
    private RandomProvider randomProvider;

    /**
     * 选择服务节点
     *
     * @param serverNodeList 服务节点列表
     * @param clientRequest 客户请求
     * @return 服务节点
     */
    @Override
    public ServerNode selectNode(List<ServerNode> serverNodeList,
ClientRequest clientRequest) {
        // 检查节点列表
        if (CollectionUtils.isEmpty(serverNodeList)) {
            return null;
        }

        // 计算随机序号
        int totalWeight =
serverNodeList.stream().mapToInt(ServerNode::getWeight).sum();
        int randomIndex = randomProvider.nextInt(totalWeight);

        // 查找对应节点
        for (ServerNode serverNode : serverNodeList) {
            int currentWeight = serverNode.getWeight();
            if (currentWeight > randomIndex) {
                return serverNode;
            }
            randomIndex -= currentWeight;
        }
        return null;
    }
}
```

2) 测试用例

```
/**
 * 随机负载均衡策略测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class RandomLoadBalanceStrategyTest {
    /** 模拟依赖方法 */
    /** 随机数提供者 */
```

```
    @Mock
    private RandomProvider randomProvider;

    /** 定义测试对象 */
    /** 随机负载均衡策略 */
    @InjectMocks
    private RandomLoadBalanceStrategy randomLoadBalanceStrategy;

    /**
     * 测试：选择服务节点-第一个节点
     */
    @Test
    public void testSelectNodeWithFirstNode() {
        // 模拟依赖方法

Mockito.doReturn(9).when(randomProvider).nextInt(Mockito.anyInt(
));

        // 调用测试方法
        ServerNode serverNode1 = new ServerNode(1L, 10);
        ServerNode serverNode2 = new ServerNode(2L, 20);
        ServerNode serverNode3 = new ServerNode(3L, 30);
        List<ServerNode> serverNodeList =
Arrays.asList(serverNode1, serverNode2, serverNode3);
        ClientRequest clientRequest = new ClientRequest();
        ServerNode serviceNode =
randomLoadBalanceStrategy.selectNode(serverNodeList,
clientRequest);
        Assert.assertEquals("服务节点不一致", serverNode1,
serviceNode);

        // 验证依赖方法
        int totalWeight =
serverNodeList.stream().mapToInt(ServerNode::getWeight).sum();
        Mockito.verify(randomProvider).nextInt(totalWeight);
    }
}
```

三、 如何测试虚基类和子类

在这次单元测试比赛中，很多选手都编写了虚基类，但是没有看到任何一个选手针对虚基类进行了单独的测试。

1. 案例代码

这里，以 Diamond 属性配置加载为例说明。

1) 虚基类定义

首先，定义一个通用的虚基类，定义了需要子类实现的虚方法，实现了通用的配置解析方法。

```
/**
 * 虚属性回调类
 *
 * @param <T> 配置类型
 */
@Slf4j
public abstract class AbstractPropertiesCallback<T> implements
DiamondDataCallback {
    /** 注入依赖对象 */
    /** 环境 */
    @Autowired
    private Environment environment;
    /** 转化服务 */
    @Autowired
    private ConversionService conversionService;

    /**
     * 接收到数据
     *
     * @param data 配置数据
     */
    @Override
    public void received(String data) {
        // 获取配置参数
        String configName = getConfigName();
        Assert.notNull(configName, "配置名称不能为空");
        T configInstance = getConfigInstance();
        Assert.notNull(configInstance, "配置实例不能为空");

        // 解析配置数据
        try {
            log.info("绑定属性配置文件开始: configName={}",
configName);
            Properties properties = new Properties();
```

```
            byte[] bytes =
Optional.ofNullable(data.getBytes()).orElseGet(() -> new
byte[0]);
            InputStream inputStream = new
ByteArrayInputStream(bytes);
            BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(inputStream));
            properties.load(bufferedReader);
            Bindable<T> bindable =
Bindable.ofInstance(configInstance);
            Binder binder = new
Binder(ConfigurationPropertySources.from(
                new PropertiesPropertySource(configName,
properties)),
                new
PropertySourcesPlaceholdersResolver(environment),
conversionService);
            BindResult<T> result = binder.bind(configName,
bindable);
            if (!result.isBound()) {
                log.error("绑定属性配置文件失败: configName={}",
configName);
                return;
            }
            log.info("绑定属性配置文件成功: configName={},
configInstance={}", configName,
JSON.toJSONString(configInstance));
        } catch (IOException | RuntimeException e) {
            log.error("绑定属性配置文件异常: configName={}",
configName, e);
        }
    }

    /**
     * 获取配置名称
     *
     * @return 配置名称
     */
    @NonNull
    protected abstract String getConfigName();

    /**
     * 获取配置实例
     *
     * @return 配置实例
```

```
     */
    @NonNull
    protected abstract T getConfigInstance();
}
```

2) 子类实现

其次，定义了具体配置的子类，简单地实现了基类定义的虚方法。

```
/**
 * 例子配置回调类
 */
@DiamondListener(groupId = "unittest-example", dataId = "example.properties",
executeAfterInit = true)
public class ExampleConfigCallback extends AbstractPropertiesCallback<ExampleConfig> {
    /** 注入依赖对象 */
    /** 例子配置 */
    @Resource
    private ExampleConfig exampleConfig;

    /**
     * 获取配置名称
     *
     * @return 配置名称
     */
    @Override
    protected String getConfigName() {
        return "example";
    }

    /**
     * 获取配置实例
     *
     * @return 配置实例
     */
    @Override
    protected ExampleConfig getConfigInstance() {
        return exampleConfig;
    }
}
```

2. 方法 1：联合测试法（不推荐）

最简单的测试方法，就是通过子类对虚基类进行联合测试，这样同时把子类和虚基类都测试了。

```
/**
 * 例子配置回调测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class ExampleConfigCallbackTest {
    /** 定义静态常量 */
    /** 资源路径 */
    private static final String RESOURCE_PATH =
"testExampleConfigCallback/";

    /** 模拟依赖对象 */
    /** 配置环境 */
    @Mock
    private ConfigurableEnvironment environment;
    /** 转化服务 */
    @Mock
    private ConversionService conversionService;

    /** 定义测试对象 */
    /** BOSS 取消费配置回调 */
    @InjectMocks
    private ExampleConfigCallback exampleConfigCallback;

    /**
     * 测试：接收-正常
     */
    @Test
    public void testReceivedWithNormal() {
        // 模拟依赖对象
        ExampleConfig exampleConfig = new ExampleConfig();
        Whitebox.setInternalState(exampleConfigCallback,
"exampleConfig", exampleConfig);

        // 调用测试方法
        String text =
ResourceHelper.getResourceAsString(getClass(), RESOURCE_PATH +
"exampleConfig.properties");
        exampleConfigCallback.received(text);

        // 验证依赖对象
        text = ResourceHelper.getResourceAsString(getClass(),
RESOURCE_PATH + "exampleConfig.json");
        Assert.assertEquals("取消费用配置不一致", text,
JSON.toJSONString(exampleConfig,
SerializerFeature.MapSortField));
```

```
    }
}
```

3. 方法 2：独立测试法（推荐）

其实，更好的方法是对虚基类和子类独立单元测试。

1) 基类测试

虚基类的单元测试，专注于虚基类的通用配置解析。

```
/**
 * 虚属性回调测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class AbstractPropertiesCallbackTest {
    /** 静态常量相关 */
    /** 资源目录 */
    private static final String RESOURCE_PATH =
"testAbstractPropertiesCallback/";

    /** 模拟依赖对象 */
    /** 环境 */
    @Mock
    private ConfigurableEnvironment environment;
    /** 转化服务 */
    @Mock
    private ConversionService conversionService;

    /** 定义测试对象 */
    /** 虚属性回调 */
    @InjectMocks
    private AbstractPropertiesCallback<ExampleConfig>
propertiesCallback =

CastUtils.cast(Mockito.spy(AbstractPropertiesCallback.class));

    /**
     * 测试：接收到-正常
     */
    @Test
    public void testReceivedWithNormal() {
        // 模拟依赖方法
```

```
        // 模拟依赖方法: propertiesCallback.getConfigName
        String configName = "example";

Mockito.doReturn(configName).when(propertiesCallback).getConfigN
ame();
        // 模拟依赖方法: propertiesCallback.getConfigInstance
        ExampleConfig configInstance = new ExampleConfig();

Mockito.doReturn(configInstance).when(propertiesCallback).getCon
figInstance();

        // 调用测试方法
        String text1 =
ResourceHelper.getResourceAsString(getClass(), RESOURCE_PATH +
"exampleConfig.properties");
        propertiesCallback.received(text1);
        String text2 =
ResourceHelper.getResourceAsString(getClass(), RESOURCE_PATH +
"exampleConfig.json");
        Assert.assertEquals("任务配置不一致", text2,
JSON.toJSONString(configInstance));

        // 验证依赖方法
        // 验证依赖方法: propertiesCallback.received
        Mockito.verify(propertiesCallback).received(text1);
        // 验证依赖方法: propertiesCallback.getConfigName
        Mockito.verify(propertiesCallback).getConfigName();
        // 验证依赖方法: propertiesCallback.getConfigInstance
        Mockito.verify(propertiesCallback).getConfigInstance();
    }
}
```

2) 子类测试

子类的单元测试，专注于对虚基类定义虚方法的实现，避免了每个子类都要针对虚基类的通用配置解析进行测试。

```
/**
 * 例子配置回调测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class ExampleConfigCallbackTest {
    /** 定义测试对象 */
    /** BOSS取消费配置回调 */
    @InjectMocks
    private ExampleConfigCallback exampleConfigCallback;

    /**
     * 测试：获取配置实例
     */
    @Test
    public void testGetConfigInstance() {
        Assert.assertEquals("配置实例不一致", exampleConfig,
exampleConfigCallback.getConfigInstance());
    }

    /**
     * 测试：获取配置名称
     */
    @Test
    public void testGetConfigName() {
        Assert.assertEquals("配置名称不一致", "example",
exampleConfigCallback.getConfigName());
    }
}
```

四、 如何测试策略模式的策略服务

1. 案例代码

在这次单元测试比赛中，很多选手都编写了策略服务类，但是没有看到任何一个选手针对策略服务类进行了单独的测试。这里，还是以负载均衡的策略服务为例说明。

1） 策略接口

首先，定义一个负载均衡策略接口。

```
/**
 * 负载均衡策略接口
 */
public interface LoadBalanceStrategy {
    /**
     * 支持策略类型
     *
     * @return 策略类型
     */
    LoadBalanceStrategyType supportType();

    /**
     * 选择服务节点
     *
     * @param serverNodeList 服务节点列表
     * @param clientRequest 客户请求
     * @return 服务节点
     */
    ServerNode selectNode(List<ServerNode> serverNodeList, ClientRequest clientRequest);
}
```

2) 策略服务

其次，实现一个负载均衡策略服务，根据负载均衡策略类型选择对应的负载均衡策略来执行。

```
/**
 * 负载均衡服务类
 */
public class LoadBalanceService {
    /** 负载均衡策略映射 */
    private final Map<LoadBalanceStrategyType,
LoadBalanceStrategy> strategyMap;

    /**
     * 构造方法
     *
     * @param strategyList 负载均衡策略列表
     */
    public LoadBalanceService(List<LoadBalanceStrategy>
strategyList) {
        strategyMap = new
EnumMap<>(LoadBalanceStrategyType.class);
        for (LoadBalanceStrategy strategy : strategyList) {
            strategyMap.put(strategy.supportType(), strategy);
        }
    }
```

```
    /**
     * 选择服务节点
     *
     * @param strategyType 策略类型
     * @param serverNodeList 服务节点列表
     * @param clientRequest 客户请求
     * @return 服务节点
     */
    public ServerNode selectNode(LoadBalanceStrategyType
strategyType,
        List<ServerNode> serverNodeList, ClientRequest
clientRequest) {
        // 获取负载均衡策略
        LoadBalanceStrategy strategy =
strategyMap.get(strategyType);
        if (Objects.isNull(strategy)) {
            throw new BusinessException("负载均衡策略不存在");
        }

        // 执行负载均衡策略
        return strategy.selectNode(serverNodeList,
clientRequest);
    }
}
```

3) 策略实现

最后，实现一个随机负载均衡策略实现类。

```
/**
 * 随机负载均衡策略类
 */
public class RandomLoadBalanceStrategy implements
LoadBalanceStrategy {
    /**
     * 支持策略类型
     *
     * @return 策略类型
     */
    @Override
    public LoadBalanceStrategyType supportType() {
        return LoadBalanceStrategyType.RANDOM;
    }
```

```
    /**
     * 选择服务节点
     *
     * @param serverNodeList 服务节点列表
     * @param clientRequest 客户请求
     * @return 服务节点
     */
    @Override
    public ServerNode selectNode(List<ServerNode> serverNodeList,
ClientRequest clientRequest) {
        // 检查节点列表
        if (CollectionUtils.isEmpty(serverNodeList)) {
            return null;
        }

        // 计算随机序号
        int totalWeight =
serverNodeList.stream().mapToInt(ServerNode::getWeight).sum();
        int randomIndex = RandomUtils.nextInt(0, totalWeight);

        // 查找对应节点
        for (ServerNode serverNode : serverNodeList) {
            int currentWeight = serverNode.getWeight();
            if (currentWeight > randomIndex) {
                return serverNode;
            }
            randomIndex -= currentWeight;
        }
        return null;
    }
}
```

2. 方法 1：联合测试法（不推荐）

很多时候，策略模式是用来优化 if-else 代码的。所以，采用联合测试法（策略服务和策略实现同时测试），能够最大限度地利用原有的单元测试代码。

```
/**
 * 负载均衡服务测试类
 */
@RunWith(PowerMockRunner.class)
@PrepareForTest(RandomUtils.class)
```

```
public class LoadBalanceServiceTest {
    /**
     * 测试：选择服务节点-正常
     */
    @Test
    public void testSelectNodeWithNormal() {
        // 模拟依赖方法
        PowerMockito.mockStatic(RandomUtils.class);
        PowerMockito.when(RandomUtils.nextInt(Mockito.eq(0),
Mockito.anyInt())).thenReturn(9);

        // 调用测试方法
        ServerNode serverNode1 = new ServerNode(1L, 10);
        ServerNode serverNode2 = new ServerNode(2L, 20);
        ServerNode serverNode3 = new ServerNode(3L, 30);
        List<ServerNode> serverNodeList =
Arrays.asList(serverNode1, serverNode2, serverNode3);
        ClientRequest clientRequest = new ClientRequest();
        RandomLoadBalanceStrategy randomLoadBalanceStrategy = new
RandomLoadBalanceStrategy();
        LoadBalanceService loadBalanceService = new
LoadBalanceService(Arrays.asList(randomLoadBalanceStrategy));
        ServerNode serviceNode =
loadBalanceService.selectNode(LoadBalanceStrategyType.RANDOM,
            serverNodeList, clientRequest);
        Assert.assertEquals("服务节点不一致", serverNode1,
serviceNode);

        // 验证依赖方法
        PowerMockito.verifyStatic(RandomUtils.class);
        int totalWeight =
serverNodeList.stream().mapToInt(ServerNode::getWeight).sum();
        RandomUtils.nextInt(0, totalWeight);
    }
}
```

策略模式的联合测试法主要有以下问题：

- 策略服务依赖于策略实现，需要了解策略实现的具体逻辑，才能写出策略服务的单元测试；
- 对于策略服务来说，该单元测试并不关心策略服务的实现，这是黑盒测试而不是白盒测试。

如果我们对策略服务进行以下破坏，该单元测试并不能发现问题：

- strategyMap 没有根据 strategyList 生成；
- strategyMap.get(strategyType) 为空时，初始化一个 RandomLoadBalanceStrategy。

```
/**
 * 负载均衡服务类
 */
public class LoadBalanceService {
    /** 负载均衡策略映射 */
    private final Map<LoadBalanceStrategyType,
LoadBalanceStrategy> strategyMap;

    /**
     * 构造方法
     *
     * @param strategyList 负载均衡策略列表
     */
    public LoadBalanceService(List<LoadBalanceStrategy>
strategyList) {
        strategyMap = new
EnumMap<>(LoadBalanceStrategyType.class);
    }

    /**
     * 选择服务节点
     *
     * @param strategyType 策略类型
     * @param serverNodeList 服务节点列表
     * @param clientRequest 客户请求
     * @return 服务节点
     */
    public ServerNode selectNode(LoadBalanceStrategyType
strategyType,
        List<ServerNode> serverNodeList, ClientRequest
clientRequest) {
        // 获取负载均衡策略
        LoadBalanceStrategy strategy =
strategyMap.get(strategyType);
        if (Objects.isNull(strategy)) {
            strategy = new RandomLoadBalanceStrategy();
        }
```

```
        // 执行负载均衡策略
        return strategy.selectNode(serverNodeList,
clientRequest);
    }
}
```

3. 方法 2：独立测试法（推荐）

现在，先假设策略实现 RandomLoadBalanceStrategy（随机负载均衡策略）不存在，直接对策略服务 LoadBalanceService（负载均衡服务）独立测试，而且是分别对构造方法和 selectNode（选择服务节点）方法进行独立测试。其中，测试构造方法是为了保证 strategyMap 构造逻辑没有问题，测试 selectNode（选择服务节点）方法是为了保证选择策略逻辑没有问题。

```
/**
 * 负载均衡服务测试类
 */
public class LoadBalanceServiceTest {
    /**
     * 测试：构造方法
     */
    @Test
    public void testConstructor() {
        // 模拟依赖方法
        LoadBalanceStrategy loadBalanceStrategy =
Mockito.mock(LoadBalanceStrategy.class);

Mockito.doReturn(LoadBalanceStrategyType.RANDOM).when(loadBalanc
eStrategy).supportType();

        // 调用测试方法
        LoadBalanceService loadBalanceService = new
LoadBalanceService(Arrays.asList(loadBalanceStrategy));
        Map<LoadBalanceStrategyType, LoadBalanceStrategy>
strategyMap =
            Whitebox.getInternalState(loadBalanceService,
"strategyMap");
        Assert.assertEquals("策略映射大小不一致", 1,
strategyMap.size());
        Assert.assertEquals("策略映射对象不一致",
loadBalanceStrategy,
strategyMap.get(LoadBalanceStrategyType.RANDOM));
```

```
        // 验证依赖方法
        Mockito.verify(loadBalanceStrategy).supportType();
    }

    /**
     * 测试：选择服务节点-正常
     */
    @Test
    public void testSelectNodeWithNormal() {
        // 模拟依赖方法
        LoadBalanceStrategy loadBalanceStrategy =
Mockito.mock(LoadBalanceStrategy.class);
        // 模拟依赖方法：loadBalanceStrategy.supportType

Mockito.doReturn(LoadBalanceStrategyType.RANDOM).when(loadBalanc
eStrategy).supportType();
        // 模拟依赖方法：loadBalanceStrategy.selectNode
        ServerNode serverNode = Mockito.mock(ServerNode.class);
        Mockito.doReturn(serverNode).when(loadBalanceStrategy)
            .selectNode(Mockito.anyList(),
Mockito.any(ClientRequest.class));

        // 调用测试方法
        List<ServerNode> serverNodeList =
CastUtils.cast(Mockito.mock(List.class));
        ClientRequest clientRequest =
Mockito.mock(ClientRequest.class);
        LoadBalanceService loadBalanceService = new
LoadBalanceService(Arrays.asList(loadBalanceStrategy));
        Assert.assertEquals("服务节点不一致", serverNode,

loadBalanceService.selectNode(LoadBalanceStrategyType.RANDOM,
serverNodeList, clientRequest));

        // 验证依赖方法
        // 验证依赖方法：loadBalanceStrategy.supportType
        Mockito.verify(loadBalanceStrategy).supportType();
        // 验证依赖方法：loadBalanceStrategy.selectNode

Mockito.verify(loadBalanceStrategy).selectNode(serverNodeList,
clientRequest);
    }
}
```

其实，不只是策略模式，很多模式下都不建议联合测试，而是推荐采用独立的单元测试。因为单元测试是白盒测试——一种专注于自身代码逻辑的测试。

五、 如何测试 Lambda 表达式

在有些单元测试中，Lambda 表达式并不一定被执行，所以导致 Lambda 表达式没有被测试。

1. 案例代码

这里，以从 ODPS 中查询用户交易订单为例说明。

1) 被测代码

交易订单查询服务，其中有一段转化订单的 Lambda 表达式。

```
/**
 * 交易订单服务
 */
@Service
public class TradeOrderService {
    /** 注入依赖对象 */
    /** 交易ODPS服务 */
    @Autowired
    private TradeOdpsService tradeOdpsService;

    /**
     * 查询交易订单
     *
     * @param userId 用户标识
     * @param maxCount 最大数量
     * @return 交易订单列表
     */
    public List<TradeOrderVO> queryTradeOrder(Long userId, Integer maxCount) {
        String format = ResourceHelper.getResourceAsString(getClass(),
"query_trade_order.sql");
        String sql = String.format(format, userId, maxCount);
        return tradeOdpsService.executeQuery(sql, record -> {
            TradeOrderVO tradeOrder = new TradeOrderVO();
            tradeOrder.setId(record.getBigint("id"));
            // ...
            return tradeOrder;
        });
    }
}
```

2) 依赖代码

封装了通用的 ODPS 查询方法。

```
/**
 * 交易 ODPS 服务类
 */
@Slf4j
@Service
public class TradeOdpsService {
    /** 注入依赖对象 */
    /** 交易 ODPS */
    @Resource(name = "tradeOdps")
    private Odps tradeOdps;

    /**
     * 执行查询
     *
     * @param <T> 模板类型
     * @param sql SQL 语句
     * @param dataParser 数据解析器
     * @return 查询结果列表
     */
    public <T> List<T> executeQuery(String sql, Function<Record,
T> dataParser) {
        try {
            // 打印提示信息
            log.info("开始执行 ODPS 数据查询...");

            // 执行 ODPS 查询
            Instance instance = SQLTask.run(tradeOdps, sql);
            instance.waitForSuccess();

            // 获取查询结果
            List<Record> recordList = SQLTask.getResult(instance);
            if (CollectionUtils.isEmpty(recordList)) {
                log.info("完成执行 ODPS 数据查询: totalSize=0");
                return Collections.emptyList();
            }

            // 依次读取数据
            List<T> dataList = new ArrayList<>();
            for (Record record : recordList) {
                T data = dataParser.apply(record);
```

```
                if (Objects.nonNull(data)) {
                    dataList.add(data);
                }
            }

            // 打印提示信息
            log.info("完成执行 ODPS 数据查询: totalSize={}",
dataList.size());

            // 返回查询结果
            return dataList;
        } catch (OdpsException e) {
            log.warn("执行 ODPS 数据查询异常: sql={}", sql, e);
            throw new BusinessException("执行 ODPS 数据查询异常", e);
        }
    }
}
```

2. 方法 1：直接测试法（不推荐）

按照通用的单元测试方法进行测试，发现 Lambda 表达式没有被测试到。

```
/**
 * 交易订单服务测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class TradeOrderServiceTest {
    /** 定义静态常量 */
    /** 资源路径 */
    private static final String RESOURCE_PATH =
"testTradeOrderService/";

    /** 模拟依赖对象 */
    /** 交易 ODPS 服务 */
    @Mock
    private TradeOdpsService tradeOdpsService;

    /** 定义测试对象 */
    /** 交易订单服务 */
    @InjectMocks
    private TradeOrderService tradeOrderService;

    /**
```

```
     * 测试：查询交易订单-正常
     */
    @Test
    public void testQueryTradeOrderWithNormal() {
        // 模拟依赖方法
        // 模拟依赖方法：tradeOdpsService.executeQuery
        List<TradeOrderVO> tradeOrderList =
CastUtils.cast(Mockito.mock(List.class));

Mockito.doReturn(tradeOrderList).when(tradeOdpsService).executeQ
uery(Mockito.anyString(), Mockito.any());

        // 调用测试方法
        Long userId = 12345L;
        Integer maxCount = 100;
        Assert.assertSame("交易订单列表不一致", tradeOrderList,
tradeOrderService.queryTradeOrder(userId, maxCount));

        // 验证依赖方法
        // 验证依赖方法：tradeOdpsService.executeQuery
        String path = RESOURCE_PATH +
"testQueryTradeOrderWithNormal/";
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"queryTradeOrder.sql");

Mockito.verify(tradeOdpsService).executeQuery(Mockito.eq(text),
Mockito.any());
    }
}
```

3. 方法 2：联合测试法（不推荐）

有人建议，可以把 TradeOrderService（交易订单服务）和 TradeOdpsService（交易 ODPS 服务）联合测试，这样就可以保证 Lambda 表达式被测试到。

```
/**
 * 交易订单服务测试类
 */
@RunWith(PowerMockRunner.class)
@PrepareForTest({SQLTask.class})
public class TradeOrderServiceTest {
    /** 定义静态常量 */
```

```
    /** 资源路径 */
    private static final String RESOURCE_PATH =
"testTradeOrderService/";

    /** 模拟依赖对象 */
    /** 交易 ODPS */
    @Mock(name = "tradeOdps")
    private Odps tradeOdps;

    /** 定义测试对象 */
    /** 交易 ODPS 服务 */
    @InjectMocks
    private TradeOdpsService tradeOdpsService =
Mockito.spy(TradeOdpsService.class);
    /** 交易订单服务 */
    @InjectMocks
    private TradeOrderService tradeOrderService;

    /**
     * 测试：查询交易订单-正常
     *
     * @throws OdpsException ODPS 异常
     */
    @Test
    public void testQueryTradeOrderWithNormal() throws
OdpsException {
        // 模拟依赖方法
        PowerMockito.mockStatic(SQLTask.class);
        // 模拟依赖方法：SQLTask.run
        Instance instance = Mockito.mock(Instance.class);
        PowerMockito.when(SQLTask.run(Mockito.eq(tradeOdps),
Mockito.anyString())).thenReturn(instance);
        // 模拟依赖方法：SQLTask.getResult
        Record record1 = PowerMockito.mock(Record.class);
        Record record2 = PowerMockito.mock(Record.class);
        List<Record> recordList = Arrays.asList(record1,
record2);

PowerMockito.when(SQLTask.getResult(instance)).thenReturn(record
List);
        // 模拟依赖方法：record.getString
        Mockito.doReturn(1L).when(record1).getBigint("id");
        Mockito.doReturn(2L).when(record2).getBigint("id");

        // 调用测试方法
```

```
        Long userId = 12345L;
        Integer maxCount = 100;
        List<TradeOrderVO> tradeOrderList =
tradeOrderService.queryTradeOrder(userId, maxCount);
        String path = RESOURCE_PATH +
"testQueryTradeOrderWithNormal/";
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"tradeOrderList.json");
        Assert.assertEquals("交易订单列表不一致", text,
JSON.toJSONString(tradeOrderList));

        // 验证依赖方法
        PowerMockito.verifyStatic(SQLTask.class);
        // 验证依赖方法：SQLTask.run
        text = ResourceHelper.getResourceAsString(getClass(),
path + "queryTradeOrder.sql");
        SQLTask.run(tradeOdps, text);
        // 验证依赖方法：SQLTask.getResult
        SQLTask.getResult(instance);
        // 验证依赖方法：instance.waitForSuccess
        Mockito.verify(instance).waitForSuccess();
        // 验证依赖方法：record.getString
        Mockito.verify(record1).getBigint("id");
        Mockito.verify(record2).getBigint("id");
    }
}
```

主要问题：需要了解 TradeOdpsService.executeQuery（执行查询）方法的逻辑并构建单元测试用例，导致 TradeOrderService.queryTradeOrder（查询交易订单）方法的单测测试用例非常复杂。

4. 方法 3：重构测试法（推荐）

其实，只需要把这段 Lambda 表达式提取成一个 convertTradeOrder（转化交易订单）方法，即可让代码变得清晰明了，又可以让代码更容易单元测试。

1） 重构代码

提取 Lambda 表达式为 convertTradeOrder（转化交易订单）方法。

```
@Slf4j
/**
 * 交易订单服务类
 */
@Service
public class TradeOrderService {
    /** 注入依赖对象 */
    /** 交易 ODPS 服务 */
    @Autowired
    private TradeOdpsService tradeOdpsService;

    /**
     * 查询交易订单
     *
     * @param userId 用户标识
     * @param maxCount 最大数量
     * @return 交易订单列表
     */
    public List<TradeOrderVO> queryTradeOrder(Long userId,
Integer maxCount) {
        String format =
ResourceHelper.getResourceAsString(getClass(),
"query_trade_order.sql");
        String sql = String.format(format, userId, maxCount);
        return tradeOdpsService.executeQuery(sql,
TradeOrderService2::convertTradeOrder);
    }

    /**
     * 转化交易订单
     *
     * @param record ODPS 记录
     * @return 交易订单
     */
    private static TradeOrderVO convertTradeOrder(Record record)
{
        TradeOrderVO tradeOrder = new TradeOrderVO();
        tradeOrder.setId(record.getBigint("id"));
        // ...
        return tradeOrder;
    }
}
```

2) 测试用例

针对 queryTradeOrder（查询交易订单）方法和 convertTradeOrder（转化交易订单）方法分别进行单元测试。

```
/**
 * 交易订单服务测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class TradeOrderServiceTest {
    /** 定义静态常量 */
    /** 资源路径 */
    private static final String RESOURCE_PATH =
"testTradeOrderService/";

    /** 模拟依赖对象 */
    /** 交易 ODPS 服务 */
    @Mock
    private TradeOdpsService tradeOdpsService;

    /** 定义测试对象 */
    /** 交易订单服务 */
    @InjectMocks
    private TradeOrderService tradeOrderService;

    /**
     * 测试：查询交易订单-正常
     */
    @Test
    public void testQueryTradeOrderWithNormal() {
        // 模拟依赖方法
        // 模拟依赖方法：tradeOdpsService.executeQuery
        List<TradeOrderVO> tradeOrderList =
CastUtils.cast(Mockito.mock(List.class));

Mockito.doReturn(tradeOrderList).when(tradeOdpsService).executeQ
uery(Mockito.anyString(), Mockito.any());

        // 调用测试方法
        Long userId = 12345L;
        Integer maxCount = 100;
        Assert.assertSame("交易订单列表不一致", tradeOrderList,
tradeOrderService.queryTradeOrder(userId, maxCount));
```

```
        // 验证依赖方法
        // 验证依赖方法：tradeOdpsService.executeQuery
        String path = RESOURCE_PATH +
"testQueryTradeOrderWithNormal/";
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"queryTradeOrder.sql");

Mockito.verify(tradeOdpsService).executeQuery(Mockito.eq(text),
Mockito.any());
    }

    /**
     * 测试：转化交易订单
     *
     * @throws Exception 异常信息
     */
    @Test
    public void testConvertTradeOrder() throws Exception {
        // 模拟依赖方法
        Long id = 12345L;
        Record record = Mockito.mock(Record.class);
        Mockito.doReturn(id).when(record).getBigint("id");

        // 调用测试方法
        TradeOrderVO tradeOrder =
Whitebox.invokeMethod(TradeOrderService2.class,
"convertTradeOrder", record);
        Assert.assertEquals("订单标识不一致", id,
tradeOrder.getId());

        // 验证依赖方法
        Mockito.verify(record).getBigint("id");
    }
}
```

六、 如何测试链式调用

在日常编码过程中，很多人都喜欢使用链式调用，这样可以让代码变得更简洁。

1. 案例代码

这里，通过修改后的添加跨域支持代码来举例说明（原方法没有返回值）。

```
/**
 * 跨域辅助类
 */
public class CorsHelper {
    /** 定义静态常量 */
    /** 最大生命周期 */
    private static final long MAX_AGE = 3600L;

    /**
     * 添加跨域支持
     *
     * @param registry 跨域注册器
     * @return 跨域注册
     */
    public static CorsRegistration addCorsMapping(CorsRegistry registry) {
        return registry.addMapping("/**")
            .allowedOrigins("*")
            .allowedMethods("GET", "HEAD", "POST", "PUT", "DELETE", "OPTIONS")
            .allowCredentials(true)
            .maxAge(MAX_AGE)
            .allowedHeaders("*");
    }
}
```

2. 方法 1：普通测试法（不推荐）

正常情况下，每一个依赖对象及其调用方法都要 mock，编写的代码如下：

```
/**
 * 跨域辅助测试类
 */
public class CorsHelperTest {

    /**
     * 测试：添加跨域支持
     */
    @Test
    public void testAddCorsMapping() {
        // 模拟依赖方法
        CorsRegistry registry = Mockito.mock(CorsRegistry.class);
        CorsRegistration registration =
Mockito.mock(CorsRegistration.class);
```

```
Mockito.doReturn(registration).when(registry).addMapping(Mockito
.anyString());

Mockito.doReturn(registration).when(registration).allowedOrigins
(Mockito.any());

Mockito.doReturn(registration).when(registration).allowedMethods
(Mockito.any());

Mockito.doReturn(registration).when(registration).allowCredentia
ls(Mockito.anyBoolean());

Mockito.doReturn(registration).when(registration).maxAge(Mockito
.anyLong());

Mockito.doReturn(registration).when(registration).allowedHeaders
(Mockito.any());

        // 调用测试方法
        Assert.assertEquals("跨域注册不一致", registration,
CorsHelper.addCorsMapping(registry));

        // 验证依赖方法
        Mockito.verify(registry).addMapping("/**");
        Mockito.verify(registration).allowedOrigins("*");
        Mockito.verify(registration).allowedMethods("GET",
"HEAD", "POST", "PUT", "DELETE", "OPTIONS");
        Mockito.verify(registration).allowCredentials(true);
        Mockito.verify(registration).maxAge(3600L);
        Mockito.verify(registration).allowedHeaders("*");
    }
}
```

3. 方法 2：利用 RETURNS_DEEP_STUBS 参数法（推荐）

对于链式调用，Mockito 提供了更加简便的单元测试方法——提供 Mockito.RETURNS_DEEP_STUBS 参数，实现链式调用返回对象的自动 mock。利用 Mockito.RETURNS_DEEP_STUBS 参数编写的测试用例如下：

```
/**
 * 跨域辅助测试类
 */
```

```
public class CorsHelperTest {
    /**
     * 测试：添加跨域支持
     */
    @Test
    public void testAddCorsMapping() {
        // 模拟依赖方法
        CorsRegistry registry = Mockito.mock(CorsRegistry.class,
Answers.RETURNS_DEEP_STUBS);
        CorsRegistration registration =
Mockito.mock(CorsRegistration.class);
        Mockito.when(registry.addMapping(Mockito.anyString())
            .allowedOrigins(Mockito.any())
            .allowedMethods(Mockito.any())
            .allowCredentials(Mockito.anyBoolean())
            .maxAge(Mockito.anyLong())
            .allowedHeaders(Mockito.any()))
            .thenReturn(registration);

        // 调用测试方法
        Assert.assertEquals("跨域注册不一致", registration,
CorsHelper.addCorsMapping(registry));

        // 验证依赖方法
        Mockito.verify(registry.addMapping("/**")
            .allowedOrigins("*")
            .allowedMethods("GET", "HEAD", "POST", "PUT",
"DELETE", "OPTIONS")
            .allowCredentials(true)
            .maxAge(3600L))
            .allowedHeaders("*");
    }
}
```

代码说明：

- 在 mock 对象时，需要指定 Mockito.RETURNS_DEEP_STUBS 参数；
- 在 mock 方法时，采用 when-then 模式，when 内容是链式调用，then 内容是返回的值；
- 在 verify 方法时，只需要验证最后 1 次方法调用，verify 内容是前 n 次链式调用；如果验证时某个方法调用的某个参数指定错误时，最后一个方法调用验证将因为这个 mock 对象没有方法调用而抛出异常。

4. 方法 3：利用 RETURNS_SELF 参数法（推荐）

对于相同返回值的链式调用，Mockito 提供了更加简便的单元测试方法——提供 Mockito.RETURNS_SELF 参数，实现链式调用返回对象的自动 mock，而且还能返回同一 mock 对象。利用 Mockito.RETURNS_SELF 参数编写的测试用例如下：

```
/**
 * 跨域辅助测试类
 */
public class CorsHelperTest {
    /**
     * 测试：添加跨域支持
     */
    @Test
    public void testAddCorsMapping() {
        // 模拟依赖方法
        CorsRegistry registry = Mockito.mock(CorsRegistry.class);
        CorsRegistration registration = Mockito.mock(CorsRegistration.class,
Answers.RETURNS_SELF);
        Mockito.doReturn(registration).when(registry).addMapping(Mockito.anyString());

        // 调用测试方法
        Assert.assertEquals("跨域注册不一致", registration,
CorsHelper.addCorsMapping(registry));

        // 验证依赖方法
        Mockito.verify(registry).addMapping("/**");
        Mockito.verify(registration).allowedOrigins("*");
        Mockito.verify(registration).allowedMethods("GET", "HEAD", "POST", "PUT", "DELETE",
"OPTIONS");
        Mockito.verify(registration).allowCredentials(true);
        Mockito.verify(registration).maxAge(3600L);
        Mockito.verify(registration).allowedHeaders("*");
    }
}
```

代码说明：

- 在 mock 对象时，对于自返回对象，需要指定 Mockito.RETURNS_SELF 参数；
- 在 mock 方法时，无需对自返回对象进行 mock 方法，因为框架已经 mock 方法返回了自身；
- 在 verify 方法时，可以像普通测试法一样优美地验证所有方法调用。

方法对比：

- 普通测试法：mock 调用方法语句较多；

- 利用 RETURNS_DEEP_STUBS 参数法：mock 调用方法语句较少，适合于链式调用返回不同值；
- 利用 RETURNS_SELF 参数法：mock 调用方法语句最少，适合于链式调用返回相同值。

七、 如何测试相同参数返回不同值

在有些场景下，存在相同参数多次调用返回不同值的情况，比如：读取文本文件的 readLine 方法。

1. 案例代码

这里，以 ODPS 的 RecordReader 为例，读取每一行数据记录。

```
/**
 * 读取数据
 *
 * @param <T> 模板类型
 * @param recordReader 记录读取器
 * @param dataParser 数据解析器
 * @return 数据列表
 * @throws IOException IO异常
 */
public static <T> List<T> readData(RecordReader recordReader, Function<Record, T>
dataParser) throws IOException {
    Record record;
    List<T> dataList = new ArrayList<>();
    while (Objects.nonNull(record = recordReader.read())) {
        T data = dataParser.apply(record);
        if (Objects.nonNull(data)) {
            dataList.add(data);
        }
    }
    return dataList;
}
```

2. 测试用例

为了 mock 相同参数返回不同值，需要使用到 Mockito.doReturn 的可变数组功能。

```
/**
 * 测试：读取数据-正常
 *
 * @throws IOException IO异常
 */
@Test
public void testReadDataWithNormal() throws IOException {
    // 模拟依赖方法
    // 模拟依赖方法：recordReader.read
    Record record1 = Mockito.mock(Record.class);
    Record record2 = Mockito.mock(Record.class);
    TunnelRecordReader recordReader = Mockito.mock(TunnelRecordReader.class);
    Mockito.doReturn(record1, record2, null).when(recordReader).read();
    // 模拟依赖方法：dataParser.apply
    Function<Record, Object> dataParser = CastUtils.cast(Mockito.mock(Function.class));
    Object object1 = new Object();
    Object object2 = new Object();
    Mockito.doReturn(object1).when(dataParser).apply(record1);
    Mockito.doReturn(object2).when(dataParser).apply(record2);

    // 调用测试方法
    List<Object> dataList = OdpsHelper.readData(recordReader, dataParser);
    Assert.assertEquals("数据列表不一致", Arrays.asList(object1, object2), dataList);

    // 验证依赖方法
    // 验证依赖方法：recordReader.read
    Mockito.verify(recordReader, Mockito.times(3)).read();
    // 验证依赖方法：dataParser.apply
    Mockito.verify(dataParser).apply(record1);
    Mockito.verify(dataParser).apply(record2);
}
```

八、 如何测试已变更的方法参数值

在单元测试中，我们通常通过 ArgumentCaptor 进行方法参数捕获并验证。但是，在有些情况下，我们捕获的可能是已经变更的方法参数，所以无法对这些方法参数值进行验证。

1. 案例代码

这里，以分批读取并保存 ODPS 数据为例说明。其中，dataList 在每次存储后，都进行了一次清除操作。

```
/**
 * 读取数据
 *
 * @param <T> 模板类型
 * @param recordReader 记录读取器
 * @param batchSize 批量大小
```

```
 * @param dataParser 数据解析器
 * @param dataStorage 数据存储器
 * @throws IOException IO 异常
 */
public static <T> void readData(RecordReader recordReader, int
batchSize,
    Function<Record, T> dataParser, Consumer<List<T>>
dataStorage) throws IOException {
    // 依次读取数据
    Record record;
    List<T> dataList = new ArrayList<>(batchSize);
    while (Objects.nonNull(record = recordReader.read())) {
        // 解析添加数据
        T data = dataParser.apply(record);
        if (Objects.nonNull(data)) {
            dataList.add(data);
        }

        // 批量存储数据
        if (dataList.size() == batchSize) {
            dataStorage.accept(dataList);
            dataList.clear();
        }
    }

    // 存储剩余数据
    if (CollectionUtils.isNotEmpty(dataList)) {
        dataStorage.accept(dataList);
        dataList.clear();
    }
}
```

2. 问题测试

通常情况下，我们利用 ArgumentCaptor 编写的测试用例如下：

```
/**
 * 测试：读取数据-正常
 *
 * @throws IOException IO 异常
 */
@Test
public void testReadDataWithNormal() throws IOException {
```

```
    // 模拟依赖方法
    // 模拟依赖方法：recordReader.read
    Record record1 = Mockito.mock(Record.class);
    Record record2 = Mockito.mock(Record.class);
    TunnelRecordReader recordReader =
Mockito.mock(TunnelRecordReader.class);
    Mockito.doReturn(record1, record2,
null).when(recordReader).read();
    // 模拟依赖方法：dataParser.apply
    Function<Record, Object> dataParser =
CastUtils.cast(Mockito.mock(Function.class));
    Object object1 = new Object();
    Object object2 = new Object();
    Mockito.doReturn(object1).when(dataParser).apply(record1);
    Mockito.doReturn(object2).when(dataParser).apply(record2);

    // 调用测试方法
    int batchSize = 2;
    Consumer<List<Object>> dataStorage =
CastUtils.cast(Mockito.mock(Consumer.class));
    OdpsHelper.readData(recordReader, batchSize, dataParser,
dataStorage);

    // 验证依赖方法
    // 验证依赖方法：recordReader.read
    Mockito.verify(recordReader, Mockito.times(3)).read();
    // 验证依赖方法：dataParser.apply
    Mockito.verify(dataParser).apply(record1);
    Mockito.verify(dataParser).apply(record2);
    // 验证依赖方法：dataStorage.test
    ArgumentCaptor<List<Object>> dataListCaptor =
CastUtils.cast(ArgumentCaptor.forClass(List.class));
    Mockito.verify(dataStorage).accept(dataListCaptor.capture());
    Assert.assertEquals("数据列表不一致", Arrays.asList(object1,
object2), dataListCaptor.getValue());
}
```

执行该单元测试后，会出现以下错误：

```
java.lang.AssertionError: 数据列表不一致 expected:<[java.lang.Object@7eaa2bc6,
java.lang.Object@6dae70f9]> but was:<[]>
```

因为，我们捕获的方法参数 dataList 只是一个对象引用，其数据内容早已被 clear 方法清除干净了。

3. 正确测试

对于这种情况，我们可以利用 Mockito.doAnswer 来保存这些临时值，最后再进行统一的数据验证。

```
/**
 * 测试：读取数据-正常
 *
 * @throws IOException IO 异常
 */
@Test
public void testReadDataWithNormal() throws IOException {
    // 模拟依赖方法
    // 模拟依赖方法：recordReader.read
    Record record1 = Mockito.mock(Record.class);
    Record record2 = Mockito.mock(Record.class);
    TunnelRecordReader recordReader =
Mockito.mock(TunnelRecordReader.class);
    Mockito.doReturn(record1, record2,
null).when(recordReader).read();
    // 模拟依赖方法：dataParser.apply
    Function<Record, Object> dataParser =
CastUtils.cast(Mockito.mock(Function.class));
    Object object1 = new Object();
    Object object2 = new Object();
    Mockito.doReturn(object1).when(dataParser).apply(record1);
    Mockito.doReturn(object2).when(dataParser).apply(record2);
    // 模拟依赖方法：dataStorage.test
    List<Object> dataList = new ArrayList<>();
    Consumer<List<Object>> dataStorage =
CastUtils.cast(Mockito.mock(Consumer.class));
    Mockito.doAnswer(invocation ->
dataList.addAll(invocation.getArgument(0)))
        .when(dataStorage).accept(Mockito.anyList());

    // 调用测试方法
    int batchSize = 2;
    OdpsHelper.readData(recordReader, batchSize, dataParser,
dataStorage);
```

```
    Assert.assertEquals("数据列表不一致", Arrays.asList(object1,
object2), dataList);

    // 验证依赖方法
    // 验证依赖方法: recordReader.read
    Mockito.verify(recordReader, Mockito.times(3)).read();
    // 验证依赖方法: dataParser.apply
    Mockito.verify(dataParser).apply(record1);
    Mockito.verify(dataParser).apply(record2);
    // 验证依赖方法: dataStorage.test
    Mockito.verify(dataStorage).accept(Mockito.anyList());
}
```

九、 如何测试相同返回值的代码分支

在业务代码中，经常会出现不同的代码分支返回相同值的情况。这个时候，仅通过验证返回值是没法判断是否命中了对应的代码分支的。那么，这种情况如何进行单元测试呢？

1. 案例代码

这里，以灰度发布服务判定方法为例说明。

```
/**
 * 灰度发布服务类
 */
@Slf4j
@Service
public class GrayReleaseService {
    /** 定义静态常量 */
    /** 灰度发布分子 */
    private static final long GRAY_NUMERATOR = 0L;
    /** 灰度发布分母 */
    private static final long GRAY_DENOMINATOR = 10000L;

    /** 注入依赖对象 */
    /** 灰度发布配置 */
    @Autowired
    private GrayReleaseConfig grayReleaseConfig;

    /**
     * 是否灰度发布
```

```
     *
     * @param key 主键
     * @param channel 渠道
     * @param userId 用户标识
     * @param value 取值
     * @return 判断结果
     */
    public boolean isGrayRelease(String key, String channel,
String userId, Object value) {
        // 判断灰度发布取值
        if (Objects.isNull(value)) {
            log.info("命中灰度取值为空");
            return false;
        }

        // 获取灰度发布映射
        Map<String, GrayReleaseItem> grayReleaseMap =
grayReleaseConfig.getGrayReleaseMap();
        if (MapUtils.isEmpty(grayReleaseMap)) {
            log.info("命中灰度发布映射为空");
            return false;
        }

        // 获取灰度发布项
        GrayReleaseItem grayReleaseItem =
grayReleaseMap.get(key);
        if (Objects.isNull(grayReleaseItem)) {
            log.info("命中灰度发布映项为空: key={}", key);
            return false;
        }

        // 判断渠道白名单
        Set<String> channelWhiteSet =
grayReleaseItem.getChannelWhiteSet();
        if (CollectionUtils.isNotEmpty(channelWhiteSet) &&
channelWhiteSet.contains(channel)) {
            log.info("命中渠道白名单灰度: key={}, channel={}", key,
channel);
            return true;
        }

        // 判断用户白名单
        Set<String> userIdWhiteSet =
grayReleaseItem.getUserIdWhiteSet();
```

```
        if (CollectionUtils.isNotEmpty(userIdWhiteSet) &&
userIdWhiteSet.contains(userId)) {
            log.info("命中用户白名单灰度: key={}, userId={}", key,
userId);
            return true;
        }

        // 判断灰度发布比例
        long grayNumerator =
Optional.ofNullable(grayReleaseItem.getGrayNumerator()).orElse(G
RAY_NUMERATOR);
        long grayDenominator =
Optional.ofNullable(grayReleaseItem.getGrayDenominator()).orElse
(GRAY_DENOMINATOR);
        boolean isGray = Math.abs(Objects.hashCode(value)) %
grayDenominator <= grayNumerator;
        log.info("命中灰度发布比例: key={}, value={}, isGray={}",
key, value, isGray);
        return isGray;
    }
}
```

2. 普通测试法（不推荐）

这里，只测试了命中渠道白名单的情况。

```
/**
 * 灰度发布服务测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class GrayReleaseServiceTest {
    /** 模拟依赖方法 */
    /** 灰度发布配置 */
    @Mock
    private GrayReleaseConfig grayReleaseConfig;

    /** 定义测试对象 */
    /** 灰度发布服务 */
    @InjectMocks
    private GrayReleaseService grayReleaseService;

    /**
     * 测试：是否灰度发布-命中渠道白名单
     */
```

```
    @Test
    public void testIsGrayReleaseWithChannelWhiteSet() {
        // 模拟依赖方法
        GrayReleaseItem grayReleaseItem = new GrayReleaseItem();

grayReleaseItem.setChannelWhiteSet(Sets.newHashSet("alipay"));

grayReleaseItem.setUserIdWhiteSet(Sets.newHashSet("123456"));
        Map<String, GrayReleaseItem> grayReleaseMap =
Maps.newHashMap();
        grayReleaseMap.put("test", grayReleaseItem);

Mockito.doReturn(grayReleaseMap).when(grayReleaseConfig).getGray
ReleaseMap();

        // 调用测试方法
        String key = "test";
        String channel = "alipay";
        String userId = "123456";
        Object value = 1234567890L;
        Assert.assertTrue("判断结果不为真",
grayReleaseService.isGrayRelease(key, channel, userId, value));

        // 验证依赖方法
        Mockito.verify(grayReleaseConfig).getGrayReleaseMap();
    }
}
```

在一次代码重构中，把“判断用户白名单”放在“判断渠道白名单”之前，这个单元测试是无法检测出来的。

3. 验证测试法（推荐）

通过对 mock 方法的验证，可以相对准确地确定命中的代码分支。

```
/**
 * 灰度发布服务测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class GrayReleaseServiceTest {
    /** 模拟依赖方法 */
    /** 灰度发布配置 */
    @Mock
```

```
    private GrayReleaseConfig grayReleaseConfig;

    /** 定义测试对象 */
    /** 灰度发布服务 */
    @InjectMocks
    private GrayReleaseService grayReleaseService;

    /**
     * 测试：是否灰度发布-命中渠道白名单
     */
    @Test
    public void testIsGrayReleaseWithChannelWhiteSet() {
        // 模拟依赖方法
        // 模拟依赖方法：grayReleaseItem.getChannelWhiteSet
        GrayReleaseItem grayReleaseItem =
Mockito.mock(GrayReleaseItem.class);

Mockito.doReturn(Sets.newHashSet("alipay")).when(grayReleaseItem
).getChannelWhiteSet();
        // 模拟依赖方法：grayReleaseItem.getUserIdWhiteSet

Mockito.doReturn(Sets.newHashSet("123456")).when(grayReleaseItem
).getUserIdWhiteSet();
        // 模拟依赖方法：grayReleaseConfig.getGrayReleaseMap
        Map<String, GrayReleaseItem> grayReleaseMap =
Maps.newHashMap();
        grayReleaseMap.put("test", grayReleaseItem);

Mockito.doReturn(grayReleaseMap).when(grayReleaseConfig).getGray
ReleaseMap();

        // 调用测试方法
        String key = "test";
        String channel = "alipay";
        String userId = "123456";
        Object value = 1234567890L;
        Assert.assertTrue("判断结果不为真",
grayReleaseService.isGrayRelease(key, channel, userId, value));

        // 验证依赖方法
        // 验证依赖方法：grayReleaseConfig.getGrayReleaseMap
        Mockito.verify(grayReleaseConfig).getGrayReleaseMap();
        // 验证依赖方法：grayReleaseItem.getChannelWhiteSet
        Mockito.verify(grayReleaseItem).getChannelWhiteSet();
```

```
        // 验证依赖对象
        Mockito.verifyNoMoreInteractions(grayReleaseConfig,
grayReleaseItem);
    }
}
```

如果把“判断用户白名单”放在“判断渠道白名单”之前，这个单元测试会报出以下错误日志：

```
Wanted but not invoked:
grayReleaseItem.getChannelWhiteSet();
```

错误日志告诉我们，grayReleaseItem.getChannelWhiteSet 方法并没有被调用，所以不可能命中渠道白名单代码分支。

4. 日志测试法（推荐）

对于有日志打印的代码，可以通过验证日志方法来确定命中的代码分支，而且这种验证方法是非常简单直白的。如果没有日志打印，我们也可以添加日志打印（可能会涉及日志存储成本的增加）。

```
/**
 * 灰度发布服务测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class GrayReleaseServiceTest {
    /** 模拟依赖方法 */
    /** 日志器 */
    @Mock
    private Logger log;
    /** 灰度发布配置 */
    @Mock
    private GrayReleaseConfig grayReleaseConfig;

    /** 定义测试对象 */
    /** 灰度发布服务 */
    @InjectMocks
    private GrayReleaseService grayReleaseService;

    /**
     * 在测试前
```

```
     */
    @Before
    public void beforeTest() {

FieldHelper.writeStaticFinalField(GrayReleaseService.class,
"log", log);
    }

    /**
     * 测试：是否灰度发布-命中渠道白名单
     */
    @Test
    public void testIsGrayReleaseWithChannelWhiteSet() {
        // 模拟依赖方法
        GrayReleaseItem grayReleaseItem = new GrayReleaseItem();

grayReleaseItem.setChannelWhiteSet(Sets.newHashSet("alipay"));

grayReleaseItem.setUserIdWhiteSet(Sets.newHashSet("123456"));
        Map<String, GrayReleaseItem> grayReleaseMap =
Maps.newHashMap();
        grayReleaseMap.put("test", grayReleaseItem);

Mockito.doReturn(grayReleaseMap).when(grayReleaseConfig).getGray
ReleaseMap();

        // 调用测试方法
        String key = "test";
        String channel = "alipay";
        String userId = "123456";
        Object value = 1234567890L;
        Assert.assertTrue("判断结果不为真",
grayReleaseService.isGrayRelease(key, channel, userId, value));

        // 验证依赖方法
        // 验证依赖方法：grayReleaseConfig.getGrayReleaseMap
        Mockito.verify(grayReleaseConfig).getGrayReleaseMap();
        // 验证依赖方法：log.info
        Mockito.verify(log).info("命中渠道白名单灰度：key={},
channel={}", key, channel);

        // 验证依赖对象
        Mockito.verifyNoInteractions(log, grayReleaseConfig);
    }
}
```

如果把“判断用户白名单”放在“判断渠道白名单”之前，这个单元测试会报出以下错误日志：

```
Argument(s) are different! Wanted:
log.info(
    "命中渠道白名单灰度：key={}, channel={}",
    "test",
    "alipay"
);
-> at ...
Actual invocations have different arguments:
log.info(
    "命中用户白名单灰度：key={}, userId={}",
    "test",
    "123456"
);
```

错误日志告诉我们，我们期望命中渠道白名单灰度代码分支，实际却命中的是用户白名单灰度代码分支。

十、 如何测试多线程并发编程

Java 多线程并发编程，就是通过多个线程同时执行多个任务来缩短执行时间、提高执行效率的方法。在 JDK1.8 中，新增了 CompletableFuture 类，实现了对任务编排的能力——可以轻松地组织不同任务的运行顺序、规则及方式。

1. 案例代码

这里，以并行获取批量交易订单为例说明。

```
/**
 * 交易订单服务类
 */
@Slf4j
@Service
public class TradeOrderService {
    /** 定义静态常量 */
    /** 等待时间(毫秒) */
    private static final long WAIT_TIME = 1000L;

    /** 注入依赖对象 */
    /** 交易订单 DAO */
```

```
    @Autowired
    private TradeOrderDAO tradeOrderDAO;
    /** 执行器服务 */
    @Autowired
    private ExecutorService executorService;

    /**
     * 获取交易订单列表
     *
     * @param orderIdList 订单标识列表
     * @return 交易订单列表
     */
    public List<TradeOrderVO> getTradeOrders(List<Long>
orderIdList) {
        // 检查订单标识列表
        if (CollectionUtils.isEmpty(orderIdList)) {
            return Collections.emptyList();
        }

        // 获取交易订单期望
        List<CompletableFuture<TradeOrderVO>> futureList =
orderIdList.stream()
            .map(this::getTradeOrder).collect(Collectors.toList())
;

        // 聚合交易订单期望
        CompletableFuture<List<TradeOrderVO>> joinFuture =
            CompletableFuture.allOf(futureList.toArray(new
CompletableFuture[0]))
                .thenApply(v ->
futureList.stream().map(CompletableFuture::join).collect(Collect
ors.toList()));

        // 返回交易订单列表
        try {
            return joinFuture.get(WAIT_TIME,
TimeUnit.MILLISECONDS);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            log.warn("获取订单中断异常", e);
            throw new BusinessException("获取订单中断异常", e);
        } catch (ExecutionException | TimeoutException |
RuntimeException e) {
            log.warn("获取订单其它异常", e);
            throw new BusinessException("获取订单其它异常", e);
```

```
        }
    }

    /**
     * 获取交易订单
     *
     * @param orderId 订单标识
     * @return 交易订单期望
     */
    private CompletableFuture<TradeOrderVO> getTradeOrder(Long
orderId) {
        return CompletableFuture.supplyAsync(() ->
tradeOrderDAO.get(orderId), executorService)
            .thenApply(TradeOrderService::convertTradeOrder);
    }

    /**
     * 转化交易订单
     *
     * @param tradeOrder 交易订单 DO
     * @return 交易订单 VO
     */
    private static TradeOrderVO convertTradeOrder(TradeOrderDO
tradeOrder) {
        TradeOrderVO tradeOrderVO = new TradeOrderVO();
        tradeOrderVO.setId(tradeOrder.getId());
        // ...
        return tradeOrderVO;
    }
}
```

2. 测试用例

对于多线程并发编程，如果采集 mock 静态方法的方式进行单元测试，将会使单元测试用例变得非常复杂。通过实践总结，采用注入线程池的方式，将会使单元测试用例变得非常简单。

```
/**
 * 交易订单服务测试类
 */
@RunWith(MockitoJUnitRunner.class)
public class TradeOrderServiceTest {
```

```
    /** 定义静态常量 */
    /** 资源路径 */
    private static final String RESOURCE_PATH =
"testTradeOrderService/";

    /** 模拟依赖对象 */
    /** 交易订单 DAO */
    @Mock
    private TradeOrderDAO tradeOrderDAO;
    /** 执行器服务 */
    @Spy
    private ExecutorService executorService =
Executors.newFixedThreadPool(10);

    /** 定义测试对象 */
    /** 交易订单服务 */
    @InjectMocks
    private TradeOrderService tradeOrderService;

    /**
     * 测试：获取交易订单列表-正常
     */
    @Test
    public void testGetTradeOrdersWithNormal() {
        // 模拟依赖方法
        // 模拟依赖方法：tradeOrderDAO.get
        String path = RESOURCE_PATH +
"testGetTradeOrdersWithNormal/";
        String text =
ResourceHelper.getResourceAsString(getClass(), path +
"tradeOrderMap.json");
        Map<Long, TradeOrderDO> tradeOrderMap =
JSON.parseObject(text, new TypeReference<Map<Long,
TradeOrderDO>>() {});
        Mockito.doAnswer(invocation ->
tradeOrderMap.get(invocation.getArgument(0)))
            .when(tradeOrderDAO).get(Mockito.anyLong());

        // 调用测试方法
        text = ResourceHelper.getResourceAsString(getClass(),
path + "orderIdList.json");
        List<Long> orderIdList = JSON.parseArray(text,
Long.class);
        List<TradeOrderVO> tradeOrderList =
tradeOrderService.getTradeOrders(orderIdList);
```

```
        text = ResourceHelper.getResourceAsString(getClass(),
path + "tradeOrderList.json");
        Assert.assertEquals("交易订单列表不一致", text,
JSON.toJSONString(tradeOrderList));

        // 验证依赖方法
        // 验证依赖方法：tradeOrderDAO.get
        ArgumentCaptor<Long> orderIdCaptor =
ArgumentCaptor.forClass(Long.class);
        Mockito.verify(tradeOrderDAO,
Mockito.atLeastOnce()).get(orderIdCaptor.capture());
        Assert.assertEquals("订单标识列表不一致", orderIdList,
orderIdCaptor.getAllValues());
    }
}
```

十一、 附录

1. 引入 Maven 单测包

```
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.1</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>3.3.3</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-module-junit4</artifactId>
    <version>2.0.9</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-api-mockito2</artifactId>
    <version>2.0.9</version>
    <scope>test</scope>
</dependency>
```

2. 使用到的工具方法

1) 以字符串方式获取资源

ResourceHelper.getResourceAsString（以字符串方式获取资源）通过 Apache 的 IOUtils.toString 方法实现，提供以字符串方式获取资源的功能。

```
/**
 * 资源辅助类
 */
public final class ResourceHelper {
    /**
     * 以字符串方式获取资源
     *
     * @param clazz 类
     * @param name 资源名称
     * @return 字符串
     */
    public static <T> String getResourceAsString(Class<T> clazz, String name) {
        try (InputStream is = clazz.getResourceAsStream(name)) {
            return IOUtils.toString(is, StandardCharsets.UTF_8);
        } catch (IOException e) {
            throw new IllegalArgumentException(String.format("以字符串方式获取资源(%s)异常",
name), e);
        }
    }
}
```

2) 写入静态常量字段

FieldHelper.writeStaticFinalField（写入静态常量字段）通过 Apache 的 FieldUtils 相关方法实现，提供写入静态常量字段的功能。

```
/**
 * 字段辅助类
 */
public final class FieldHelper {
    /**
     * 写入静态常量字段
     *
     * @param clazz 类
     * @param fieldName 字段名称
     * @param fieldValue 字段取值
     */
    public static void writeStaticFinalField(Class<?> clazz, String fieldName, Object
fieldValue) {
        try {
            Field field = clazz.getDeclaredField(fieldName);
            FieldUtils.removeFinalModifier(field);
            FieldUtils.writeStaticField(field, fieldValue, true);
        } catch (NoSuchFieldException | IllegalAccessException e) {
            throw new UnsupportedOperationException("写入静态常量字段异常", e);
        }
    }
}
```

后记

其实在很久之前，有人就希望我整理一个单元测试案例库。我迟迟没有行动，主要原因如下：

- 单元测试案例是无穷无尽的，如何系统化地呈现给读者是个大工程；
- 单元测试案例必须典型、合理、有意义，如何构建这些案例也很消耗精力。

现在，终于鼓起勇气整理这篇《Java 单元测试典型案例集锦》，主要是因为单元测试案例是单元测试重要的一环，也是为了给我的 Java 单元测试系列文章划上一个完美的句号。

最后，根据本文主题吟诗一首：

《单测案例》

单元测试百家说，
案例总结方法多。
芳草满园花满目，
绿肥红瘦自斟酌。

单测认证：单元测试认证问题、答案和解析

1. 找出单测中存在的问题

题目：号称“单测小王子”的小明，写起单测用例毫不费力，且新增代码的单测覆盖率达到 90%+。在一次代码评审中，小明的单测用例却被评审者挑战了，你认为会被评审者挑战的单测问题有哪些？

```
@Test
public void testQueryUserWithData() {
    // 模拟依赖方法
    // 模拟依赖方法：userDAO.countByCompany
    Long companyId = 123L;
    Mockito.doReturn(91L).when(userDAO).countByCompany(companyId);
    // 模拟依赖方法：userDAO.queryByCompany
    Long startIndex = 90L;
    Integer pageSize = 10;
    List<UserVO> dataList = buildUserList(); // 使用new初始化对象，几十上百行代码
    Mockito.doReturn(dataList).when(userDAO).queryByCompany(companyId, startIndex,
pageSize);

    // 调用测试方法
    PageDataVO<UserVO> pageData = userService.queryUser(companyId, startIndex, pageSize);
    Assert.assertEquals("数据列表不一致", dataList, pageData.getDataList());
}
```

选项：

A. 无需用代码构建真实 dataList 对象，直接 mock 或 new 一个虚拟 dataList 对象
B. 没有验证返回值 pageData 的全部属性
C. 没有验证 mock 的方法调用
D. 同一对象实例，建议使用 Assert.assertSame 代替 Assert.assertEquals

答案：ABCD

2. 学习单测偷懒大法

题目：在业务需求交付的压力下，小明总结了一套快速提高单测覆盖率的“方法论”。在只保证单测覆盖率、不保证单测有效性的前提下，以下哪些方法是有效的？

选项：

A. 不验证数据对象（属性、参数、返回值）
B. 不验证抛出异常
C. 不验证方法调用
D. 不模拟方法返回

答案：ABC

解析：不模拟方法返回，就无法进入某些分支，所以不利于提高单测覆盖率。

3. 如何单测虚基类

题目：在一次新需求开发时，小明遇到了虚基类单元测试编写问题。在跟周围的同事请教后，小明得到了 4 套解决方案，他一时无法分辨孰好孰坏。你帮他看看哪套方案最佳？

选项：

A. 找一个子类测试，连带测试了基类
B. 实现一个子类测试，然后基于子类测试基类
C. 先 spy 基类对象，再 mock 虚方法，最后测试基类所有方法
D. 以上方案都可取

答案：C

解析：针对基类要单独测试，尽量不采用联合测试。

4. 单元测试和集成测试的区别

题目：小白刚入职，分不清单元测试和集成测试。向师兄们请教后，发现他们对单元测试和集成测试的理解也不尽相同。请你帮小白分辨下，以下哪些观点是正确的？

选项：

A. 满足 AIR、FIRST 原则的测试用例都是单元测试
B. 单元测试是对软件设计最小单位的程序模块进行测试，集成测试是对这些程序模块组装成的系统模块进行测试
C. 单元测试和集成测试的主要区别就是有没有外部依赖
D. 单元测试和集成测试没有严格区分，只要满足 AIR、FIRST 原则，执行时间小于一定门限（比如 100ms）就可以认为是单元测试

答案：B

解析：

A. 集成测试也可能满足 AIR、FIRST 原则
B. 正确
C. 集成测试也可能没有外部依赖
D. 单元测试和集成测试概念不一样，虽然集成测试可以替换单元测试，但是不能混淆

5. 如何单测异常

题目：

小白在帮同事做 Code Review 时，发现同一个项目中存在 4 种异常验证方法：

1) 通过 try-catch 语句验证异常；
2) 通过@Test 注解验证异常；
3) 通过@Rule 注解验证异常；
4) 通过 Assert.assertThrows 方法验证异常。

关于这些异常验证方法，下面说法正确的是？

选项：

A. 方法 2 最简单，推荐大家使用

B. 方法 3 已废弃，不推荐大家使用
C. 方法 4 最完美，推荐大家使用
D. 方法 1/4 可以验证方法，而方法 2/3 无法验证方法

答案：BCD

解析：

A. 方法 2 无法验证方法
B. 正确
C. 正确
D. 正确

6. 单测的回归验证功能

题目：小明对原有代码进行了重构后，执行原有单元测试用例进行回归验证。针对单测的回归验证，下列说法是正确的是？

选项：

A. 新增的对象属性字段（保证不为 null），可以通过 Assert.assertEquals 等方法验证出来
B. 方法是否被调用，可以通过 Mockito.verify 方法验证出来
C. 多余的方法调用，可以通过 Mockito.verifyNoMoreInteractions 方法验证出来
D. 执行单测的回归验证完全通过，就表示重构的代码逻辑没有问题

答案：ABC

解析：执行单测出错，表示重构的代码逻辑肯定有问题；执行单测不出错，并不表示重构的代码逻辑没有问题。

7. 关于单测简化方法

题目：小明一直致力于单元测试的简化，希望利用最少的代码测试最多的代码。通过阅读相关单元测试资料，小明总结了以下单测简化方法。请问，下面单测简化方法中，哪些是有效的？

选项：

A. 对于简单方法（比如 Objects.equals），可以写在同一个测试方法中
B. 只写一个测试用例，通过条件判断是否 mock 或 verify 方法，利用参数化测试传入不同数据来测试所有代码分支流程
C. 利用测试用例特性简化测试用例，比如：不关心对象内容时，可以用虚拟对象代替真实数据
D. 利用工具类特性简化测试用例，比如：利用 JSON 反序列化简化对象构建语句，利用 JSON 序列化简化对象验证据
E. 为了减少单测代码量，可以不写对象和方法验证语句

答案：ACD

解析：

B.测试用例复杂且不可读
E.属于偷懒行为，写出来的是无效单测

8. 关于 mock 方法没生效

题目：杭州西溪园区的食堂里，小明跟老白边吃饭边聊起了今天写的单测， 小明提到 mock 的方法返回没有生效。老白根据自己多年的单元测试编写经验给出了下列建议。你认为正确的是？

选项：

A. 如果方法存在重载，可能 mock 方法时指定了错误的参数类型
B. 如果方法存在可变参数，可能 mock 方法时指定了错误的参数个数
C. 可能 mock 方法的参数值，匹配不上真实的调用参数值

D. 可能由于输入参数等问题，没有走到对应的代码分支

答案：ABCD

9. 关于单元测试流程

题目：小明在网上搜索了很多单元测试资料，发现编写单元测试都是有流程的，但是各种资料给出的单测测试流程又不尽相同。请问，你认为下面哪些说法是正确的？

选项：

A. 国外资料的“Given-When-Then”单测流程证
B. 小意给出的“定义对象-模拟方法-调用方法-验证方法”单测四步骤
C. 小杰总结的“模拟-调用-验证”单测三阶段
D. 以上方法只是划分标准不一致，本质上都是一样的

答案：ABCD

10.测试驱动开发（TDD）

题目：小明接手了离职同学的旧代码，发现这些代码都不容易进行单测。于是，小明进行了代码重构，使代码结构变得更清晰，深刻地体会到测试驱动开发（TDD）的好处。请问，下面哪些属于单测范畴的 TDD？

选项：

A. 利用类拆分，把一个类拆分成几个职责分明的类，使编写单元测试用例更简单
B. 利用方法提取，把一个大方法拆分成几个小方法，可以针对小方法进行单元测试
C. 提取 Lambda 表达式为一个独立方法，可以针对这个方法进行单元测试
D. 根据测试用例，先定义方法，再实现代码

答案：ABCD

11.关于测试覆盖率

题目：“单测小王子”小强的测试覆盖率非常高，经常给新人小明分享关于测试覆盖率的观点。请大家帮小明判断下小强的哪些观点是正确的？

选项：

A. 单测覆盖率指标包括：行覆盖、分支覆盖、条件覆盖、路径覆盖等
B. 如果做到单测分支覆盖 100%，那么单测行覆盖率也是 100%
C. 最完美的单元测试，需要做到路径覆盖率 100%
D. 如果一个项目的单测覆盖率达到 90%以上，就能说明这个项目的单测做得非常好

答案：ABC

解析：单测覆盖率高，并不代表单测做得好，关键在于单测是否有效。

12.关于方法验证

题目：老白帮小明 Code Review 时，总结出了小明的方法验证的几种写法。请问，哪些写法是有效的方法验证？

选项：

A. Mockito.verify(userDAO).delete(userId)
B. Mockito.verify(userDAO, Mockito.atLeastOnce()).delete(Mockito.anyLong());
C. Mockito.verify(userDAO).query(Mockito.any(UserQuery.class))
D. ArgumentCaptor<UserQuery> userQueryCaptor = ArgumentCaptor.forClass(UserQuery.class);
Mockito.verify(userDAO).query(userQueryCaptor.capture());
Assert.assertEquals("用户查询不一致", text, userQueryCaptor.getValue());

答案：AD

潇弈

这篇文章真可谓是雪中送炭，在学习了关于单元测试的种种理论知识之后，早都对于单元测试的实践早就急不可待了。优秀的理论，一定还需要优秀而趁手的工具其实践，而不论是对于测试驱动开发还是单元测试而言，该怎样能够符合标准的写出单元测试这一方法论是必不可少的。

作为个人而言，之前对单元测试的认识与实践还仅仅停留在Junit测试框架以及SpringBootTest组件的使用上，虽然已经能实现一定程度上的对于Bean容器的测试，但确实没有办法更为优雅的实现被测对象与其依赖对象的完全解耦。而这一问题间接导致了在进行单元测试的时候会同时将对依赖对象的测试也一同进行了，从而让使用者将单元测试与集成测试混淆，反而会产生对理论指导的疑惑。学无止境，这篇文章确实是获益匪浅。

悟鸣

读完这篇文章非常佩服作者，简直是保姆级单测教程，太详细了，日常写单测时某些场景不太会写在这篇文章里几乎都有。

编写单测每次需要 Mock 私有方法、Mock 静态方法等都需要去百度找示例，有了这个文档以后直接在这里找案例就可以了，点在哪。

之前在单元测试的图书中看到 Given - When -Then 的单测流程，和文章提到的 4 个流程异曲同工。

Given is the section that lays out the pre-conditions for the test.

The When clause performs the action being tested.

The Then statement checks that the post condition holds.

日常开发中最大的问题是很多同学并没有体会到单测的价值，加之项目工期比较紧张，而且很多项目对单元测试覆盖率的要求较高，会选择“面向覆盖率编程”。

另外，如果代码本身“不可测”，写单测就会非常痛苦，需要挖空心思去思考究竟如何去写单测。正如《代码重构：面向单元测试》讲到的一样，这种情况下其实需要对代码进行重构了。

必成

最近，在追求极致方面，我有了很深的体会。乔布斯曾经说过，你追求极致的人生，人生才会给你极致的体验。在一起我并不理解这句话，觉得事情做的差不多就行了，何必和自己死磕呢？这有啥好处？但是最近，我重新审视自己近几年的发展，发现和学生时代相比，自己就是少了这种“死磕”的精神，所以凡事都做到差不多就行了，工作量上也差不多，可是再也没有之前那种追求极致的精神，也就获得不了成为第一的快乐。长期来看，就是沦为一个平庸的人。极致和差一点，这是什么差距？就是第一名和第二名的差距，就是被别人记住和转瞬即忘的差距，就是激发全部潜力、追求卓越和凡事总差一线、庸庸碌碌的差距。我这里说的不是一定要和别人去做比较，而是问自己，是不是还可以做的更好？从自己做起，从身边的点点滴滴做起，是不是减脂健身，可以让自己做到在人群中脱颖而出？在工作中，是不是能够竭尽全力，给出最好的结果？在家人陪伴里，是不是抽出时间和心力，合舟共济？总有地方，我们可以追求极致。不单单是技术的极致，而是人生追求极致，因为极致的人生，才能有极致的体验。

虚愚

这个流程非常清晰，对具体执行非常有指导意义，无论是不是做java服务开发，都能够使用。虽然本文主要讲的是java的单元测试的编写流程，对于做c++开发的同学，使用的框架可能没法直接借鉴，但这个流程个人认为还是比较通用的，使用其他开发语言的同学其实都是可以使用的。以下是个人对这个流程的浅显理解：定义被测对象和依赖对象是基础，定义是否明确清晰，决定了后面的流程是否可能顺畅执行；模拟依赖对象和方法环节的重点关注点是是否可能容易得模拟出依赖对象，这直接决定了单元测试编写成本；第三步执行测试方法则需要重点关注的是是否可以容易的模拟出依赖的参数及验证；第四步验证方法则是重点要进行被测方法和依赖方法的验证，还是重点要关注的是成本。可能不同语言、不同框架的单元测试框架差异会比较大，实现流程也完全不一样，但其实目标是一样的，怎么样能使用单元测试来覆盖尽可能多的代码行和代码分支，并且进行高质量的断言，这样才能有效保证代码的质量。个人认为要保证一件事情有效的执行，指导流程非常关键。这个流程总结感觉对编写单元测试很有帮助。

问取

这篇文章对于新人来说相当于单元测试的一本词典了。属于在没有经验的阶段去做单元测试相关任务的时候，可以一条条对照着去做的一篇文章，真正让同学们做到了有章可循。

尤其是在前文的铺垫下，我们知道了编写单元测试应当遵循的框架、应当注意的事项，我们了解过单元测试的重要性与必要性后，在结合这篇文章进行单元测试的编写的时候，作者所提供的规范并且成体系的单元测试编写思路，能够很好的去帮助我们融会贯通（当然真正去编写单测的时候，还是需要再去回顾一遍，才能做到动手有所得）。

最后其实有一个比较困惑的点，不管是代码重构还是单元测试其实都是一个能够钻得很深的领域，但是他们归根结底还是为了让我们的代码整体更可靠更优秀，这些还是更适合在写代码的时候就考虑到，整体上效率才会更高。

寻求同道之人